

BỘ TÀI LIỆU HỌC BẢO VỆ ĐỒ ÁN CƠ SỞ

TRAVELMATE — WEBSITE ĐẶT PHÒNG VÀ GỢI Ý DU LỊCH

Bản Word dễ học: tóm tắt source, lộ trình học, câu hỏi có đáp án, demo script

Dành cho nhóm 3 sinh viên học chung trước ngày bảo vệ

Cách dùng tài liệu

Không học thuộc từng chữ. Với mỗi câu hỏi, đọc câu trả lời ngắn trước, sau đó mở đúng file/class được ghi bên dưới để tự giải thích lại bằng lời của mình. Cả 3 thành viên phải học phần Must know; mỗi người học sâu thêm phần mình demo.

Nguyên tắc khi bảo vệ

Trả lời trung thực: project đang ở mức đồ án/demo local, VNPAY Sandbox, một số tích hợp phụ thuộc cấu hình môi trường. Không nói quá thành production. Nếu có dùng công cụ AI hỗ trợ, hãy nói nhóm đã đọc lại source, kiểm thử lại và chịu trách nhiệm phần trình bày.

Mục lục nhanh

- Phần 0 — Chốt đối chiếu source hiện tại.
- Phần 1 — Tóm tắt project cho người mới.
- Phần 2 — Bản đồ source code.
- Phần 3 — Lộ trình học từ HTML/CSS lên Spring Boot MVC.
- Phần 4 — Quy trình nghiệp vụ lỗi.
- Phần 5 — Bộ câu hỏi có đáp án chi tiết.
- Phần 6 — Bộ câu hỏi nhanh có đáp án một dòng.
- Phần 7 — Phân chia học và demo cho nhóm 3 người.
- Phần 8 — Kịch bản demo bảo vệ.
- Phần 9 — Checklist trước ngày bảo vệ.
- Phần 10 — Những câu trả lời nên tránh.
- Phần 11 — Bài kiểm tra mô phỏng.
- Phần C — Câu hỏi bảo vệ Q001-Q030: Tổng quan, kiến trúc, database và Use Case.
- Phần D — Payment/VNPAY và Partner workflow (TC031-TC060).
- Phần E — Kịch bản demo nên trình bày trước giảng viên.
- Phần F — Câu hỏi bảo vệ Q031-Q056: Use Case, Booking, VNPAY và cọc 30%.
- Phần G — Câu hỏi bảo vệ Q057-Q083: Voucher, Availability, Partner và Admin workflow.
- Phần H — Câu hỏi bảo vệ Q084-Q110: Admin, settlement/wallet, security và transaction.
- Phần I — Câu hỏi bảo vệ Q111-Q136: Gateway/DB lỗi, testing, demo/deployment và hạn chế production.
- Phần J — Câu hỏi bảo vệ Q137-Q160: Mở rộng, so sánh thực tế và hard defense.
- Phần 12 — Cập nhật trạng thái sau rà soát.
- Phần 13 — Cập nhật sau fix commission, voucher và UI/UX.
- Phần 14 — Rà soát checklist bad-ending và fix tiếp.
- Phần 15 — Cập nhật luồng đặt lại đơn đã hủy/no-show.
- Phần 18 — Cập nhật ngày 04/06/2026: TravelMate tự giữ phòng và Admin xem trước trang user.

Phần 0 — Chốt đối chiếu source hiện tại

Mục này được bổ sung sau khi kiểm tra trực tiếp project trong C:\TravelMate, gồm pom.xml, src/main/java, src/test/java, README_DEMO.md, docs/07_Test_Report.md, docs/08_Demo_Script.md và travelmate_db.sql gốc. Dùng phần này để tránh học nhầm số liệu hoặc luồng cũ.

- Công nghệ hiện tại: Java 21, Spring Boot 3.5.14, Spring MVC, Thymeleaf, MySQL, Spring Security, OAuth2 Google optional, Spring Mail SMTP optional, VNPAY Sandbox, Apache POI Excel và chatbot rule-based + Groq optional. Google OAuth chỉ dành cho USER; Admin/Partner vẫn đăng nhập bằng tài khoản hệ thống. Email quên mật khẩu chỉ gửi thật khi SMTP đã được cấu hình; email xác nhận booking sau khi VNPAY success cũng là luồng SMTP riêng và lỗi gửi mail không rollback booking.
- Luồng booking online chuẩn: tạo Booking/Payment ở PENDING_PAYMENT, chuyển sang VNPAY Sandbox, VNPAY ghi nhận thành công thì Payment APPROVED, Booking CONFIRMED và partner_status = PARTNER_CONFIRMED. PENDING_ADMIN_APPROVAL chỉ nên nói là legacy/ngoại lệ đối soát thủ công.
- Test hiện tại sau bản fix tài chính/voucher/UI: 279 test pass, 0 failure, 0 error, 0 skipped. Khi bảo vệ vẫn nên chạy lại trên máy demo trước khi nói con số này.
- Seed chính travelmate_db.sql có tài khoản demo cơ bản: admin@travelmate.vn, user@travelmate.vn, user2@travelmate.vn, user3@travelmate.vn, partner@travelmate.vn, partner2@travelmate.vn, partner3@travelmate.vn, partner4@travelmate.vn. File này đã có nhiều kịch bản booking hơn bản cũ, nên không học thuộc số 32/35 booking.
- Dữ liệu ví/quyết toán/rút tiền đã được đưa vào travelmate_db.sql gốc: có partner wallet, settlement PAID/PENDING, withdrawal PENDING/PAID/REJECTED và transaction history để demo trực quan trên máy khác. Không cần phụ thuộc file SQL bổ sung khi trình bày nhóm nghiệp vụ này.
- Role của user là enum lồng trong User.java (User.Role). Các enum trạng thái/loại nghiệp vụ khác nằm trong entity/enums như BookingStatus, PaymentStatus, PaymentOption, PropertyType, VoucherScope, BookingSource.
- Tài liệu này đã được cập nhật theo source hiện tại: giữ core service ổn định, chuẩn hóa seed SQL, nhấn hiển thị trạng thái, review 1-10, email sau VNPAY success, đặt lại đơn hủy/no-show, settlement/voucher, Admin preview bar và phân học để nhóm trả lời đúng nghiệp vụ khi bảo vệ.

Phần 1 — Tóm tắt project cho người mới

TravelMate là một website đặt phòng/lưu trú du lịch được xây dựng bằng Java Spring Boot MVC, Thymeleaf và MySQL. Hệ thống mô phỏng nền tảng trung gian giữa khách hàng, đối tác lưu trú và quản trị viên. Khách hàng tìm kiếm nơi lưu trú, đặt phòng, thanh toán qua VNPAY Sandbox và theo dõi booking. Đối tác quản lý nơi lưu trú/phòng, tiếp nhận booking đã được TravelMate giữ, check-in/check-out và theo dõi quyết toán. Quản trị viên kiểm duyệt dữ liệu, quản lý booking, voucher, review, support và doanh thu/quyết toán.

Project này không chỉ là HTML/CSS vì giao diện chỉ là phần hiển thị. Phía sau còn có database, đăng nhập, phân quyền, xử lý nghiệp vụ booking, kiểm tra phòng trống, thanh toán, voucher, settlement, chatbot và test tự động. Khi bảo vệ, cần giải thích được luồng từ màn hình người dùng đến Controller, Service, Repository và Database.

- Kiến trúc: monolithic server-side rendering, Spring Boot MVC + layered architecture.
- Layer chính: Controller nhận request, Service xử lý nghiệp vụ, Repository truy vấn MySQL, Entity ánh xạ bảng, Thymeleaf render HTML.
- Phạm vi thật: demo/local/sandbox; không nói là hệ thống production hoàn chỉnh.
- Câu trả lời trung thực về AI: có dùng công cụ hỗ trợ nhưng nhóm đã đọc lại source, kiểm thử lại và hiểu các luồng chính.

Phần 2 — Bản đồ source code

Khu vực source code	Vai trò	File/class tiêu biểu	Người mới cần hiểu gì?	Câu hỏi giảng viên có thể hỏi
Controller	Nhận request, trả view hoặc JSON	HomeController, BookingPageController, PaymentController, PartnerPageController, AdminPageController, ChatbotApiController	Biết route nào do controller nào xử lý; controller không nên chứa quá nhiều nghiệp vụ.	Khi bấm Đặt phòng request đi vào controller nào?
Service	Xử lý nghiệp vụ chính	BookingService, PaymentService, VnpayService, AvailabilityService, VoucherService, SettlementService, PartnerWalletService, ChatbotService	Đây là nơi cần học sâu nhất: tính tiền, kiểm tra phòng, thanh toán, voucher, quyết toán.	Vì sao không đặt logic tính tiền trong Controller?
Repository	Truy vấn database bằng Spring Data JPA	BookingRepository, RoomRepository, UserRepository, VoucherRepository, PartnerSettlementRepository	Hiểu repository là cầu nối giữa Service và MySQL.	Repository khác Service ở điểm nào?
Entity	Ảnh xạ bảng MySQL thành class Java	User, Accommodation, Room, Booking, Payment, Voucher, Review, PartnerSettlement, PartnerWallet	Nắm quan hệ User-Booking-Room-Payment-Review.	Vì sao Entity không nên chứa quá nhiều logic nghiệp vụ?
Security	Đăng nhập, phân quyền, OAuth2	SecurityConfig, CustomAuthSuccessHandler, OAuth2LoginSuccessHandler, UserDetailsServiceImpl	Nắm /admin/**, /partner/**, /booking/**; BCrypt; Google Login chỉ USER.	Route admin được bảo vệ ở đâu?
Templates	Giao diện Thymeleaf	templates/user, templates/admin, templates/partner, templates/auth, fragments	Biết Thymeleaf nhận model từ Controller và render HTML.	th:each, th:if, th:href dùng làm gì?
Static assets	CSS, JS, ảnh	static/assets/css, static/assets/js, static/assets/images	CSS/JS hỗ trợ UI, modal, chatbot, voucher, filter; nghiệp vụ chính vẫn ở backend.	Frontend gửi dữ liệu booking như thế nào?
Config	Cấu hình app	application.properties, VnpayConfigProperties, InternationalizationConfig	Biết DB local, VNPAY sandbox, SMTP, Google OAuth, ddl-auto=update, CSRF demo.	Cấu hình nào không nên dùng y như vậy khi production?
Scheduler	Tác vụ tự động	BookingExpiryScheduler	Học ý tưởng hết hạn booking nếu user không thanh toán.	Nếu user tạo booking nhưng không trả tiền thì sao?
Tests	Kiểm thử tự động	src/test/java/com/travelmate	Có test cho booking, voucher, payment, password reset, settlement, template.	Không được nói số test pass nếu chưa chạy lại thật.
Docs/SQL	Tài liệu và seed demo	README_DEMO.md, docs/*.md, travelmate_db.sql	Dùng để học demo account, yêu cầu nghiệp vụ, test report.	Database demo có những bảng chính nào?

Phần 3 — Lộ trình học từ HTML/CSS lên project này

Thứ tự học	Mục tiêu	Kiến thức cần nắm	File nên đọc	Dấu hiệu đã hiểu
1. HTML/CSS/Thymeleaf	Hiểu giao diện được render như thế nào.	HTML form, th:text, th:each, th:if, fragments, static assets.	templates/user/*.html, templates/auth/*.html, fragments/*.html	Giải thích được dữ liệu từ Controller ra trang HTML.
2. Java căn bản	Đọc được class, method, enum, list, optional.	class, method, constructor, annotation, exception, BigDecimal, LocalDate.	entity/*.java, service/*.java	Đọc được BookingService ở mức luồng xử lý chính.
3. OOP	Hiểu object trong project.	Entity, DTO, service object, enum trạng thái.	User, Booking, Payment, Voucher, Room	Nói được Booking chứa dữ liệu gì và liên kết với ai.
4. HTTP request/response	Hiểu browser gửi request tới Spring.	GET/POST, form submit, redirect, session, model attribute.	controller/page/*.java	Nói được khi bấm Đặt phòng thì request đi đâu.

5. MVC	Hiểu Model-View-Controller trong TravelMate.	Controller nhận request, Service xử lý, Repository lấy DB, Thymeleaf hiển thị.	HomeController, BookingPageController, templates	Về được luồng request từ UI đến DB và quay lại.
6. Spring Boot	Biết vì sao dùng Spring Boot.	starter dependency, embedded Tomcat, application.properties, Maven.	pom.xml, TravelmateApplication.java	Chạy được app và giải thích port 8080.
7. Controller-Service-Repository	Nắm kiến trúc phân tầng.	Controller không làm nghiệp vụ nặng; Service xử lý; Repository truy vấn.	BookingService, BookingRepository, BookingPageController	Chỉ ra được file nào xử lý nghiệp vụ booking.
8. Entity/JPA/Database	Hiểu bảng và quan hệ.	@Entity, @ManyToOne, @OneToMany, repository, SQL seed.	entity/*.java, travelmate_db.sql	Giải thích được User-Booking-Payment-Review.
9. Form/Validation	Biết kiểm tra dữ liệu nhập.	Ngày check-in/out, số lượng phòng, sức chứa, voucher.	BookingService, AvailabilityService, VoucherService	Giải thích được vì sao phải validate ở backend.
10. Security	Nắm đăng nhập và phân quyền.	BCrypt, UserDetailsService, role, route protection, OAuth2.	SecurityConfig, UserDetailsServiceImpl	Trả lời được USER/PARTNER/ADMIN khác nhau thế nào.
11. Business logic	Nắm booking, payment, voucher, settlement.	PENDING_PAYMENT, CONFIRMED, DEPOSIT_30, FULL_PAYMENT, commission.	BookingService, PaymentService, SettlementService	Tự nói được flow booking từ đầu đến cuối.
12. Test & bảo vệ	Biết demo, test và giới hạn project.	JUnit, Mockito, MockMvc, sandbox, local, production risks.	src/test/java, docs/07_Test_Report.md	Trả lời trung thực về điểm yếu và hướng phát triển.

Phần 4 — Quy trình nghiệp vụ lõi cần nói được

Luồng	Tóm tắt cần thuộc	File/class cần mở
Booking + VNPAY	User chọn phòng/ngày/số lượng -> hệ thống kiểm tra dữ liệu và availability -> tạo Booking/Payment PENDING_PAYMENT -> chuyển VNPAY Sandbox -> return/ipn -> Payment APPROVED, Booking CONFIRMED nếu hợp lệ.	BookingService, PaymentService, VnpayService, PaymentController
Cọc 30%	DEPOSIT_30 thu 30% online qua VNPAY; 70% còn lại trả tại cơ sở. Quyết toán TravelMate chỉ tính phần online mà hệ thống thu.	PaymentOption, BookingService, SettlementService
Partner xử lý booking	Partner xem booking cơ sở mình -> tiếp nhận đơn đã được TravelMate giữ -> check-in -> check-out -> booking COMPLETED; no-show nếu khách không đến theo nghiệp vụ.	PartnerPageController, BookingService, PartnerBookingStatus
Voucher	User nhập voucher -> hệ thống kiểm tra scope/điều kiện -> tính giảm giá -> xác định cost bearer ADMIN/PARTNER cho settlement.	VoucherService, Voucher, VoucherCostBearer
Settlement/wallet	Admin tạo kỳ quyết toán -> hệ thống tổng hợp booking đủ điều kiện -> tính commission/voucher -> ghi settlement/wallet nội bộ -> withdrawal request nếu partner yêu cầu.	SettlementService, PartnerWalletService, PartnerSettlement, PartnerWallet
Review	Sau khi booking hoàn tất, User mới được đánh giá; review liên kết booking/user/accommodation; admin có thể quản lý/ẩn nếu không phù hợp.	ReviewService, Review, templates/user/mybooking.html
Security	Form login -> UserDetailsService load user -> BCrypt so sánh password -> CustomAuthSuccessHandler redirect theo role; route /admin/** và /partner/** được bảo vệ.	SecurityConfig, UserDetailsServiceImpl, CustomAuthSuccessHandler

Phần 5 — Bộ câu hỏi có đáp án chi tiết

Phần này dùng để học sâu. Mỗi câu gồm đáp án ngắn, giải thích cho người mới, file cần mở và lỗi trả lời thường gặp.

Tổng quan project

Câu 01 [Dễ] TravelMate là gì?

Đáp án ngắn: TravelMate là website đặt phòng/lưu trú du lịch, hỗ trợ khách hàng tìm nơi ở, đặt phòng, thanh toán sandbox và theo dõi booking; đồng thời có khu vực Partner và Admin.

Giải thích cho người mới: Nói đơn giản: đây không chỉ là trang giới thiệu khách sạn. Hệ thống có database, phân quyền, nghiệp vụ booking, payment, partner xử lý lưu trú và admin quản trị.

File/class nên mở: README_DEMO.md; docs/01_SRS.md; HomePageController; BookingService

Lỗi trả lời thường gặp: Đừng trả lời “web đặt phòng giống Traveloka” mà không nói rõ phạm vi demo/sandbox.

Câu 02 [Dễ] Project giải quyết vấn đề gì?

Đáp án ngắn: Giúp người dùng tìm kiếm nơi lưu trú, đặt phòng trực tuyến và tham khảo gợi ý du lịch; giúp Partner quản lý cơ sở/phòng/booking; giúp Admin kiểm duyệt và quản lý hệ thống.

Giải thích cho người mới: Ba bên trong hệ thống có nhu cầu khác nhau. User cần đặt phòng; Partner cần bán/quản lý phòng; Admin cần kiểm soát dữ liệu và vận hành.

File/class nên mở: docs/01_SRS.md; templates/user; templates/partner; templates/admin

Lỗi trả lời thường gặp: Đừng chỉ nói “giúp du lịch tiện hơn”, vì quá chung chung.

Câu 03 [Dễ] Project có những actor chính nào?

Đáp án ngắn: Có Guest, User/Customer, Partner và Admin. Nếu báo cáo rút gọn thì nhấn mạnh 3 actor chính: User, Partner, Admin.

Giải thích cho người mới: Guest xem trang public. User đặt phòng và thanh toán. Partner quản lý nơi lưu trú và booking của cơ sở. Admin quản lý, duyệt và theo dõi toàn hệ thống.

File/class nên mở: docs/01_SRS.md; SecurityConfig.java

Lỗi trả lời thường gặp: Đừng lẫn Partner với Admin; Partner không được xem dữ liệu của đối tác khác.

Câu 04 [Dễ] Vì sao project này không chỉ là HTML/CSS?

Đáp án ngắn: Vì ngoài giao diện, hệ thống còn có backend Java Spring Boot, database MySQL, đăng nhập, phân quyền, booking, payment, JPA, service và test.

Giải thích cho người mới: HTML/CSS chỉ là phần hiển thị. Khi user đặt phòng, dữ liệu phải được validate, lưu vào DB, tạo payment và cập nhật trạng thái.

File/class nên mở: pom.xml; service/*.java; repository/*.java; entity/*.java

Lỗi trả lời thường gặp: Đừng nói “em chỉ làm giao diện”.

Câu 05 [Trung bình] Vì sao dùng Spring Boot MVC + Thymeleaf + MySQL?

Đáp án ngắn: Vì phù hợp demo đồ án: một project chạy local, Controller trả Thymeleaf view, MySQL lưu dữ liệu quan hệ, dễ phân quyền và kiểm thử.

Giải thích cho người mới: Cách này đơn giản hơn tách frontend/backend riêng, nhưng vẫn đủ kiến trúc phần mềm: Controller-Service-Repository-Entity.

File/class nên mở: pom.xml; application.properties; templates

Lỗi trả lời thường gặp: Đừng nói “vì AI chọn” hoặc “vì framework tự làm”.

Câu 06 [Trung bình] Kiến trúc tổng thể của TravelMate là gì?

Đáp án ngắn: TravelMate là ứng dụng monolithic server-side rendering theo Spring Boot MVC và layered architecture.

Giải thích cho người mới: Monolithic nghĩa là một ứng dụng chính chứa backend và view. Layered nghĩa là tách controller, service, repository, entity.

File/class nên mở: TravelmateApplication.java; controller/page; service; repository; entity

Lỗi trả lời thường gặp: Đừng gọi là SPA hoặc microservice.

Câu 07 [Gài bẫy] Nếu thầy hỏi em dùng AI/vibe coding thì trả lời sao?

Đáp án ngắn: Nhóm em có dùng công cụ hỗ trợ, nhưng đã đọc lại source, kiểm thử lại luồng chính và chịu trách nhiệm giải thích phần mình trình bày.

Giải thích cho người mới: Không nên phủ nhận nếu có dùng công cụ. Tập trung vào việc hiểu lại: mở code, giải thích file, chạy demo, test và biết hạn chế.

File/class nên mở: README_DEMO.md; docs/18_Project_Analysis_For_Report_And_Defense.md

Lỗi trả lời thường gặp: Tránh nói “AI làm nên em không biết”.

Câu 08 [Khó] Nếu chỉ có 1 phút giới thiệu TravelMate, em nói gì?

Đáp án ngắn: TravelMate là website đặt phòng du lịch dùng Spring Boot MVC, Thymeleaf và MySQL. Hệ thống có User đặt phòng/thanh toán, Partner quản lý cơ sở và booking, Admin kiểm duyệt/quản lý/settlement. Thanh toán dùng VNPAY Sandbox, có voucher, review, chatbot, forgot password và Google Login ở mức demo.

Giải thích cho người mới: Câu trả lời cần đủ: mục tiêu, actor, công nghệ, nghiệp vụ, phạm vi demo. Không cần kể hết file.

File/class nên mở: README_DEMO.md; docs/01_SRS.md

Lỗi trả lời thường gặp: Đừng nói quá như “hệ thống thương mại hoàn chỉnh”.

Source/MVC

Câu 09 [Dễ] Controller làm gì trong project?

Đáp án ngắn: Controller nhận request từ trình duyệt, gọi Service xử lý và trả về view Thymeleaf hoặc JSON.

Giải thích cho người mới: Ví dụ BookingPageController nhận request liên quan đặt phòng, PaymentController xử lý return/ipn VNPAY, AdminPageController trả trang admin.

File/class nên mở: controller/page/*.java; controller/api/*.java

Lỗi trả lời thường gặp: Đừng nói Controller trực tiếp xử lý toàn bộ nghiệp vụ.

Câu 10 [Dễ] Service làm gì?

Đáp án ngắn: Service chứa nghiệp vụ chính như tính tiền booking, kiểm tra phòng trống, xử lý thanh toán, voucher, review, settlement.

Giải thích cho người mới: Đây là lớp quan trọng nhất khi bảo vệ backend. Giảng viên thường hỏi “logic nằm ở đâu?”.

File/class nên mở: BookingService, PaymentService, AvailabilityService, VoucherService, SettlementService

Lỗi trả lời thường gặp: Đừng trả lời “Service là gọi database”. Đó là Repository.

Câu 11 [Dễ] Repository làm gì?

Đáp án ngắn: Repository truy vấn và cập nhật dữ liệu trong MySQL thông qua Spring Data JPA.

Giải thích cho người mới: Repository thường kế thừa JpaRepository. Service gọi Repository để lấy User, Booking, Room, Payment...

File/class nên mở: repository/*.java

Lỗi trả lời thường gặp: Đừng viết SQL lung tung nếu source dùng JPA query methods.

Câu 12 [Trung bình] Entity là gì?

Đáp án ngắn: Entity là class Java ánh xạ với bảng trong database, ví dụ User, Booking, Room, Payment.

Giải thích cho người mới: Mỗi Entity thường có id, thuộc tính và quan hệ như ManyToOne/OneToMany. JPA/Hibernate dùng Entity để lưu/đọc DB.

File/class nên mở: entity/*.java

Lỗi trả lời thường gặp: Đừng nhầm Entity với DTO hoặc Controller.

Câu 13 [Trung bình] Luồng request từ giao diện đến database diễn ra như thế nào?

Đáp án ngắn: Browser gửi request -> Controller nhận -> Service xử lý -> Repository truy vấn DB -> Entity nhận dữ liệu -> Controller đưa Model sang Thymeleaf -> trả HTML.

Giải thích cho người mới: Đây là cấu trúc MVC. Nên vẽ được mũi tên trên bảng khi bảo vệ.

File/class nên mở: HomeController, BookingPageController, BookingService, BookingRepository, templates/user/*.html

Lỗi trả lời thường gặp: Đừng bỏ qua Service trong sơ đồ.

Câu 14 [Khó] Vì sao không nên để logic nghiệp vụ trong Controller?

Đáp án ngắn: Vì Controller sẽ bị phình to, khó test và khó bảo trì. Logic nên đặt trong Service để tái sử dụng và kiểm thử được.

Giải thích cho người mới: Ví dụ tính tổng tiền booking, validate voucher, cập nhật payment nên ở Service, không trộn vào Controller.

File/class nên mở: BookingService, VoucherService, PaymentService

Lỗi trả lời thường gặp: Đừng trả lời “để code đẹp hơn” quá chung.

Câu 15 [Trung bình] Thymeleaf lấy dữ liệu từ đâu?

Đáp án ngắn: Controller đưa dữ liệu vào Model, Thymeleaf dùng các biểu thức như th:text, th:each, th:if để render ra HTML.

Giải thích cho người mới: Ví dụ danh sách khách sạn, booking, voucher, dashboard đều được controller chuẩn bị rồi template hiển thị.

File/class nên mở: templates/user/index.html; templates/user/hotels.html; templates/admin/dashboard.html

Lỗi trả lời thường gặp: Đừng nói Thymeleaf tự gọi database.

Câu 16 [Khó] Project này là REST API backend hay server-rendered web?

Đáp án ngắn: Chủ yếu là server-rendered web bằng Spring MVC + Thymeleaf; có một số API nhỏ cho chatbot, notification, voucher và travel post.

Giải thích cho người mới: Không gọi là SPA. Người dùng nhận HTML render từ server, không phải React/Vue frontend tách biệt.

File/class nên mở: controller/page; controller/api; templates

Lỗi trả lời thường gặp: Đừng nói frontend/backend tách riêng nếu project chạy một Spring Boot app.

Java/OOP

Câu 17 [Dễ] Enum dùng để làm gì trong project?

Đáp án ngắn: Enum dùng để giới hạn các trạng thái/loại cố định như BookingStatus, PaymentStatus, PaymentOption, User.Role, PropertyType, VoucherScope và BookingSource.

Giải thích cho người mới: Dùng enum giúp tránh dùng string tự do, giảm lỗi chính tả trạng thái. Riêng role USER/ADMIN/PARTNER là enum lồng trong User.java, không nằm trong thư mục entity/enums.

File/class nên mở: entity/enums/*.java; User.java

Lỗi trả lời thường gặp: Đừng sửa tên enum tùy tiện vì DB/code/template có thể phụ thuộc.

Câu 18 [Trung bình] BigDecimal thường dùng ở đâu và vì sao?

Đáp án ngắn: BigDecimal dùng cho tiền để tránh sai số số thực.

Giải thích cho người mới: Trong booking/payment/voucher/settlement, tính tiền bằng double có thể lệch do lỗi làm tròn.

File/class nên mở: BookingService, Payment, Voucher, PartnerSettlement

Lỗi trả lời thường gặp: Đừng nói dùng int/double cho tiền là đủ.

Câu 19 [Trung bình] Annotation @Entity, @Service, @Controller có ý nghĩa gì?

Đáp án ngắn: @Entity đánh dấu class mapping DB; @Service đánh dấu lớp nghiệp vụ; @Controller đánh dấu lớp xử lý request trả view.

Giải thích cho người mới: Spring quét các annotation để tạo bean và quản lý vòng đời class.

File/class nên mở: entity/*.java; service/*.java; controller/page/*.java

Lỗi trả lời thường gặp: Đừng nói annotation chỉ để trang trí.

Câu 20 [Khó] Vì sao project dùng DTO ở một số chỗ?

Đáp án ngắn: DTO dùng để gom dữ liệu trả ra giao diện hoặc báo cáo mà không cần phơi toàn bộ Entity.

Giải thích cho người mới: Ví dụ revenue summary, room availability, settlement detail có thể không tương ứng đúng một bảng DB.

File/class nên mở: dto/*.java; RevenueService; AvailabilityService

Lỗi trả lời thường gặp: Đừng nhầm DTO với bảng DB.

Database/JPA

Câu 21 [Dễ] Các bảng/entity chính của TravelMate là gì?

Đáp án ngắn: users, accommodations, rooms, bookings, payments, vouchers, reviews; thêm settlement/wallet/support/travel post cho module mở rộng.

Giải thích cho người mới: Nhóm phải nắm ít nhất 7 bảng chính vì đó là xương sống nghiệp vụ đặt phòng.

File/class nên mở: entity/User.java; Accommodation.java; Room.java; Booking.java; Payment.java; Voucher.java; Review.java

Lỗi trả lời thường gặp: Đừng chỉ kể tên mà không nói quan hệ.

Câu 22 [Dễ] User liên kết với Booking như thế nào?

Đáp án ngắn: Một User có thể có nhiều Booking; mỗi Booking thuộc về một User.

Giải thích cho người mới: Trong flow thực tế, User đăng nhập rồi tạo Booking; booking lưu user_id để biết đơn này của ai.

File/class nên mở: User.java; Booking.java; BookingRepository.java

Lỗi trả lời thường gặp: Đừng để User xem booking của người khác.

Câu 23 [Trung bình] Accommodation và Room liên kết ra sao?

Đáp án ngắn: Một Accommodation có nhiều Room; mỗi Room thuộc về một Accommodation.

Giải thích cho người mới: Accommodation là cơ sở lưu trú, Room là phòng/căn cụ thể để khách đặt.

File/class nên mở: Accommodation.java; Room.java; RoomRepository.java

Lỗi trả lời thường gặp: Đừng gọi room là khách sạn.

Câu 24 [Trung bình] Payment liên kết với Booking như thế nào?

Đáp án ngắn: Payment gắn với Booking để lưu thông tin giao dịch như phương thức, option, số tiền, trạng thái, thời điểm thanh toán.

Giải thích cho người mới: Booking là đơn đặt phòng; Payment là chứng từ/ghi nhận thanh toán của booking.

File/class nên mở: Payment.java; Booking.java; PaymentService.java

Lỗi trả lời thường gặp: Đừng nhập chung booking_status và payment_status.

Câu 25 [Trung bình] Tại sao khóa chính trong bảng nhiều nơi chỉ tên là id?

Đáp án ngắn: Không sai. Với JPA, dùng id làm khóa chính trong từng bảng là phổ biến; khóa ngoại mới cần tên rõ như user_id, room_id, booking_id.

Giải thích cho người mới: Điều quan trọng là quan hệ phải rõ trong Entity và column mapping, không nhất thiết phải đặt user_id làm khóa chính trong bảng users.

File/class nên mở: entity/*.java; travelmate_db.sql

Lỗi trả lời thường gặp: Đừng vội sửa toàn bộ DB chỉ vì học ví dụ user_id.

Câu 26 [Khó] ddl-auto=update có rủi ro gì?

Đáp án ngắn: Phù hợp dev/demo nhưng không nên dùng production vì Hibernate tự cập nhật schema có thể gây lệch/mất kiểm soát migration.

Giải thích cho người mới: Khi production nên dùng validate hoặc migration tool như Flyway/Liquibase.

File/class nên mở: application.properties

Lỗi trả lời thường gặp: Đừng nói update là tốt nhất mọi môi trường.

Câu 27 [Khó] Vì sao cần seed data travelmate_db.sql?

Đáp án ngắn: Để có dữ liệu mẫu cho demo: user/admin/partner, accommodations, rooms, bookings, payments, vouchers, settlement, review...

Giải thích cho người mới: Không có seed, demo sẽ thiếu tài khoản và dữ liệu để trình bày luồng.

File/class nên mở: src/main/resources/travelmate_db.sql; README_DEMO.md

Lỗi trả lời thường gặp: Đừng nói database tự sinh đầy đủ dữ liệu.

Câu 28 [Gài bẫy] Nếu xóa database thì project còn chạy đủ chức năng không?

Đáp án ngắn: Không. App có thể khởi động nếu cấu hình đúng, nhưng các chức năng cần dữ liệu như login, booking, room, voucher, settlement sẽ không demo được nếu thiếu seed.

Giải thích cho người mới: Database là phần bắt buộc của hệ thống, không phải dữ liệu phụ.

File/class nên mở: application.properties; travelmate_db.sql

Lỗi trả lời thường gặp: Đừng nói “chạy được vì code có sẵn”.

Security/Auth

Câu 29 [Dễ] Spring Security dùng làm gì?

Đáp án ngắn: Dùng để xác thực đăng nhập, phân quyền route theo role và mã hóa mật khẩu bằng BCrypt.

Giải thích cho người mới: TravelMate có USER/PARTNER/ADMIN nên cần kiểm soát ai được vào /admin, /partner, /booking.

File/class nên mở: SecurityConfig.java; UserDetailsServiceImpl.java

Lỗi trả lời thường gặp: Đừng chỉ nói “để bảo mật”.

Câu 30 [Dễ] Mật khẩu lưu trong database như thế nào?

Đáp án ngắn: Mật khẩu được mã hóa bằng BCrypt, không lưu plain text.

Giải thích cho người mới: BCrypt có salt và hash, không thể so sánh trực tiếp string password gốc.

File/class nên mở: SecurityConfig.java; UserService.java; UserDetailsServiceImpl.java

Lỗi trả lời thường gặp: Đừng nói mật khẩu được mã hóa MD5 nếu source dùng BCrypt.

Câu 31 [Trung bình] Route /admin/** được bảo vệ như thế nào?

Đáp án ngắn: Trong SecurityConfig, /admin/** yêu cầu hasRole("ADMIN").

Giải thích cho người mới: Nếu User thường cố vào /admin, Spring Security sẽ chặn hoặc chuyển hướng.

File/class nên mở: SecurityConfig.java

Lỗi trả lời thường gặp: Đừng nói kiểm tra bằng JavaScript phía client.

Câu 32 [Trung bình] Route /partner/** dành cho ai?

Đáp án ngắn: Dành cho tài khoản role PARTNER.

Giải thích cho người mới: Partner chỉ thao tác dữ liệu của cơ sở mình, còn kiểm tra owner/permission chi tiết nằm trong service/controller.

File/class nên mở: SecurityConfig.java; PartnerPageController.java

Lỗi trả lời thường gặp: Đừng nói Admin và Partner dùng chung khu quản trị.

Câu 33 [Trung bình] Google Login trong TravelMate dùng cho ai?

Đáp án ngắn: Google Login chủ yếu dành cho USER. Nếu email chưa tồn tại thì tạo USER; không dùng cho ADMIN/PARTNER để tránh rủi ro phân quyền.

Giải thích cho người mới: Admin/Partner có quyền nhạy cảm nên dùng tài khoản nội bộ kiểm soát chặt hơn.

File/class nên mở: OAuth2LoginSuccessHandler.java; GoogleOAuthClientConfig.java

Lỗi trả lời thường gặp: Đừng nói Google login dùng cho mọi role.

Câu 34 [Khó] CSRF đang xử lý thế nào và rủi ro là gì?

Đáp án ngắn: Trong SecurityConfig, CSRF đang tắt để dễ test/demo. Production cần bật lại và thêm CSRF token cho form/API thay đổi dữ liệu.

Giải thích cho người mới: Tắt CSRF giúp demo nhanh nhưng là điểm yếu nếu deploy thật.

File/class nên mở: SecurityConfig.java

Lỗi trả lời thường gặp: Đừng nói tắt CSRF là không có vấn đề.

Câu 35 [Trung bình] Quên mật khẩu hoạt động thế nào?

Đáp án ngắn: User nhập email, hệ thống tạo token có hạn dùng, gửi link qua SMTP nếu cấu hình, sau đó đổi mật khẩu mới đã mã hóa BCrypt.

Giải thích cho người mới: Token phải hết hạn và dùng một lần để tránh bị lợi dụng.

File/class nên mở: PasswordResetService.java; EmailService.java; PasswordResetToken.java

Lỗi trả lời thường gặp: Đừng nói hệ thống gửi mật khẩu cũ qua mail.

Câu 36 [Gai bẫy] Nếu SMTP chưa cấu hình thì quên mật khẩu có gửi mail thật không?

Đáp án ngắn: Không chắc. Hệ thống có logic gửi mail nếu cấu hình SMTP đầy đủ; môi trường demo có thể không gửi mail thật tùy biến môi trường. Mail xác nhận booking sau VNPAY success cũng là luồng SMTP riêng và lỗi gửi mail không rollback booking.

Giải thích cho người mới: Khi bảo vệ phải nói “nếu SMTP được cấu hình” để không phóng đại.

File/class nên mở: application.properties; EmailService.java; PasswordResetService.java

Lỗi trả lời thường gặp: Đừng khẳng định gửi mail thật nếu chưa test SMTP trên máy demo.

Booking/Payment

Câu 37 [Dễ] Luồng booking tổng quát diễn ra thế nào?

Đáp án ngắn: User chọn phòng/ngày/số lượng -> hệ thống kiểm tra hợp lệ và phòng trống -> tạo booking/payment PENDING_PAYMENT -> chuyển VNPAY -> thanh toán thành công thì payment APPROVED, booking CONFIRMED.

Giải thích cho người mới: Đây là luồng quan trọng nhất. Cần nói đúng các trạng thái và ai xử lý tiếp theo.

File/class nên mở: BookingPageController.java; BookingService.java; PaymentService.java; VnpayService.java

Lỗi trả lời thường gặp: Đừng bỏ qua bước kiểm tra phòng trống/voucher.

Câu 38 [Dễ] Trạng thái ban đầu của booking online là gì?

Đáp án ngắn: PENDING_PAYMENT.

Giải thích cho người mới: Vì user vừa tạo booking nhưng chưa thanh toán xong. Phòng được giữ tạm trong thời gian cấu hình.

File/class nên mở: BookingStatus.java; BookingService.java

Lỗi trả lời thường gặp: Đừng nói PENDING_ADMIN_APPROVAL là trạng thái chuẩn; đó là legacy/ngoại lệ.

Câu 39 [Trung bình] PaymentStatus APPROVED nghĩa là gì?

Đáp án ngắn: Giao dịch VNPAY đã được hệ thống xác minh thành công hoặc được xử lý hợp lệ trong trường hợp đặc biệt.

Giải thích cho người mới: Khi APPROVED, booking có thể chuyển CONFIRMED và đã được TravelMate giữ phòng/căn.

File/class nên mở: PaymentStatus.java; PaymentService.java

Lỗi trả lời thường gặp: Đừng nhầm APPROVED với Partner đã check-in.

Câu 40 [Trung bình] FULL_PAYMENT và DEPOSIT_30 khác nhau thế nào?

Đáp án ngắn: FULL_PAYMENT thu 100% qua VNPAY; DEPOSIT_30 chỉ thu 30% online, 70% còn lại trả tại cơ sở.

Giải thích cho người mới: Điểm quan trọng là dòng tiền quyết toán xuất phát từ phần TravelMate thu online; hoa hồng của booking online tính theo tổng đơn gốc trước voucher, còn 70% tại cơ sở chỉ dùng để đối soát.

File/class nên mở: PaymentOption.java; BookingService.java; PaymentService.java

Lỗi trả lời thường gặp: Đừng cộng 70% trả tại cơ sở vào doanh thu online của TravelMate.

Câu 41 [Trung bình] VNPAY trong project là môi trường gì?

Đáp án ngắn: VNPAY Sandbox, dùng mô phỏng thanh toán, không phải giao dịch production thật.

Giải thích cho người mới: Sandbox giúp demo luồng tạo URL, redirect, return/ipn, xác minh chữ ký và cập nhật trạng thái.

File/class nên mở: application.properties; VnpayService.java; PaymentController.java

Lỗi trả lời thường gặp: Đừng nói hệ thống xử lý tiền thật.

Câu 42 [Khó] VNPAY return và IPN khác nhau thế nào?

Đáp án ngắn: Return là browser user quay về hệ thống sau thanh toán; IPN là callback server-to-server từ VNPAY nếu có URL public.

Giải thích cho người mới: Trong local không có ngrok, Return vẫn có thể xử lý demo. IPN cần URL public để VNPAY gọi được.

File/class nên mở: PaymentController.java; application.properties

Lỗi trả lời thường gặp: Đừng nói IPN chắc chắn chạy trên localhost nếu không có tunnel.

Câu 43 [Khó] Idempotent trong payment nghĩa là gì?

Đáp án ngắn: Cùng một callback/return nếu gọi lại nhiều lần thì không được cộng tiền hoặc cập nhật trùng nhiều lần.

Giải thích cho người mới: Payment/payment callback có thể bị gọi lại. Service cần kiểm tra trạng thái hiện tại để tránh double update.

File/class nên mở: PaymentService.java; VnpayService.java

Lỗi trả lời thường gặp: Đừng bỏ qua rủi ro callback lặp.

Câu 44 [Trung bình] Availability/overlap xử lý ý tưởng như thế nào?

Đáp án ngắn: Kiểm tra các booking đã giữ/phát sinh trong khoảng ngày trùng với ngày user muốn đặt; nếu số lượng đã chiếm vượt quota thì không cho đặt.

Giải thích cho người mới: Quy tắc overlap thường là $\text{existing.checkIn} < \text{requested.checkOut}$ và $\text{existing.checkOut} > \text{requested.checkIn}$.

File/class nên mở: AvailabilityService.java; BookingRepository.java; AvailabilityOverlapTest.java

Lỗi trả lời thường gặp: Đừng trừ stock toàn cục mà bỏ qua ngày.

Câu 45 [Trung bình] Voucher được áp dụng ở bước nào?

Đáp án ngắn: Trong quá trình đặt phòng trước khi tạo payment/tính tổng thanh toán, hệ thống kiểm tra voucher hợp lệ rồi tính giảm giá.

Giải thích cho người mới: Voucher có scope, điều kiện, owner/cost bearer; không phải mã nào cũng áp dụng mọi phòng.

File/class nên mở: VoucherService.java; Voucher.java; VoucherApiController.java

Lỗi trả lời thường gặp: Đừng nói voucher chỉ giảm trực tiếp mà không kiểm tra điều kiện.

Câu 46 [Khó] Voucher cost bearer ADMIN/PARTNER có ý nghĩa gì?

Đáp án ngắn: Cho biết ai chịu chi phí giảm giá: Admin/TravelMate chịu thì không trừ payout partner; Partner chịu thì trừ vào số tiền quyết toán cho partner. Với DEPOSIT_30, voucher Partner chịu được phép tối đa 10% tổng đơn gốc; đúng 10% vẫn hợp lệ, chỉ chặn khi vượt 10% hoặc khiến payout cọc âm.

Giải thích cho người mới: Đây là điểm nghiệp vụ tài chính quan trọng khi hỏi settlement.

File/class nên mở: VoucherCostBearer.java; VoucherService.java; SettlementService.java

Lỗi trả lời thường gặp: Đừng luôn trừ voucher vào partner.

Câu 47 [Gài bẫy] Nếu user thanh toán 30% rồi không đến thì xử lý sao?

Đáp án ngắn: Booking có thể chuyển NO_SHOW, khoản cọc 30% bị giữ theo nghiệp vụ; phòng/căn được mở lại theo điều kiện hệ thống.

Giải thích cho người mới: Phần 70% không ghi nhận là doanh thu TravelMate vì khách không trả tại cơ sở.

File/class nên mở: NoShowDepositTest.java; BookingService.java; PaymentStatus.java

Lỗi trả lời thường gặp: Đừng nói hoàn đủ 30% nếu nghiệp vụ là mất cọc.

Câu 48 [Khó] Nếu VNPAY thành công nhưng hệ thống lỗi khi cập nhật thì sao?

Đáp án ngắn: Đây là rủi ro production. Cần có log, đối soát, IPN, idempotent và admin xử lý ngoại lệ. Trong demo, hệ thống có luồng trạng thái/đối soát nhưng production cần hoàn thiện hơn.

Giải thích cho người mới: Câu này kiểm tra tư duy rủi ro, không cần bịa hệ thống đã hoàn hảo.

File/class nên mở: PaymentService.java; PaymentController.java; logs; admin/bookings.html

Lỗi trả lời thường gặp: Đừng nói “không bao giờ lỗi”.

Partner/Admin

Câu 49 [Dễ] Partner làm được gì?

Đáp án ngắn: Partner quản lý nơi lưu trú/phòng, xem booking thuộc cơ sở mình, tiếp nhận booking đã được TravelMate giữ, check-in/check-out/no-show và xem doanh thu/quyết toán.

Giải thích cho người mới: Partner không phải admin. Partner chỉ thao tác dữ liệu của mình.

File/class nên mở: PartnerPageController.java; templates/partner/*.html

Lỗi trả lời thường gặp: Đừng nói partner duyệt phòng của chính mình.

Câu 50 [Dễ] Admin làm được gì?

Đáp án ngắn: Admin quản lý tài khoản, duyệt accommodation/room, quản lý booking, voucher, review, travel post, support, revenue, settlement, withdrawal.

Giải thích cho người mới: Admin là actor quản lý toàn hệ thống, dùng để kiểm soát dữ liệu và ngoại lệ.

File/class nên mở: AdminPageController.java; templates/admin/*.html

Lỗi trả lời thường gặp: Đừng nói admin cũng đặt phòng như user.

Câu 51 [Trung bình] Sau VNPAY thành công partner phải làm gì?

Đáp án ngắn: Partner xem booking đã được TravelMate giữ. Sau đó xử lý check-in/check-out khi khách đến và rời cơ sở.

Giải thích cho người mới: VNPAY/TravelMate ghi nhận thanh toán không đồng nghĩa khách đã nhận phòng.

File/class nên mở: PartnerPageController.java; BookingService.java; PartnerBookingStatus.java

Lỗi trả lời thường gặp: Đừng nhầm CONFIRMED với COMPLETED.

Câu 52 [Trung bình] Check-in và check-out khác nhau thế nào?

Đáp án ngắn: Check-in ghi nhận khách đã nhận phòng; check-out ghi nhận khách đã hoàn tất lưu trú, booking có thể chuyển COMPLETED.

Giải thích cho người mới: Sau COMPLETED, user có thể đánh giá thang 1-10 nếu đủ điều kiện.

File/class nên mở: BookingService.java; PartnerPageController.java; ReviewService.java

Lỗi trả lời thường gặp: Đừng cho user review trước khi hoàn tất.

Câu 53 [Trung bình] Settlement là gì?

Đáp án ngắn: Settlement là kỳ quyết toán doanh thu cho partner. Tiền quyết toán xuất phát từ phần TravelMate đã thu online, còn hoa hồng của booking online được tính theo tổng đơn gốc trước voucher cho cả FULL_PAYMENT và DEPOSIT_30.

Giải thích cho người mới: Trong project đây là sổ ghi nhận nội bộ, không phải chuyển khoản ngân hàng tự động.

File/class nên mở: SettlementService.java; PartnerSettlement.java

Lỗi trả lời thường gặp: Đừng gọi là ví ngân hàng thật.

Câu 54 [Khó] Partner wallet là gì?

Đáp án ngắn: Partner wallet là ví/sổ quyết toán nội bộ ghi số dư, tiền đã kiểm, yêu cầu rút và giao dịch ví; không phải ví điện tử thật.

Giải thích cho người mới: Admin đánh dấu settlement paid thì ví có thể được ghi nhận; withdrawal request mô phỏng yêu cầu rút tiền.

File/class nên mở: PartnerWalletService.java; PartnerWallet.java; PartnerWithdrawalRequest.java

Lỗi trả lời thường gặp: Đừng nói hệ thống tự chuyển khoản ngân hàng.

Câu 55 [Khó] Commission được tính thế nào?

Đáp án ngắn: Commission là hoa hồng nền tảng, có thể dựa theo loại lưu trú hoặc cấu hình riêng của phòng. Với booking online, hoa hồng dùng tổng đơn gốc trước voucher; khoản partner nhận về lấy từ tiền TravelMate đã thu online sau khi tách hoa hồng và voucher Partner chịu.

Giải thích cho người mới: Cần nói “theo cấu hình trong hệ thống”, không tự bịa cố định nếu source có room-level commission.

File/class nên mở: CommissionService.java; SettlementService.java; Room.java

Lỗi trả lời thường gặp: Đừng nói mọi hotel luôn 15% nếu source có override.

Câu 56 [Gài bẫy] Partner có xem được booking của partner khác không?

Đáp án ngắn: Không được. Về nghiệp vụ và phân quyền, Partner chỉ xem booking thuộc cơ sở mình quản lý.

Giải thích cho người mới: Cần kiểm tra bằng controller/service/query và test nếu có.

File/class nên mở: PartnerPageController.java; BookingRepository.java; AvailabilityServicePartnerTypeGuardTest.java

Lỗi trả lời thường gặp: Đừng nói “cứ có link là xem được”.

Review/Support

Câu 57 [Dễ] Review hoạt động khi nào?

Đáp án ngắn: User đánh giá sau khi booking đã hoàn tất bằng thang 1-10. Review liên kết với user, booking và accommodation.

Giải thích cho người mới: Review khác với hạng sao cơ sở lưu trú. UI hiện tại không để Partner/Admin nhập hạng sao; rating hiển thị là điểm trung bình review /10 sau trải nghiệm thật.

File/class nên mở: ReviewService.java; Review.java; templates/user/mybooking.html

Lỗi trả lời thường gặp: Đừng cho phép đánh giá khi chưa ở xong.

Câu 58 [Trung bình] Rating của user và điểm hiển thị có thể khác nhau không?

Đáp án ngắn: Có. User chấm review theo thang 1-10; hệ thống hiển thị điểm trung bình của cơ sở lưu trú theo /10. Nếu chưa có review thì UI hiện "Chưa có đánh giá".

Giải thích cho người mới: Khi bảo vệ, nói rõ review là phản hồi sau booking completed; hệ thống không dùng hạng sao làm điểm đánh giá hiển thị, mà dùng rating tổng hợp /10 từ review.

File/class nên mở: ReviewService.java; Accommodation.java

Lỗi trả lời thường gặp: Dùng lần “hạng sao khách sạn” với “đánh giá của khách”.

Support/Notification

Câu 59 [Trung bình] Support ticket dùng để làm gì?

Đáp án ngắn: Dùng để user/partner gửi yêu cầu hỗ trợ, admin theo dõi và xử lý.

Giải thích cho người mới: Support giúp hệ thống có kênh phản hồi ngoài booking.

File/class nên mở: SupportTicket.java; SupportTicketService.java; templates/admin/support.html

Lỗi trả lời thường gặp: Dùng nói support là chat trực tiếp nếu source là ticket.

Câu 60 [Trung bình] Notification dùng để làm gì?

Đáp án ngắn: Thông báo trong hệ thống cho user/partner/admin về các sự kiện như booking, thanh toán, xử lý trạng thái hoặc hỗ trợ; email xác nhận booking sau VNPAY success là luồng SMTP riêng.

Giải thích cho người mới: Notification giúp người dùng biết thay đổi mà không phải tự kiểm tra mọi trang.

File/class nên mở: Notification.java; NotificationService.java; NotificationApiController.java

Lỗi trả lời thường gặp: Dùng nói notification là email nếu source là notification trong hệ thống; email booking/password reset là luồng SMTP riêng.

Chatbot

Câu 61 [Dễ] Chatbot trong TravelMate làm gì?

Đáp án ngắn: Hỗ trợ hỏi nhanh về đặt phòng, voucher, chính sách, gợi ý lưu trú/gợi ý du lịch trong phạm vi TravelMate.

Giải thích cho người mới: Chatbot không phải AI tổng quát. Nó có rule-based và có thể gọi Groq API nếu cấu hình.

File/class nên mở: ChatbotService.java; ChatbotApiController.java; application.properties

Lỗi trả lời thường gặp: Dùng để chatbot trả lời chuyện ngoài du lịch như yêu đương, chứng khoán, y tế.

Câu 62 [Khó] Nếu không có GROQ_API_KEY thì chatbot có hoạt động không?

Đáp án ngắn: Vẫn có thể xử lý một số intent rule-based/fallback trong phạm vi hệ thống; API AI chỉ là phần mở rộng nếu cấu hình.

Giải thích cho người mới: Nên test demo trước để biết chính xác phản hồi hiện tại.

File/class nên mở: ChatbotService.java; application.properties

Lỗi trả lời thường gặp: Dùng khẳng định AI luôn chạy nếu không có key.

File/Excel

Câu 63 [Trung bình] File upload dùng cho phần nào?

Đáp án ngắn: Dùng cho ảnh phòng/nơi lưu trú; hệ thống có cấu hình multipart và service lưu file.

Giải thích cho người mới: Ảnh giúp Partner/Admin quản lý UI thực tế hơn. Production cần kiểm soát loại file/kích thước/an toàn.

File/class nên mở: FileStorageService.java; RoomImageService.java; application.properties

Lỗi trả lời thường gặp: Đừng cho upload mọi file không kiểm soát.

Câu 64 [Trung bình] Export Excel dùng để làm gì?

Đáp án ngắn: Dùng xuất dữ liệu báo cáo như settlement/withdrawal/revenue để admin/partner kiểm tra.

Giải thích cho người mới: Project dùng Apache POI trong pom.xml và ExcelExportService.

File/class nên mở: pom.xml; ExcelExportService.java; AdminPageController.java

Lỗi trả lời thường gặp: Đừng nói export Excel là chức năng database.

Test

Câu 65 [Dễ] Project có test không?

Đáp án ngắn: Có test trong src/test/java cho booking, payment, voucher, settlement, password reset, chatbot, template, security...

Giải thích cho người mới: Chỉ nói số test pass nếu nhóm chạy lại mvn test thật trên máy demo.

File/class nên mở: src/test/java/com/travelmate

Lỗi trả lời thường gặp: Đừng đọc số cũ nếu chưa xác nhận.

Câu 66 [Trung bình] JUnit, Mockito, MockMvc dùng để làm gì?

Đáp án ngắn: JUnit viết test, Mockito giả lập dependency, MockMvc test request/response web layer của Spring. Report hiện tại ghi 279/279 test pass, nhưng nhóm nên chạy lại trước khi bảo vệ nếu muốn nói con số này.

Giải thích cho người mới: Không phải tất cả test đều mở browser thật. Nhiều test kiểm tra service hoặc template.

File/class nên mở: src/test/java; pom.xml; docs/07_Test_Report.md

Lỗi trả lời thường gặp: Đừng nói test tự động thay thế hoàn toàn test demo thủ công.

Production

Câu 67 [Khó] Nếu đưa TravelMate lên production cần bổ sung gì?

Đáp án ngắn: Bật CSRF, quản lý secret bằng env/secret manager, dùng migration DB, log/monitoring, backup, HTTPS, kiểm thử tải, đối soát payment thật, CI/CD.

Giải thích cho người mới: Đây là câu phản biện cuối buổi. Trả lời trung thực sẽ được đánh giá tốt hơn nói hệ thống hoàn hảo.

File/class nên mở: SecurityConfig.java; application.properties;
docs/18_Project_Analysis_For_Report_And_Defense.md

Lỗi trả lời thường gặp: Đừng nói “chỉ cần up hosting là xong”.

Câu 68 [Gai bẫy] Điểm yếu lớn nhất của project hiện tại là gì?

Đáp án ngắn: Ở mức demo: chạy local, VNPAY sandbox, dữ liệu mẫu, CSRF tạm tắt, ddl-auto=update, chưa kiểm thử tải/e2e production đầy đủ.

Giải thích cho người mới: Nói điểm yếu không làm mất điểm nếu biết hướng khắc phục.

File/class nên mở: application.properties; SecurityConfig.java;
docs/18_Project_Analysis_For_Report_And_Defense.md

Lỗi trả lời thường gặp: Đừng phủ nhận mọi hạn chế.

Câu 69 [Gài bẫy] Nếu hai người cùng đặt phòng cuối cùng một lúc thì sao?

Đáp án ngắn: Hệ thống có logic availability/overlap và đã dùng lock ghi khi tạo booking; production vẫn cần stress test/concurrency test mạnh hơn.

Giải thích cho người mới: Đây là câu bẫy về concurrency. Không nên nói chắc chắn tuyệt đối nếu chưa có locking rõ.

File/class nên mở: AvailabilityService.java; BookingService.java; AvailabilityOverlapTest.java

Lỗi trả lời thường gặp: Đừng nói “không xảy ra vì đã kiểm tra trước”.

Câu 70 [Gài bẫy] Nếu secret VNPAY/Google/SMTP bị commit lên Git thì sao?

Đáp án ngắn: Đó là rủi ro bảo mật. Project nên dùng biến môi trường hoặc .env local không commit; production cần secret manager.

Giải thích cho người mới: application.properties dùng placeholder env, nhưng cần kiểm tra repo không lộ secret thật.

File/class nên mở: application.properties; .env.local (không nên commit)

Lỗi trả lời thường gặp: Đừng đọc secret thật trong báo cáo/bảo vệ.

Câu 71 [Gài bẫy] Nếu thầy hỏi “em hiểu backend đến đâu?” thì trả lời sao?

Đáp án ngắn: Em hiểu luồng chính và file liên quan: Controller nhận request, Service xử lý nghiệp vụ, Repository truy vấn DB, Entity ánh xạ bảng. Em học sâu các module booking, payment, security và phân nhóm em demo.

Giải thích cho người mới: Trả lời trung thực: không nhận mình là expert toàn bộ framework, nhưng chứng minh mình mở được file và giải thích được luồng.

File/class nên mở: controller/page, service, repository, entity, security

Lỗi trả lời thường gặp: Đừng nói “em không biết backend vì AI làm”.

Frontend/Thymeleaf

Câu 72 [Dễ] Form đăng nhập gửi dữ liệu đi đâu?

Đáp án ngắn: Form login gửi POST tới /auth/login, Spring Security xử lý thông tin email/password.

Giải thích cho người mới: Trong SecurityConfig có loginProcessingUrl /auth/login và usernameParameter email.

File/class nên mở: templates/auth/login.html; SecurityConfig.java

Lỗi trả lời thường gặp: Đừng nói AuthPageController tự so sánh password nếu Spring Security xử lý.

Câu 73 [Trung bình] th:href hoặc th:action dùng để làm gì?

Đáp án ngắn: Tạo URL động theo route Spring thay vì hardcoded chuỗi thuần.

Giải thích cho người mới: Giúp link/form đúng context path và có thể truyền tham số.

File/class nên mở: templates/**/*.html

Lỗi trả lời thường gặp: Đừng nghĩ th:* chạy ở trình duyệt; nó được render phía server.

Câu 74 [Trung bình] Fragments dùng để làm gì?

Đáp án ngắn: Dùng tái sử dụng header, footer, sidebar, floating buttons giữa nhiều trang.

Giải thích cho người mới: Trong TravelMate có user-header, user-footer, admin-sidebar, partner-sidebar.

File/class nên mở: templates/fragments/*.html

Lỗi trả lời thường gặp: Đừng copy header lặp ở mọi trang nếu có fragment.

Booking/Payment

Câu 75 [Trung bình] BookingSource dùng làm gì?

Đáp án ngắn: Phân biệt nguồn booking như booking online qua TravelMate, direct booking hoặc manual block nếu source dùng.

Giải thích cho người mới: Nghiệp vụ direct/manual block giúp partner chặn quota offline hoặc bảo trì mà không tính settlement như booking online.

File/class nên mở: BookingSource.java; BookingService.java; PartnerPageController.java

Lỗi trả lời thường gặp: Đừng tính commission cho manual block.

Câu 76 [Khó] Direct booking khác booking online thế nào?

Đáp án ngắn: Direct booking do partner tạo cho khách offline, khách trả tại cơ sở, không đi qua VNPAY và không tính doanh thu online TravelMate như booking online.

Giải thích cho người mới: Direct booking vẫn có thể giữ quota phòng để tránh overbooking.

File/class nên mở: PartnerPageController.java; BookingService.java; BookingSource.java

Lỗi trả lời thường gặp: Đừng nhầm direct booking với user online booking.

Câu 77 [Khó] Manual block dùng để làm gì?

Đáp án ngắn: Partner khóa phòng/căn cho bảo trì hoặc giữ nội bộ; không có doanh thu, không payment, nhưng ảnh hưởng availability trong khoảng ngày đó.

Giải thích cho người mới: Đây là nghiệp vụ vận hành tồn kho phòng.

File/class nên mở: BookingService.java; PartnerPageController.java; BookingSource.java

Lỗi trả lời thường gặp: Đừng đưa manual block vào settlement.

Admin

Câu 78 [Trung bình] Admin duyệt accommodation/room để làm gì?

Đáp án ngắn: Để kiểm soát dữ liệu partner đăng trước khi hiển thị/cho đặt phòng.

Giải thích cho người mới: Nếu không duyệt, partner có thể đăng thông tin sai, ảnh hưởng user.

File/class nên mở: AdminPageController.java; AccommodationService.java; ApprovalStatus.java

Lỗi trả lời thường gặp: Đừng để partner tự duyệt dữ liệu của mình.

Câu 79 [Trung bình] Admin quản lý voucher khác partner gán voucher thế nào?

Đáp án ngắn: Admin phát hành/quản lý voucher; partner chỉ gán/chọn voucher phù hợp cho phòng/cơ sở của mình nếu chức năng cho phép.

Giải thích cho người mới: Điểm này liên quan cost bearer và quyền quản trị.

File/class nên mở: VoucherService.java; RoomVoucherAssignment.java; templates/partner/vouchers.html

Lỗi trả lời thường gặp: Đừng nói partner tự tạo mọi voucher.

Settlement

Câu 80 [Khó] Vì sao quyết toán theo tháng hợp lý cho báo cáo?

Đáp án ngắn: Vì settlement tháng dễ quản lý và khớp nghiệp vụ TravelMate hiện tại; admin tổng hợp kỳ và chi trả nội bộ.

Giải thích cho người mới: Nếu báo cáo từng nói weekly thì cần sửa về monthly theo phiên bản hiện tại.

File/class nên mở: SettlementService.java; PartnerSettlement.java; README_DEMO.md

Lỗi trả lời thường gặp: Đừng nói vừa tuần vừa tháng không thống nhất.

Security/Auth

Câu 81 [Khó] CustomAuthSuccessHandler làm gì?

Đáp án ngắn: Sau form login thành công, handler redirect người dùng đến trang phù hợp theo role.

Giải thích cho người mới: Ví dụ USER về trang user/home, PARTNER về partner dashboard, ADMIN về admin dashboard tùy cấu hình source.

File/class nên mở: CustomAuthSuccessHandler.java

Lỗi trả lời thường gặp: Đừng nói tất cả user vào cùng một trang.

Câu 82 [Khó] UserDetailsServiceImpl làm gì?

Đáp án ngắn: Load user từ database để Spring Security xác thực email/password và role.

Giải thích cho người mới: Spring Security không tự biết database của mình; cần UserDetailsService để lấy user.

File/class nên mở: UserDetailsServiceImpl.java; UserRepository.java

Lỗi trả lời thường gặp: Đừng nói Security đọc trực tiếp table users không qua code.

Password Reset

Câu 83 [Trung bình] Token reset password vì sao phải dùng một lần?

Đáp án ngắn: Để sau khi đổi mật khẩu, link cũ không thể dùng lại, giảm rủi ro bị chiếm tài khoản.

Giải thích cho người mới: Token còn có expiresAt để giới hạn thời gian hiệu lực.

File/class nên mở: PasswordResetToken.java; PasswordResetService.java

Lỗi trả lời thường gặp: Đừng để token vĩnh viễn.

Chatbot

Câu 84 [Trung bình] Chatbot nên từ chối câu hỏi nào?

Đáp án ngắn: Câu hỏi ngoài phạm vi TravelMate như yêu đương, y tế, chứng khoán, code... chatbot nên chuyển hướng về du lịch/đặt phòng.

Giải thích cho người mới: Điều này giữ chatbot đúng phạm vi đề án.

File/class nên mở: ChatbotService.java

Lỗi trả lời thường gặp: Đừng để chatbot trả lời mọi chủ đề như ChatGPT tổng quát.

Test

Câu 85 [Khó] AvailabilityOverlapTest kiểm tra ý gì?

Đáp án ngắn: Kiểm tra logic trùng khoảng ngày để tránh overbooking theo check-in/check-out.

Giải thích cho người mới: Đây là test quan trọng vì đặt phòng phụ thuộc ngày, không phải stock tổng.

File/class nên mở: AvailabilityOverlapTest.java; AvailabilityService.java

Lỗi trả lời thường gặp: Đừng nói chỉ cần available_quantity là đủ.

Câu 86 [Khó] VoucherCalculationTest kiểm tra gì?

Đáp án ngắn: Kiểm tra cách tính giảm giá, điều kiện voucher và ảnh hưởng đến tổng tiền/thanh toán.

Giải thích cho người mới: Voucher sai có thể gây sai doanh thu nên cần test.

File/class nên mở: VoucherCalculationTest.java; VoucherService.java

Lỗi trả lời thường gặp: Đừng nghĩ voucher chỉ là UI.

Production

Câu 87 [Gài bẫy] Open-in-view=true có lưu ý gì?

Đáp án ngắn: Giúp tránh LazyInitializationException trong demo, nhưng production nên cân nhắc vì có thể che giấu việc service chưa chuẩn bị đủ dữ liệu.

Giải thích cho người mới: Nên fetch dữ liệu rõ ràng ở service/view model.

File/class nên mở: application.properties

Lỗi trả lời thường gặp: Đừng nói cứ bật open-in-view là tốt nhất.

Câu 88 [Gài bẫy] Nếu Google OAuth chưa cấu hình client secret thì sao?

Đáp án ngắn: Google login sẽ không hoạt động đầy đủ; app có logic optional để không bắt buộc cấu hình khi chạy demo local.

Giải thích cho người mới: Phải nói “nếu được cấu hình”, không claim luôn chạy.

File/class nên mở: GoogleOAuthClientConfig.java; application.properties

Lỗi trả lời thường gặp: Đừng demo Google nếu chưa cấu hình thật.

Phần 6 — Bộ câu hỏi nhanh có đáp án một dòng

Phần này dùng để ôn nhanh trước khi bảo vệ. Nếu còn thời gian, hãy mở file liên quan để tự giải thích lại.

Nhóm	Câu hỏi	Đáp án ngắn	File/class liên quan
Must know	Spring Boot chạy port nào?	Mặc định cấu hình server.port=8080 trong application.properties.	application.properties
Must know	DB dùng hệ gì?	MySQL, kết nối qua JDBC URL trong application.properties.	application.properties
Must know	Build project bằng gì?	Maven/Maven Wrapper, phụ thuộc khai báo trong pom.xml.	pom.xml
Must know	Role chính là gì?	USER, PARTNER, ADMIN.	SecurityConfig.java; User.java
Must know	User đặt phòng ở route nào?	Các route /booking, /my-bookings dành cho USER.	SecurityConfig.java; BookingPageController.java
Must know	Admin route chính?	/admin/**.	SecurityConfig.java; AdminPageController.java
Must know	Partner route chính?	/partner/**.	SecurityConfig.java; PartnerPageController.java
Must know	VNPAY dùng URL nào?	Sandbox pay-url trong application.properties.	application.properties
Must know	Payment option có mấy loại?	FULL PAYMENT và DEPOSIT 30.	PaymentOption.java
Must know	Booking COMPLETED khi nào?	Sau khi lưu trú hoàn tất/check-out theo luồng partner.	BookingStatus.java; BookingService.java
Must know	Review liên kết với gì?	User, booking và accommodation.	Review.java
Must know	Partner wallet có phải ví thật không?	Không, là ví/sổ quyết toán nội bộ.	PartnerWallet.java; PartnerWalletService.java
Must know	Chatbot có phải AI tổng quát	Không, chỉ trong phạm vi TravelMate; Groq	ChatbotService.java

	không?	là optional.	
Must know	Ảnh upload giới hạn dung lượng?	multipart max-file-size=10MB, max-request-size=20MB.	application.properties
Must know	Tài liệu demo DB nằm đâu?	README DEMO.md và travelmate_db.sql.	README DEMO.md
Should know	Vì sao dùng BCrypt?	Hash password an toàn hơn plain text, có salt.	SecurityConfig.java
Should know	Vì sao dùng JPA?	Ảnh xạ object Java với bảng DB, giảm SQL thủ công.	entity; repository
Should know	Vì sao cần Service?	Tập trung nghiệp vụ, dễ test và tái sử dụng.	service/*.java
Should know	Voucher scope là gì?	Phạm vi áp dụng voucher: hệ thống/cơ sở/phòng tùy thiết kế.	Voucher.java; VoucherScope.java
Should know	Remaining 70% lưu ý gì?	Với DEPOSIT_30, 70% trả tại cơ sở, không là doanh thu online TravelMate.	PaymentOption.java; SettlementService.java
Should know	Booking expired là gì?	Booking chưa thanh toán quá thời gian bị hết hạn/hủy, phòng mở lại.	BookingExpiryScheduler.java; PaymentStatus.java
Should know	No-show là gì?	Khách không đến nhận phòng; cọc có thể bị giữ theo nghiệp vụ.	BookingStatus.java; NoShowDepositTest.java
Should know	Support ticket là gì?	Yêu cầu hỗ trợ từ user/partner để admin xử lý.	SupportTicket.java
Should know	Notification dùng cho ai?	Thông báo sự kiện hệ thống cho user/partner/admin tùy nghiệp vụ.	NotificationService.java
Should know	Excel export phục vụ gì?	Xuất dữ liệu settlement/withdrawal/revenue.	ExcelExportService.java
Should know	Internationalization là gì?	Hỗ trợ ngôn ngữ/locale qua messages và config.	InternationalizationConfig.java
Should know	Destination alias để làm gì?	Chuẩn hóa tìm kiếm Đà Lạt/dalat/da lat...	DestinationAliasUtil.java
Should know	BusinessLabelUtil để làm gì?	Hiển thị nhãn nghiệp vụ thân thiện thay vì enum kỹ thuật.	BusinessLabelUtil.java
Should know	Thymeleaf cache=false để làm gì?	Dev dễ reload giao diện; production nên bật cache.	application.properties
Hard defense	Race condition đặt phòng xử lý triệt để chưa?	Có kiểm tra availability/overlap, production nên thêm transaction/locking kỹ hơn.	AvailabilityService.java; BookingService.java
Hard defense	CSRF tất có nguy hiểm không?	Có, chỉ phù hợp demo; production cần bật và thêm token.	SecurityConfig.java
Hard defense	ddl-auto=update có nên dùng production?	Không nên; production cần migration/validate.	application.properties
Hard defense	Secret nằm ở đâu là an toàn?	Biến môi trường/.env local không commit, production dùng secret manager.	application.properties
Hard defense	VNPAY thật cần thêm gì?	Đối soát, IPN public URL, log, refund, dispute handling.	PaymentController.java; VnpayService.java
Hard defense	Có load test chưa?	Chưa nên claim nếu chưa chạy; mới có unit/integration/template tests.	src/test/java
Hard defense	Có thể deploy ngay không?	Chạy demo được, production cần bổ sung bảo mật, config, DB migration, monitoring.	application.properties; SecurityConfig.java
Hard defense	Nếu admin thao tác nhầm settlement thì sao?	Cần audit log, idempotent và kiểm soát trạng thái; source có audit/logging nội bộ nhưng production cần chặt hơn.	AdminActionLog.java; SettlementService.java
Hard defense	Nếu callback VNPAY bị giả mạo?	Cần kiểm tra chữ ký/hash secret và response code trước khi cập nhật payment.	VnpayService.java; PaymentService.java
Hard defense	Nếu user nhập voucher làm tiền quá thấp?	Service cần guard mức tối thiểu VNPAY; test report có nhắc guard.	VoucherService.java; BookingService.java

Phần 7 — Phân chia học cho nhóm 3 người

Không chia theo kiểu ai biết phần nấy. Cả 3 thành viên đều phải học tổng quan, actor, MVC, database chính, security và booking flow. Sau đó mỗi người học sâu phần mình demo.

Thành viên	Phản chịu trách nhiệm chính	File/class cần đọc	Luồng demo nên trình bày	Câu hỏi chéo vẫn phải biết
Thành viên 1	User flow + giao diện + Thymeleaf + chatbot	HomeController, AccommodationPageController, BookingPageController; templates/user; static/assets/css/js;	Demo trang chủ -> tìm kiếm -> chi tiết -> chatbot -> my booking	Phải biết booking tổng quan, role, DB chính.

		ChatbotService		
Thành viên 2	Booking + payment + voucher + availability	BookingService, PaymentService, VnpayService, AvailabilityService, VoucherService; Booking, Payment, Voucher	Demo đặt phòng -> VNPAY sandbox -> kết quả thanh toán -> trạng thái booking	Phải biết Security, Partner/Admin xử lý sau thanh toán.
Thành viên 3	Partner/Admin + Security + database + settlement/wallet/test	SecurityConfig, AdminPageController, PartnerPageController, SettlementService, PartnerWalletService; entity/repository; tests	Demo TravelMate giữ phòng -> Partner check-in/check-out -> Admin settlement/wallet -> test/report	Phải biết user booking và payment để nối luồng.

Phần 8 — Kịch bản demo bảo vệ

Bước demo	Người trình bày	Màn hình/route	Nói gì trong 30–60 giây	Có thể bị hỏi gì
1. Giới thiệu project	Cả nhóm hoặc trưởng nhóm	/	TravelMate là website đặt phòng/gợi ý du lịch dùng Spring Boot MVC + Thymeleaf + MySQL, có 3 nhóm chính User/Partner/Admin.	Vì sao chọn Spring Boot MVC?
2. Guest/User tìm kiếm	TV1	/ hoặc /accommodations	Nhập điểm đến, xem danh sách nơi lưu trú, giải thích search dùng dữ liệu accommodation/room và alias điểm đến.	Search theo ngày có kiểm phòng không?
3. Chi tiết và booking	TV1 + TV2	/accommodations/{id}, /booking	Chọn phòng, ngày, số lượng, voucher/payment option; backend validate trước khi tạo booking.	Nếu phòng không còn thì sao?
4. VNPAY Sandbox	TV2	/payment/vnpay/create/... và return	Hệ thống tạo URL VNPAY sandbox; thành công thì Payment APPROVED, Booking CONFIRMED.	Return khác IPN thế nào?
5. Partner xử lý	TV3	/partner/bookings	Partner chỉ thấy booking cơ sở mình, tiếp nhận booking đã được TravelMate giữ, check-in/check-out/no-show.	Partner có xem booking người khác không?
6. Admin quản lý	TV3	/admin/bookings, /admin/accommodations, /admin/rooms	Admin duyệt dữ liệu, quản lý booking toàn hệ thống, voucher, review, support.	Admin có thay Partner check-in không?
7. Settlement/wallet	TV3	/admin/settlements, /partner/wallet	Settlement là sổ quyết toán nội bộ: online amount - commission - partner-borne voucher.	Cọc 30% tính quyết toán thế nào?
8. Chatbot/review/support	TV1	chatbot floating, /my-bookings, support	Các chức năng phụ hỗ trợ trải nghiệm; chatbot trong phạm vi TravelMate, review sau booking hoàn tất.	Chatbot là AI thật hay rule-based?

Phần 9 — Checklist học trước ngày bảo vệ

Bắt buộc cả nhóm phải thuộc

- TravelMate là gì, giải quyết vấn đề gì, có actor nào.
- Luồng User đặt phòng từ tìm kiếm đến VNPAY và my bookings.
- BookingStatus, PaymentStatus, PaymentOption cơ bản.
- Controller-Service-Repository-Entity khác nhau thế nào.
- Spring Security bảo vệ /admin/**, /partner/**, /booking/** như thế nào.
- VNPAY là Sandbox, không phải giao dịch thật.
- DEPOSIT_30 chỉ thu 30% online, 70% còn lại trả tại cơ sở.
- Partner xử lý booking: tiếp nhận đơn đã được TravelMate giữ, check-in, check-out, no-show.
- Admin quản lý/duyệt/settlement ở mức mô phỏng nội bộ.

- Các điểm yếu production: CSRF, secret, ddl-auto update, load test, sandbox.

Nên hiểu để trả lời sâu hơn

- Availability overlap theo ngày.
- Voucher scope và cost bearer.
- Settlement/wallet/withdrawal.
- Password reset token và SMTP.
- Google OAuth2 chỉ USER.
- Chatbot rule-based + Groq optional.
- File upload và export Excel.
- Test tự động trong src/test/java.

Có thể nói là hướng phát triển thêm

- Triển khai cloud/production, HTTPS, monitoring.
- CI/CD và secret manager.
- Kiểm thử tải và e2e browser test.
- Hoàn tiền/đối soát thanh toán thật.
- Mobile app và bản đồ trực tuyến.
- Chatbot hiểu ngữ cảnh tốt hơn.

Phần 10 — Những câu trả lời nên tránh

Câu trả lời dễ mất điểm	Cách nói lại trung thực và chuyên nghiệp
“Em không biết vì AI làm.”	Nhóm em có dùng công cụ hỗ trợ, nhưng đã đọc lại source, chạy demo, kiểm thử và hiểu luồng mình trình bày.
“Framework tự làm hết.”	Spring hỗ trợ nhiều phần, nhưng nhóm vẫn phải cấu hình Controller, Service, Entity, Repository, Security và nghiệp vụ.
“Em chỉ làm giao diện.”	Em phụ trách giao diện nhưng vẫn hiểu cách form gửi request, Controller nhận dữ liệu và Service xử lý nghiệp vụ.
“Code chạy được là được.”	Ngoài chạy được, hệ thống cần đúng nghiệp vụ, phân quyền, database, kiểm thử và xử lý lỗi.
“Database tự sinh.”	JPA có thể update schema khi dev, nhưng dữ liệu demo và quan hệ bảng được nhóm chuẩn bị trong entity/sql seed.
“Thanh toán là gọi API thôi.”	Thanh toán gồm tạo URL, ký tham số, redirect, nhận return/ipn, xác minh chữ ký và cập nhật trạng thái idempotent.
“Hệ thống đã production được.”	Hiện hệ thống ở mức demo local/sandbox; production cần CSRF, secret manager, HTTPS, monitoring, load test và đối soát thật.

Phần 11 — Bài kiểm tra mô phỏng 3 vòng

Vòng	Câu hỏi	Đáp án mẫu	Nhận xét nếu trả lời thiếu
Vòng 1 — Tổng quan	TravelMate có những nhóm người dùng nào?	Guest/User/Partner/Admin; khi rút gọn báo cáo nhấn mạnh User, Partner, Admin.	Nếu chỉ nói user/admin thì thiếu partner và guest public.
Vòng 1 — Tổng quan	Vì sao project không chỉ là HTML/CSS?	Vì có backend Spring Boot, MySQL, security, booking, payment, voucher, settlement, test.	Nếu trả lời “vì có giao diện đẹp” là sai trọng tâm.
Vòng 1 — Tổng quan	Kiến trúc project là gì?	Spring Boot MVC + Thymeleaf server-side rendering, layered architecture.	Không gọi là React SPA hay microservice.
Vòng 2 — Code/nghiệp vụ	Booking được tạo với trạng thái gì?	PENDING_PAYMENT cho luồng online chuẩn.	Nếu nói PENDING_ADMIN_APPROVAL cần giải thích đó là legacy/ngoại lệ.
Vòng 2 — Code/nghiệp vụ	Thanh toán VNPAY thành công cập nhật gì?	Payment APPROVED, Booking CONFIRMED, PartnerStatus phản ánh bước vận hành; sau payment success luồng chính là	Nếu nói COMPLETED là sai vì khách chưa lưu trú xong.

		PARTNER_CONFIRMED.	
Vòng 2 — Code/nghiệp vụ	TravelMate tự giữ phòng/căn nằm ở đâu?	PartnerPageController/BookingService, trạng thái PartnerBookingStatus.	Nếu nói Admin xác nhận thay mọi booking là sai nghiệp vụ chính.
Vòng 2 — Code/nghiệp vụ	Repository có nhiệm vụ gì?	Truy vấn/cập nhật DB bằng Spring Data JPA.	Không nhầm với Service nghiệp vụ.
Vòng 3 — Phân biện khó	Nếu hai user đặt cùng phòng cùng lúc thì sao?	Có availability/overlap và lock ghi ở backend; production vẫn cần stress test/concurrency test mạnh hơn.	Không nói chắc chắn tuyệt đối nếu chưa có stress test/concurrency test production.
Vòng 3 — Phân biện khó	CSRF đang tắt có sao không?	Demo được nhưng production cần bật CSRF token.	Không phủ nhận rủi ro.
Vòng 3 — Phân biện khó	Nếu VNPAY callback lặp lại thì sao?	Service cần idempotent để không cập nhật/cộng tiền nhiều lần.	Không bỏ qua rủi ro callback lặp.

Phụ lục — File/class nên mở trước bảo vệ

- Surefire mới nhất — nguồn số test hiện tại: 279/279 PASS, 0 fail, 0 error, 0 skipped.
- travelmate_wallet_settlement_test_data.sql — dữ liệu bổ sung cho demo ví/quyết toán Palace/Rừng Thông; không nhầm với travelmate_db.sql chính.
- pom.xml — xác nhận Spring Boot, Java 21, Thymeleaf, Security, OAuth2, Mail, JPA, POI, Test.
- application.properties — DB, port 8080, VNPAY Sandbox, SMTP, Google OAuth, chatbot Groq, ddl-auto=update, CSRF lưu ý trong SecurityConfig.
- SecurityConfig.java — phân quyền route, BCrypt, form login, OAuth2 optional, CSRF đang tắt demo.
- BookingService.java — nghiệp vụ đặt phòng, trạng thái, check-in/out/no-show, direct/manual nếu có.
- PaymentService.java + VnpayService.java + PaymentController.java — tạo URL VNPAY, return/ipn, cập nhật payment/booking.
- AvailabilityService.java — kiểm tra phòng còn theo ngày/overlap.
- VoucherService.java — kiểm tra voucher, discount, cost bearer.
- SettlementService.java + PartnerWalletService.java — quyết toán/ví nội bộ/withdrawal.
- ReviewService.java — đánh giá sau lưu trú.
- ChatbotService.java — chatbot rule-based + Groq optional.
- templates/user, templates/partner, templates/admin — giao diện demo.
- src/test/java/com/travelmate — test tự động cần mở khi bị hỏi về kiểm thử.

Phần 12 — Cập nhật trạng thái sau rà soát

Kết luận rà soát: gợi ý của ChatGPT Plus đúng về hướng chính. Source hiện tại đã có luồng online chuẩn PENDING_PAYMENT -> VNPAY -> APPROVED/CONFIRMED, email xác nhận sau VNPAY success, đặt lại đơn hủy/no-show, rating theo review và guard voucher Partner cho cọc 30%. Vì project đã hoàn thành, phần chỉnh mới chỉ chuẩn hóa quy tắc tài chính, SQL seed và nhãn hiển thị.

Các chỉnh sửa cần nhớ: travelmate_db.sql đổi default booking/payment sang PENDING_PAYMENT, DIRECT/MANUAL_BLOCK dùng PaymentStatus.NOT_REQUIRED, và bổ sung các mã BK-OPS-* để xem đủ trạng thái PENDING_PAYMENT, CANCELLED, FAILED, EXPIRED, REFUND_PENDING, REFUNDED. UI chỉ chỉnh wording nhỏ để phân biệt “đã ghi nhận booking” với “đã check-in/hoàn tất”.

12.1. Bảng chốt BookingStatus

BookingStatus	Ý nghĩa đúng	Câu trả lời khi bị hỏi	File/dữ liệu nên mở
PENDING_PAYMENT	Booking online vừa tạo, đang chờ khách thanh	Đây là flow chính khi User đặt	BookingService.createBooking; BK-OPS-PENDPAY-01

	toán VNPAY; phòng được giữ tạm.	online, chưa phải Admin duyệt.	
PENDING_ADMIN_APPROVAL	Trạng thái legacy/ngoại lệ cần Admin đối soát thủ công.	Không dùng làm flow chính; chỉ dùng khi giao dịch cần kiểm tra đặc biệt.	BookingService.approve/rejectBookingByAdmin; BK-ANM-GDN-0001
CONFIRMED	Booking đã được hệ thống ghi nhận sau thanh toán hợp lệ hoặc Admin xử lý ngoại lệ.	CONFIRMED chưa có nghĩa khách đã ở; Partner vẫn cần xử lý check-in/check-out.	PaymentService.markGatewaySuccess; partner_status
CHECKED_IN	Khách đã nhận phòng và đang lưu trú.	CHECKED_IN khác COMPLETED vì khách chưa trả phòng.	BookingService.checkInByPartner
COMPLETED	Khách đã check-out, đủ điều kiện review và settlement nếu payment hợp lệ.	Settlement chỉ lấy booking đủ điều kiện, không lấy đơn đang lưu trú.	BookingService.markCompletedByPartner; SettlementService
NO_SHOW	Khách không đến, áp dụng rõ nhất cho đơn cọc 30%.	No-show cọc 30% giữ cọc, không cộng 70% chưa thu vào doanh thu TravelMate.	BookingService.markNoShowByPartner; NoShowDepositTest
CANCELLED	Booking đã hủy do user/admin/payment fail/cancel/expire/partner issue.	Phải nhìn PaymentStatus để biết hủy chưa trả tiền, giữ cọc hay chờ hoàn tiền.	BookingService.cancelBooking; PaymentService.markGatewayFailed

12.2. Bảng chốt PaymentStatus

PaymentStatus	Ý nghĩa đúng	Điểm để nhầm	Mã dữ liệu nên xem
PENDING_PAYMENT	Payment mới tạo, chưa có kết quả từ VNPAY.	Không gọi là đã thanh toán.	BK-OPS-PENDPAY-01
APPROVED	Thanh toán đã được xác nhận hợp lệ.	APPROVED là trạng thái payment, khác BookingStatus.CONFIRMED.	Các booking VNPAY thành công
FAILED / CANCELLED / EXPIRED	Giao dịch lỗi, user hủy, hoặc quá hạn.	Booking liên quan bị CANCELLED và quota được mở lại.	BK-OPS-FAILED-01; BK-OPS-CANCEL-01; BK-OPS-EXPIRED-01
DEPOSIT_FORFEITED	Cọc 30% bị giữ khi khách hủy sau cọc hoặc no-show.	Booking có thể là CANCELLED nếu user hủy, hoặc NO_SHOW nếu khách không đến.	BK-TLP-STD-0001; NoShowDepositTest
REFUND_PENDING	Đã thanh toán full nhưng hủy, đang chờ Admin xử lý hoàn tiền.	Không nói hệ thống đã hoàn tiền thật qua VNPAY.	BK-OPS-REFUND-PENDING-01
REFUNDED	Admin đã xác nhận hoàn tiền trong hệ thống.	Đây là ghi nhận trong project/sandbox, không phải giao dịch ngân hàng thật.	BK-OPS-REFUNDED-01
NOT_REQUIRED	Không áp dụng thanh toán TravelMate, dùng cho DIRECT/MANUAL BLOCK.	Không tính doanh thu online TravelMate và không tạo payment gateway.	BK-PL-DIRECT-001; BK-TLP-STD-BLOCK-01

12.3. Câu trả lời phản biện cần thuộc

- Booking mới online là PENDING_PAYMENT vì khách đang chờ thanh toán VNPAY; PENDING_ADMIN_APPROVAL chỉ là ngoại lệ đối soát thủ công.
- CONFIRMED nghĩa là booking đã được hệ thống ghi nhận, chưa phải khách đã check-in và chưa đủ điều kiện review.

- APPROVED nói về payment; CONFIRMED nói về booking. Hai trạng thái liên quan nhưng không cùng ý nghĩa.
- Đơn cọc 30% nếu khách hủy hoặc không đến thì payment có thể chuyển DEPOSIT_FORFEITED; 70% còn lại chưa thu tại cơ sở nên không tính vào doanh thu online.
- DIRECT/MANUAL_BLOCK là nghiệp vụ Partner quản lý lịch phòng, không qua VNPAY, không tạo gateway payment, không đưa vào settlement TravelMate online.
- Full payment no-show là chính sách phức tạp hơn; trong đồ án nên trả lời là cần Admin/Partner xử lý theo chính sách, không tự nói mất 100% nếu báo cáo chưa chốt.

12.4. Checklist sau khi import SQL trên máy khác

- Mở /admin/bookings và lọc Chờ thanh toán để thấy BK-OPS-PENDPAY-01.
- Mở /admin/bookings và kiểm tra các booking BK-OPS-CANCEL/FAILED/EXPIRED hiển thị đã hủy với payment tương ứng.
- Mở /admin/bookings và kiểm tra BK-OPS-REFUND-PENDING-01, BK-OPS-REFUNDED-01 để giải thích refund.
- Mở booking DIRECT hoặc MANUAL_BLOCK để thấy nhãn Thu tại cơ sở / Không qua TravelMate.
- Chạy test liên quan PaymentServiceTest, BookingCalculationTest, RevenueServiceHomestayTest, DataInitializerTest và UserPortalSqlSeedTest nếu có thời gian trước buổi bảo vệ.

Bổ sung — Làm rõ tài chính, quyết toán và voucher

Phần này được bổ sung sau khi rà soát các trạng thái tài chính/settlement/voucher trong source. Mục tiêu là trình bày đúng phạm vi demo: TravelMate ghi nhận nghiệp vụ trong hệ thống, còn chuyển tiền thật hoặc hoàn tiền ngân hàng chưa được tích hợp production.

Nhóm trạng thái	Cách hiểu đúng khi bảo vệ	Điểm cần tránh nói sai
REFUND_PENDING / REFUNDED	REFUND_PENDING là chờ Admin xử lý hoàn tiền; REFUNDED là đã ghi nhận kết quả xử lý trong hệ thống.	Không nói hệ thống tự động hoàn tiền ngân hàng thật trong demo Sandbox.
NOT_REQUIRED	Dùng cho DIRECT booking và MANUAL_BLOCK: có thể chiếm phòng nhưng không thanh toán qua TravelMate.	Không đưa DIRECT/MANUAL_BLOCK vào commission, revenue online hoặc settlement.
PARTNER_CONFIRMED	Sau VNPAY success: payment APPROVED, booking CONFIRMED, TravelMate đã giữ phòng/căn; partner tiếp nhận booking.	Không gọi là đã check-in/nhận phòng hoặc đã hoàn tất khi Partner chưa thao tác.
UNPAID / PAID_AT_PROPERTY	Với DEPOSIT_30, 70% còn lại do Partner thu tại cơ sở và được theo dõi bằng remainingPaymentStatus.	Không cộng 70% tại cơ sở vào doanh thu online của Admin hoặc ví TravelMate.
Settlement PENDING / PAID	PENDING là đã tạo kỳ quyết toán; PAID là Admin ghi nhận đã chi trả ngoài hệ thống và cộng vào ví nội bộ.	Không nói TravelMate tự chuyển khoản ngân hàng cho Partner.
Withdrawal PENDING / PAID / REJECTED	PENDING giữ số dư; PAID ghi nhận Admin đã xử lý ngoài hệ thống; REJECTED hoàn tiền về available balance.	Không xử lý lại cùng request và không để reject làm mất số dư Partner.
USER_GLOBAL / PARTNER_ROOM	USER_GLOBAL do Admin phát hành dùng toàn hệ thống; PARTNER_ROOM do Admin phát hành để Partner gán vào phòng/căn của mình.	Không nói Partner tự tạo voucher gốc hoặc gán voucher vào phòng của Partner khác.
Voucher ADMIN / PARTNER chịu	ADMIN chịu: không trừ payout Partner. PARTNER chịu: trừ voucher deduction vào settlement.	Không tính sai payout khi không lưu hoặc bỏ qua cost bearer của voucher.

Cách nói ngắn khi bị hỏi tài chính

- Settlement tách hai lớp số liệu: tiền TravelMate thực thu online và tổng đơn gốc trước voucher. FULL_PAYMENT thu 100% online, DEPOSIT_30 thu 30% online và ghi nhận 70% tại cơ sở để đối soát; hoa hồng của cả hai loại booking online đều dựa trên tổng đơn gốc trước voucher.
- Khoản Partner nhận về được tính từ tiền TravelMate đã thu online sau khi tách hoa hồng nền tảng và voucher Partner chịu. Voucher Admin chịu là chi phí nền tảng nên không trừ vào Partner. Với DEPOSIT_30, voucher Partner chịu được phép tối đa 10% tổng đơn gốc; đúng 10% hợp lệ, vượt 10% hoặc làm payout cộc âm thì bị chặn.
- Partner wallet là sổ quyết toán nội bộ, không phải ví điện tử thật. Admin xử lý chuyển khoản ngoài hệ thống rồi ghi nhận trạng thái trong TravelMate.
- Refund trong demo là trạng thái nghiệp vụ: hệ thống ghi nhận chờ hoàn tiền hoặc đã ghi nhận hoàn tiền; chưa tích hợp API refund ngân hàng production.

Các case nên test trước khi demo

- Full payment bị hủy chuyển REFUND_PENDING, Admin ghi nhận thành REFUNDED và quota đã được mở lại.
- Direct booking và manual block có paymentStatus NOT_REQUIRED, chiếm phòng nhưng không xuất hiện trong settlement.
- Deposit 30% sau check-in phải ghi nhận PAID_AT_PROPERTY trước khi hoàn tất nếu rule hiện tại yêu cầu thu 70%.
- Settlement PAID chỉ credit wallet đúng một lần; bấm lại không cộng trùng.
- Withdrawal REJECTED hoàn số dư về available balance; PAID chỉ ghi nhận xử lý ngoài hệ thống.
- Voucher PARTNER chịu bị trừ payout; voucher ADMIN chịu không bị trừ payout Partner.

Trạng thái	Cách hiểu đúng	Câu trả lời khi bị hỏi	Lỗi cần tránh
REFUND_PENDING	Booking/payment đã thanh toán và đang chờ Admin xử lý hoàn tiền ngoài hệ thống.	Demo chỉ ghi nhận yêu cầu hoàn tiền trong hệ thống, chưa tích hợp API refund ngân hàng thật.	Không nói tiền đã tự động về tài khoản khách.
REFUNDED	Admin đã ghi nhận kết quả hoàn tiền trong TravelMate.	Đây là trạng thái nghiệp vụ để đối soát; nếu production cần log giao dịch refund thật.	Không gọi đây là bằng chứng chuyển khoản ngân hàng tự động.
NOT_REQUIRED	Không thanh toán qua TravelMate, dùng cho DIRECT hoặc MANUAL BLOCK.	Direct/block vẫn chiếm phòng để tránh overbooking nhưng không vào revenue/commission/settlement.	Không đưa vào quyết toán online.
PARTNER_CONFIRMED	VNPAY đã thành công, booking được ghi nhận và TravelMate đã giữ phòng/căn; Partner chưa check-in.	Sau thanh toán, Partner đăng nhập để tiếp nhận booking đã được TravelMate giữ thuộc cơ sở mình.	Không nói khách đã check-in hoặc booking đã hoàn tất.
PARTNER_CONFIRMED	TravelMate đã giữ phòng/căn, khách chưa nhất thiết đã nhận phòng.	Partner chỉ thao tác booking thuộc accommodation do mình sở hữu.	Không cho Partner xem/xử lý booking của đối tác khác.
PARTNER_COMPLETED	Partner đã hoàn tất lưu trú/check-out; BookingStatus thường là COMPLETED.	Hoàn tất lưu trú cần đồng bộ trạng thái tổng và trạng thái phía Partner.	Không nhầm với PARTNER_CONFIRMED.
Trạng thái	Cách hiểu đúng	Câu trả lời khi bị hỏi	Lỗi cần tránh
UNPAID	Với DEPOSIT_30, khách mới thanh toán 30% online; 70% còn lại chưa được Partner ghi nhận đã thu tại cơ sở.	Cần theo dõi khoản 70% để Partner biết khách đã hoàn tất nghĩa vụ tại cơ sở trước khi check-out.	Không coi 70% là tiền TravelMate đang giữ.
PAID_AT_PROPERTY	Partner đã xác nhận thu 70% tại cơ sở.	70% là tiền Partner thu trực tiếp, không cộng vào revenue online của Admin.	Không đưa 70% vào settlement như tiền online.

Trạng thái	Cách hiểu đúng	Câu trả lời khi bị hỏi	Lỗi cần tránh
Settlement PENDING	Kỳ quyết toán tháng đã tạo, đã tính gross/commission/payout nhưng chưa ghi nhận chi trả.	Chỉ khi Admin chuyển PAID thì ví nội bộ mới được credit theo thiết kế hiện tại.	Không cộng ví ngay khi mới tạo PENDING.
Settlement PAID	Admin ghi nhận đã chi trả ngoài hệ thống và hệ thống cộng payout vào ví nội bộ đúng một lần.	Partner wallet là sổ quyết toán nội bộ, không phải ví ngân hàng thật.	Không nói TravelMate tự chuyển khoản.
Withdrawal PENDING	Partner gửi yêu cầu rút; số tiền bị giữ khỏi available balance để tránh rút trùng.	Thiếu thông tin ngân hàng hoặc rút vượt số dư thì hệ thống chặn.	Không cho tạo request nếu bank info không hợp lệ.
Withdrawal PAID	Admin xác nhận đã xử lý chuyển khoản ngoài hệ thống.	TravelMate chỉ cập nhật request, pending amount và lịch sử nội bộ.	Không xử lý lại request đã PAID.
Withdrawal REJECTED	Admin từ chối yêu cầu rút; tiền đang giữ được hoàn về available balance.	Reject phải hoàn số dư, nếu không Partner bị mất tiền nội bộ.	Không reject lặp làm cộng tiền trùng.
Khái niệm	Cách hiểu đúng	Câu trả lời khi bị hỏi	Lỗi cần tránh
USER_GLOBAL	Voucher toàn hệ thống do Admin phát hành, User dùng nếu đủ điều kiện.	Không cần Partner gắn phòng.	Không kiểm như voucher theo phòng.
PARTNER_ROOM	Admin phát hành voucher kho; Partner chỉ gắn voucher được phép vào phòng/căn của mình.	Partner hiện không tự tạo voucher gốc trong thiết kế này.	Không cho Partner gắn phòng của người khác.
ADMIN chịu	Voucher là chi phí nền tảng/TravelMate; không trừ payout Partner.	Payout = online amount - commission.	Không trừ voucher ADMIN vào settlement Partner.
PARTNER chịu	Voucher là chi phí của Partner; settlement phải trừ voucher deduction.	Voucher Partner chịu được trừ khỏi khoản Partner nhận. Hoa hồng tính trên tổng đơn gốc trước voucher cho cả FULL_PAYMENT và DEPOSIT_30; với đơn cọc 30%, voucher Partner chịu tối đa 10% tổng đơn gốc và bị chặn nếu làm khoản nhận từ cọc bị âm.	Không bỏ qua cost bearer khi tính payout.

Phần C — Câu hỏi bảo vệ Q001-Q030: Tổng quan, kiến trúc, database và Use Case

Mục này đối chiếu trực tiếp phần F trong file

TravelMate_Bo_kich_ban_nghiep_vu_va_cau_hoi_phan_bien.docx. Trước khi bổ sung, tài liệu đã có nội dung nền nhưng chưa gắn mã Q001-Q030 rõ ràng; phần này giúp nhóm học đủ 160/160 câu theo đúng bảng nguồn.

- Kết luận rà soát: ChatGPT Plus đúng khi gom nhóm câu hỏi theo tổng quan, kiến trúc, database và UML. Điểm cần chỉnh là không nói TravelMate production-ready; chỉ nói demo/local/sandbox có guard nghiệp vụ chính.
- Không cần sửa code, UI hoặc travelmate_db.sql cho nhóm Q001-Q030 vì đây là nhóm câu hỏi giải thích bản chất hệ thống. Source hiện đã có đủ actor, module, entity và dữ liệu mẫu để dẫn chứng.
- Khi bị hỏi bất kỳ câu nào trong nhóm này, trả lời theo cấu trúc: phạm vi đề án -> actor/module liên quan -> file/code dẫn chứng -> giới hạn production nếu có.

C.1 — Tổng quan sản phẩm (Q001-Q008)

ID	Câu hỏi	Câu trả lời nên nói	Dẫn chứng mở nhanh
Q001	TravelMate là gì?	TravelMate là website đặt phòng/lưu trú du lịch dạng OTA-	README_DEMO.md;

		mini cho đồ án: User đặt phòng, Partner vận hành cơ sở/phòng, Admin quản trị và đối soát.	docs/01_SRS.md; BookingService
Q002	Khác gì website giới thiệu khách sạn?	Không chỉ hiển thị thông tin tĩnh; hệ thống có đăng nhập, phân quyền, booking, payment, availability, voucher, workflow Partner/Admin và test.	templates/user; service/*.java; entity/*.java
Q003	Vì sao chọn đề tài này?	Vì đề tài gom nhiều mảng thực tế của web: UI, backend, DB quan hệ, bảo mật, thanh toán sandbox, quản trị và kiểm thử.	pom.xml; docs/*.md; src/test/java
Q004	Hệ thống có actor chính nào?	Có Guest, User/Customer, Partner và Admin. Khi báo cáo rút gọn có thể nhấn mạnh 3 actor vận hành chính: User, Partner, Admin.	SecurityConfig.java; docs/03_UseCase_Diagram.puml
Q005	Tại sao gọi OTA-mini?	Vì TravelMate mô phỏng sản phẩm trung gian: User đặt phòng qua hệ thống, Partner cung cấp phòng, Admin kiểm soát và quyết toán.	BookingService; PartnerPageController; AdminPageController
Q006	Project demo hay production?	Project ở mức demo/local/sandbox. Có guard nghiệp vụ chính nhưng production cần CSRF, migration, secret manager, monitoring, load/load-test và tích hợp tiền thật.	application.properties; SecurityConfig.java; docs/07_Test_Report.md
Q007	Vì sao cần Partner?	Partner là chủ/vận hành cơ sở nên phải quản lý phòng, tiếp nhận booking đã được TravelMate giữ, check-in/check-out. Admin chỉ kiểm soát, duyệt và xử lý ngoại lệ.	PartnerPageController; BookingService; partner templates
Q008	Chức năng quan trọng nhất?	Luồng booking-payment-partner-confirm-settlement là lõi vì nối User, Partner, Admin, DB, trạng thái nghiệp vụ và VNPAY Sandbox.	BookingService; PaymentService; SettlementService

C.2 — Kiến trúc hệ thống (Q009-Q016)

ID	Câu hỏi	Câu trả lời nên nói	Dẫn chứng mở nhanh
Q009	Vì sao dùng Spring Boot MVC + Thymeleaf?	Phù hợp đồ án server-rendered: dễ demo, tích hợp Security/JPA/Validation, không phải tách frontend phức tạp.	pom.xml; controller/page; templates
Q010	REST API hay server-rendered web?	TravelMate chủ yếu là Spring MVC + Thymeleaf server-rendered web. Một số endpoint xử lý form/callback, không phải SPA REST API thuần.	HomeController; PaymentController; templates
Q011	Controller-Service-Repository khác nhau?	Controller nhận request và trả view; Service xử lý nghiệp vụ; Repository truy vấn DB; Entity ánh xạ bảng.	controller; service; repository; entity
Q012	Logic nghiệp vụ nằm ở đâu?	Logic chính nằm trong Service, có @Transactional, kiểm tra trạng thái, quyền sở hữu, availability, payment và settlement.	BookingService; AvailabilityService; SettlementService
Q013	Nếu tách frontend/backend thì đổi gì?	Giữ Service/Repository, thêm REST DTO/API, auth token/session phù hợp, CORS, validation response và frontend gọi API thay vì Thymeleaf.	service layer; SecurityConfig; controller
Q014	Monolithic có nhược điểm gì?	Dễ phát triển/demo nhưng khi lớn sẽ khó scale từng module riêng, deploy một khối và dễ phụ thuộc chéo nếu không giữ layer rõ.	pom.xml; package com.travelmate
Q015	Maven dùng để làm gì?	Maven quản lý dependency, build, test, đóng gói và chạy lifecycle thống nhất trên máy khác.	pom.xml; mvnw
Q016	Tại sao dùng MySQL?	Dữ liệu booking/payment/settlement có quan hệ và cần transaction; MySQL phù hợp demo đồ án và dễ import travelmate_db.sql.	travelmate_db.sql; application.properties

C.3 — Database design (Q017-Q024)

ID	Câu hỏi	Câu trả lời nên nói	Dẫn chứng mở nhanh
Q017	Các bảng chính là gì?	Nhóm chính gồm users, accommodations, rooms, bookings, payments, vouchers, reviews/support, settlements, partner_wallets, withdrawals và transaction history.	travelmate_db.sql; entity/*.java
Q018	Quan hệ User-Booking-Payment?	User tạo nhiều booking; booking lưu thông tin đặt phòng và snapshot tài chính; payment ghi nhận giao dịch thanh toán của booking.	User.java; Booking.java; Payment.java
Q019	Accommodation và Room khác nhau?	Accommodation là cơ sở lưu trú; Room là đơn vị phòng có giá, sức chứa, số lượng và được đặt theo ngày.	Accommodation.java; Room.java
Q020	Vì sao khóa chính là id?	id dạng số giúp JPA mapping và quan hệ FK ổn định; mã hiển thị như booking code có thể tách riêng để người dùng dễ đọc.	entity id fields; travelmate_db.sql
Q021	Vì sao cần snapshot tài chính trong Booking?	Để giữ nguyên tiền phòng, voucher, commission, payout tại thời điểm đặt; giá/voucher thay đổi sau này không làm sai booking cũ.	Booking.java; BookingService; SettlementService
Q022	Vì sao Payment riêng Booking?	Booking là đặt chỗ; Payment là giao dịch có method/status/gateway transaction/callback. Tách riêng giúp xử lý fail, retry, IPN và đối soát.	Payment.java; PaymentService; PaymentController

Q023	Direct booking có payment không?	Direct booking/manual block thường có paymentStatus NOT_REQUIRED; mục tiêu là giữ quota/vận hành nội bộ, không tạo VNPAY hoặc settlement online.	BookingSource; BookingService; travelmate_db.sql
Q024	partner_wallets dùng làm gì?	Là sổ quyết toán nội bộ cho Partner: sổ dư, khoản đang chờ rút, lịch sử giao dịch và yêu cầu rút tiền; không phải ví điện tử thật.	PartnerWalletService; partner_wallets; partner_wallet_transactions

C.4 — Use Case/UML nền tảng (Q025-Q030)

ID	Câu hỏi	Câu trả lời nên nói	Dẫn chứng mở nhanh
Q025	Use Case tổng quát có 3 actor chính nào?	Ba actor vận hành chính là User, Partner, Admin. Guest có thể được vẽ riêng cho chức năng xem/tìm kiếm public.	docs/03_UseCase_Diagram.puml; SecurityConfig
Q026	Include và extend dùng ở đâu?	Include dùng cho bước bắt buộc như đăng nhập/kiểm tra phòng/thanh toán; extend dùng cho nhánh tùy điều kiện như voucher, review, hủy/hoàn tiền.	Use Case diagram; BookingService
Q027	Vì sao không vẽ actor tương tác trực tiếp?	Use Case thể hiện actor tương tác với hệ thống qua chức năng; quan hệ User-Partner-Admin diễn ra thông qua TravelMate, không vẽ như chat trực tiếp giữa actor.	docs/03_UseCase_Diagram.puml
Q028	Luồng booking nên thể hiện bằng diagram nào?	Use Case để nói chức năng; Activity để nói các bước/rẽ nhánh; Sequence để nói thứ tự gọi Controller-Service-Repository-VNPAY.	docs/*.puml; BookingService; PaymentController
Q029	Use Case khác Activity Diagram thế nào?	Use Case trả lời ai dùng hệ thống để làm gì; Activity trả lời quy trình chạy qua các bước và điều kiện như thế nào.	docs/03_UseCase_Diagram.puml; docs/06_Activity_*.puml
Q030	Nếu bỏ Guest khỏi Use Case có sai không?	Không sai nếu phạm vi báo cáo chỉ xét user đã đăng nhập, nhưng với TravelMate nên giữ/nhắc Guest vì có xem trang chủ, search, chi tiết và bài viết public.	HomeController; public templates

C.5 — Checklist học nhanh Q001-Q030

- Thuộc một câu chốt: TravelMate là OTA-mini demo/local/sandbox, không phải production-ready.
- Mở được các file dẫn chứng nhanh: pom.xml, application.properties, SecurityConfig.java, BookingService.java, PaymentService.java, travelmate_db.sql và docs/03_UseCase_Diagram.puml.
- Khi bị hỏi kiến trúc, nói đúng layered architecture: Controller -> Service -> Repository -> Entity/DB -> Thymeleaf view.
- Khi bị hỏi database, nhấn mạnh Booking lưu snapshot tài chính và Payment tách riêng để xử lý gateway/callback/trạng thái.
- Khi bị hỏi UML, phân biệt Use Case cho chức năng, Activity cho quy trình và Sequence cho thứ tự gọi kỹ thuật.

Phần D — Payment/VNPAY và Partner workflow (TC031-TC060)

Phần này đối chiếu trực tiếp source hiện tại với gợi ý ChatGPT Plus cho nhóm TC031-TC060. Kết luận: hướng ưu tiên giữ core ổn định là đúng; các luồng payment, partner workflow, direct/manual block và settlement đã có guard chính. Bổ sung nhỏ đã thực hiện: Return URL VNPAY báo success tới muộn sau khi payment đã EXPIRED/CANCELLED/FAILED sẽ không còn hiển thị thành công giả.

Phạm vi đã kiểm tra: BookingService, PaymentService, PaymentController, PartnerPageController, AccommodationService, FileStorageService, RevenueService, SettlementService, travelmate_db.sql và các test liên quan. Kết quả chạy lại nhóm phần D: 103 tests pass, 0 fail, 0 error.

D.1. Bảng chốt nhanh TC031-TC060

Nhóm case	Kết luận đúng để học	File/test chứng minh	Demo/né khi bảo vệ
TC031-TC033	FULL_PAYMENT thu 100% online, không cần thu thêm. DEPOSIT_30 thu 30% online, 70% tại cơ sở. Sau VNPAY success booking CONFIRMED nhưng TravelMate đã giữ phòng/căn; Partner tiếp nhận	BookingService, PaymentServiceTest, templates/user/payment-result.html	Nên demo một flow full payment hoặc deposit 30% đã test chắc.

	booking.		
TC034-TC040	Gateway timeout/cancel/expired/sai chữ ký/sai amount phải không xác nhận booking. Voucher làm tiền quá thấp bị chặn tối thiểu VNPAY 5.000đ. Return success tới muộn sau EXPIRED/CANCELLED không hiển thị thành công giả.	PaymentController, VnpayService, PaymentServiceTest, PaymentControllerTest	Không demo timeout/callback trễ bằng UI; chỉ giải thích.
TC041-TC048	Partner tạo accommodation/phòng theo owner và property type; upload ảnh validate JPG/PNG/WEBP và dung lượng; Partner chỉ xem/xác nhận booking thuộc cơ sở mình.	AccommodationService, FileStorageServiceTest, BookingService.validatePartnerOwnership	Nên demo Partner booking list và check-in/check-out.
TC049	Partner báo không thể tiếp nhận khách là ngoại lệ; hệ thống đánh dấu PARTNER_CANCELLED để Admin xử lý hủy/hoàn tiền ngoài hệ thống.	BookingService.cancelByPartner, handlePartnerCancelledByAdmin, admin/booking-detail.html	Không demo nếu chưa cần; chỉ dùng để trả lời phản biện.
TC050-TC053	Full payment check-in không cần thu thêm. Deposit 30% phải xác nhận đã thu 70% tại cơ sở trước khi đi tiếp. Check-out chuyển COMPLETED và đủ điều kiện review/settlement.	BookingCalculationTest, PartnerPageController, partner/booking-detail.html	Có thể demo nếu dữ liệu seed đang rõ trạng thái.
TC054-TC055	No-show chỉ nên dùng luồng DEPOSIT_30: giữ cọc, mở lại quota. Full-payment no-show cần chính sách hoàn tiền riêng nên không demo trực tiếp.	NoShowDepositTest, BookingService.markNoShowByPartner	Nên nói full-payment no-show cần Admin xử lý theo chính sách.
TC056-TC058	Direct booking và manual block chiếm availability nhưng paymentStatus NOT_REQUIRED, không qua VNPAY, không commission, không settlement.	travelmate_db.sql, SettlementEligibilityTest, RevenueServiceHomestayTest	Nên mở màn Partner/Admin để chứng minh không phải doanh thu online.
TC059-TC060	Revenue/settlement chỉ đưa tiền TravelMate thu online vào payout: 100% với FULL_PAYMENT, 30% với DEPOSIT_30, cọc bị giữ nếu no-show/hủy hợp lệ; riêng commission snapshot của DEPOSIT_30 tính trên tổng đơn gốc. Partner chỉ xem settlement Admin tạo.	RevenueService, SettlementServiceTest, PartnerWalletServiceTest	Nên demo revenue/settlement nếu số liệu seed đã import đúng.

D.2. Những điểm ChatGPT Plus nói đúng và điểm đã bổ sung

- Đúng: không nên refactor lớn khi project đã ổn; chỉ kiểm tra guard nghiệp vụ và wording dễ gây hiểu nhầm.
- Đúng: DEPOSIT_30 không cộng 70% tại cơ sở vào revenue/settlement online; TravelMate chỉ tính trên tiền online đã thu.
- Đúng: Partner ownership là P0; source đã kiểm tra owner và partnerPropertyType ở service/controller, không chỉ ẩn nút trên UI.
- Đúng: Direct booking/manual block là nghiệp vụ quản lý quota, không phải doanh thu TravelMate.
- Bổ sung đã fix: Return URL VNPAY success tới muộn sau khi payment đã EXPIRED/CANCELLED/FAILED sẽ hiển thị lỗi trạng thái cũ, không báo thanh toán thành công giả.

D.3. Câu trả lời tổng hợp cần thuộc

Sau khi khách thanh toán VNPAY thành công, TravelMate xác nhận Payment APPROVED và Booking CONFIRMED, và TravelMate đã giữ phòng/căn trên hệ thống trước khi Partner check-in. Với FULL_PAYMENT, khách đã trả 100% online

nên không cần thu thêm tại cơ sở. Với DEPOSIT_30, TravelMate chỉ thu 30% online, còn 70% Partner thu trực tiếp tại cơ sở và hệ thống chỉ ghi nhận trạng thái PAID_AT_PROPERTY để vận hành; 70% này không vào revenue/settlement online. Direct booking và manual block chỉ giữ quota để tránh overbooking, không tạo commission hoặc settlement. Settlement chỉ tính booking online đủ điều kiện như COMPLETED hoặc cọc bị giữ do hủy/no-show.

D.4. Dữ liệu/màn hình nên mở để demo trực quan

- User flow: tạo booking mới bằng FULL_PAYMENT hoặc DEPOSIT_30 rồi xem /payment/vnpay-return và /my-bookings.
- Partner flow: /partner/bookings để thấy đơn đã được TravelMate giữ, đơn đã check-in, đơn direct và manual block.
- Admin flow: /admin/bookings để xem partner_status, refund/no-show và nhấn DIRECT/MANUAL_BLOCK không tính doanh thu online.
- Revenue/settlement: /partner/revenue, /admin/revenue, /admin/settlements; nhấn mạnh công thức payout = tiền online - commission - voucher Partner chịu, riêng DEPOSIT_30 commission snapshot lấy theo tổng đơn gốc.
- Seed có các mã để mở: BK-LATA-STD-0001 hoặc BK-HLR-STD-0002 cho cọc/chờ Partner, BK-TLP-STD-0001 cho no-show mất cọc, các mã DIRECT/MANUAL_BLOCK trong travelmate_db.sql để chứng minh không settlement.

D.5. Case nên tránh demo trực tiếp

- Gateway timeout, callback trễ hoặc IPN fail đến sau: khó tạo ổn định bằng UI, chỉ dùng để trả lời kỹ thuật.
- Voucher giảm tiền xuống quá thấp: đã có guard tối thiểu VNPAY nhưng không nên làm trong demo chính.
- Partner báo không thể tiếp nhận khách và full-payment no-show: dễ kéo sang chính sách refund production, nên nói là ngoại lệ Admin xử lý.
- Refund/chuyển khoản ngân hàng thật: hiện chỉ là trạng thái đối soát trong hệ thống demo, chưa tích hợp API refund/chuyển khoản production.

D.6. Checklist sau fix phần D

- Đã chạy nhóm test phần D: PaymentServiceTest, PaymentControllerTest, BookingCalculationTest, NoShowDepositTest, RevenueServiceHomestayTest, SettlementEligibilityTest, SettlementServiceTest, PartnerWalletServiceTest, UserPortalSqlSeedTest, FileStorageServiceTest — 103/103 pass.
- PaymentController đã có guard không hiển thị success nếu Return URL thành công tới sau trạng thái EXPIRED/CANCELLED/FAILED.
- travelmate_db.sql hiện đã có dữ liệu đủ cho full payment, cọc 30%, pending partner confirmation, no-show, direct booking, manual block, revenue/settlement/wallet; chưa cần thêm seed mới cho phần này.
- Không chỉnh BaoCaoTravelMate_1.docx trong phần này để tránh lệch wording/UI báo cáo đã chốt.

Phần E — Kịch bản demo nên trình bày trước giảng viên

Phần này chốt cách demo thực tế cho TravelMate. Thứ tự nên đi từ luồng an toàn đến luồng nghiệp vụ mạnh và cuối cùng là quản trị/doanh thu. Khi trình bày, nhóm không nên phụ thuộc hoàn toàn vào mạng hoặc gateway; luôn chuẩn bị dữ liệu seed để mở trực quan.

E.1. Demo 1 — Flow ngắn, an toàn, ít lỗi

Mục	Nội dung
Mục tiêu	Chứng minh hệ thống có dữ liệu thật, UI chạy ổn, search/chỉ tiết lưu trú hoạt động và phân quyền cơ bản rõ ràng.
Account	Guest và user@travelmate.vn / user123.
Steps	Guest mở trang chủ -> tìm Đà Lạt -> mở chi tiết nơi lưu trú -> đăng nhập User -> mở my-bookings -> mở chatbot hoặc travel post nếu cần.
Nói gì	TravelMate là MVC server-rendered; dữ liệu không phải HTML tĩnh, lấy từ Controller/Service/Repository/MySQL.
Rủi ro	Ít phụ thuộc VNPAY; phù hợp khi mạng không ổn.

E.2. Demo 2 — Flow nghiệp vụ mạnh nhất

Mục	Nội dung
Mục tiêu	Chứng minh luồng User đặt phòng -> VNPAY -> TravelMate giữ phòng/căn -> Partner check-in/check-out -> review/settlement.
Account	user@travelmate.vn / user123; partner@travelmate.vn / partner123; admin@travelmate.vn / admin123 nếu cần đối soát.
Steps	User chọn phòng còn trống -> chọn FULL_PAYMENT hoặc DEPOSIT_30 -> thanh toán VNPAY success nếu môi trường sẵn sàng -> TravelMate giữ phòng/căn -> Partner mở /partner/bookings -> check-in -> check-out.
Chọn flow	FULL_PAYMENT gọn và ít lỗi; DEPOSIT_30 mạnh hơn vì chứng minh 70% trả tại cơ sở và không vào revenue online.
Không demo	Không demo full-payment no-show hoặc refund ngân hàng thật nếu chưa có chính sách/API production rõ ràng.

E.3. Demo 3 — Flow quản trị, doanh thu và đối soát

Mục	Nội dung
Mục tiêu	Chứng minh hệ thống không chỉ CRUD: có revenue, settlement, wallet, withdrawal và đối soát.
Account	admin@travelmate.vn / admin123; partner demo có ví/quyết toán trong SQL.
Steps	Admin mở revenue -> settlements -> tạo settlement tháng trước nếu chưa có -> mở detail/export Excel -> mark paid -> Partner xem wallet/settlement.
Nói gì	Payout = tiền online TravelMate thu - commission - voucher Partner chịu. DIRECT/MANUAL BLOCK không tạo commission/settlement.
Rủi ro	Nếu seed chưa import đủ, chỉ mở màn hình đã có dữ liệu và giải thích theo source/test.

E.4. Thứ tự demo khuyến nghị

- Nếu thời gian ngắn hoặc mạng không ổn: đi Demo 1 trước, sau đó mở vài màn admin/partner có dữ liệu seed.
- Nếu máy demo đã cấu hình VNPAY ổn: đi Demo 1 -> Demo 2 -> Demo 3. Demo 2 là phần dễ gây ấn tượng nhất nhưng cũng có rủi ro gateway cao nhất.
- Nếu giảng viên hỏi sâu production: trả lời trung thực rằng VNPAY là Sandbox, wallet là sổ nội bộ, refund/chuyển khoản ngân hàng thật chưa tự động hóa, CSRF/secret/migration cần nâng cấp.

E.5. Checklist trước khi vào phòng bảo vệ

- Import travelmate_db.sql trên máy demo và đăng nhập được user/admin/partner.
- Mở sẵn một tab guest search Đà Lạt, một tab my-bookings, một tab partner bookings và một tab admin settlements/withdrawals.
- Chuẩn bị câu trả lời fallback nếu VNPAY, Groq, SMTP hoặc Google OAuth không có mạng/cấu hình.
- Không nói hệ thống production-ready; nói hệ thống đạt mức demo đồ án với guard nghiệp vụ chính và có hướng nâng cấp production.

E.6. Kết luận ngắn cần thuộc

Kịch bản demo tốt nhất là đi từ an toàn đến phức tạp: trước hết chứng minh dữ liệu và UI, sau đó chứng minh booking/payment/partner workflow, cuối cùng chứng minh quản trị doanh thu/quyết toán. Khi bị hỏi về giới hạn, trả lời trung thực theo phạm vi demo local/sandbox.

Phần F — Câu hỏi bảo vệ Q031-Q056: Use Case, Booking, VNPAY và cọc 30%

Phần này được bổ sung cho nhóm câu hỏi Q031-Q056. Mục tiêu là học bảo vệ theo đúng source hiện tại, không nói quá phạm vi demo và không kéo project sang production thật.

- Kết luận rà soát: gợi ý ChatGPT Plus đúng hướng với flow Booking/Payment/VNPAY/cọc 30%.
- Điểm đã chỉnh thêm cho sát thực tế: Admin override no-show không còn tự chuyển đơn FULL_PAYMENT sang CHECKED_IN; no-show tự động chỉ áp dụng cho DEPOSIT_30. Review dùng thang 1-10, email xác

nhận booking gửi sau VNPAY success và lỗi mail không rollback booking; voucher Partner chịu cho cọc được phép tối đa 10% tổng đơn gốc; đúng 10% hợp lệ, vượt 10% hoặc làm payout cọc âm thì bị chặn.

- DB hiện đã đủ seed cho full payment, cọc 30%, pending partner confirmation, no-show, direct/manual block, revenue/settlement; chưa cần thêm dữ liệu mới cho nhóm Q031-Q056.
- Flow cần học: PENDING_PAYMENT -> Payment APPROVED / Booking CONFIRMED -> TravelMate giữ phòng/căn -> Partner CHECKED_IN -> COMPLETED.

F.1 — Use Case/UML (Q031-Q032)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q031	Use Case quan trọng nhất là Đặt phòng, vì nó nói User chọn nơi lưu trú/phòng, availability, booking, VNPAY, TravelMate giữ phòng, Partner check-in/check-out, review và settlement.	Báo cáo Use Case; BookingService; PaymentService; AvailabilityService; SettlementService	Không nói đăng nhập là nghiệp vụ quan trọng nhất. Đăng nhập là nền tảng, không phải lỗi OTA-mini.
Q032	Cần Sequence Booking/Payment vì Use Case chỉ nói actor dùng chức năng gì, còn sequence thể hiện thứ tự User tạo booking -> tạo payment -> VNPAY -> callback/return -> cập nhật trạng thái.	BookingPageController; BookingService; PaymentController; PaymentService; VnpayService	Không nhầm Use Case với Sequence. Use Case là chức năng; Sequence là tương tác theo thời gian.

Câu tổng hợp: Đặt phòng là luồng trung tâm của TravelMate. Use Case giúp nhìn chức năng theo actor, còn Sequence/Activity cần dùng cho booking-payment vì luồng này có nhiều trạng thái và có hệ thống ngoài là VNPAY.

F.2 — Booking Flow (Q033-Q040)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q033	Booking online mới tạo ở PENDING_PAYMENT; Payment tương ứng cũng PENDING_PAYMENT vì đang chờ khách thanh toán qua VNPAY.	BookingService.createBooking; BookingStatus; PaymentStatus	Không nói flow chính mới tạo là PENDING_ADMIN_APPROVAL. Trạng thái đó chỉ là ngoại lệ/dối soát.
Q034	VNPAY success hợp lệ làm Payment APPROVED, Booking CONFIRMED và PartnerStatus PARTNER_CONFIRMED; TravelMate tự giữ phòng/căn để Partner tiếp nhận/check-in.	PaymentService.markGatewaySuccess; PaymentController	Không nói VNPAY success nghĩa là khách đã check-in.
Q035	CONFIRMED chưa phải đã ở. Nó chỉ nghĩa là booking hợp lệ sau thanh toán. CHECKED_IN mới là khách đã đến nhận phòng/căn.	BookingStatus; BookingService.checkInByPartner	Không dịch CONFIRMED thành đã hoàn tất hoặc đã lưu trú.
Q036	TravelMate tự giữ phòng/căn sau thanh toán hợp lệ. Partner tiếp nhận đơn để vận hành check-in/check-out, không xác nhận cọc online.	BookingService.confirmBookingHoldByPartner; PartnerPageController	Không nói Partner nhận tiền online. Tiền online do TravelMate/VNPAY ghi nhận.
Q037	Check-in là khách đến nhận phòng và booking chuyển CHECKED_IN. Check-out/complete là khách trả phòng, booking chuyển COMPLETED.	BookingService.checkInByPartner; markCompletedByPartner	Không cho review ngay sau check-in.
Q038	Booking COMPLETED khi khách đã hoàn tất lưu trú và Partner/Admin check-out/complete. Đây là trạng	BookingService.markCompletedByPartner; SettlementService; ReviewService	Không đưa booking mới CONFIRMED vào settlement.

	thái phù hợp cho review và settlement.		
Q039	User chỉ review được sau khi booking COMPLETED và mỗi booking chỉ review một lần.	ReviewService.createReview; ReviewRepository.existsByBookingId	Không cho review booking pending, cancelled, no-show hoặc chưa lưu trữ xong.
Q040	Booking hết hạn thanh toán được chuyển khỏi PENDING_PAYMENT, payment EXPIRED/CANCELLED tùy flow và quota phòng được mở lại để tránh giữ phòng ảo.	PaymentService.expirePayment; BookingService.expirePendingPaymentByAdmin	Không để booking pending vô thời hạn.

Câu tổng hợp: User tạo booking -> PENDING_PAYMENT -> VNPAY success -> Payment APPROVED, Booking CONFIRMED -> TravelMate giữ phòng/căn -> Partner CHECKED_IN -> COMPLETED. CONFIRMED không phải CHECKED_IN.

F.3 — Payment/VNPAY (Q041-Q048)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q041	VNPAY Sandbox là môi trường thử nghiệm thanh toán, giúp demo redirect, mã giao dịch và kết quả trả về nhưng không phát sinh tiền thật.	VnpayService; VnpayConfigProperties; application.properties	Không nói đã chạy merchant production hoặc thanh toán tiền thật.
Q042	Return URL là trình duyệt user quay về để hiển thị kết quả. IPN là callback server-to-server, phù hợp đối soát backend hơn trong production.	PaymentController.vnpayReturn; PaymentController.vnpayIpn	Không phụ thuộc hoàn toàn vào user quay lại trình duyệt nếu nói production.
Q043	Verify chữ ký để bảo đảm params thật từ VNPAY và không bị sửa amount, transaction status hoặc order id.	VnpayService.verifySignature; PaymentController	Không tin query params nếu chưa verify.
Q044	Phải validate amount vì số tiền trả về phải khớp Payment.amount đã tạo từ DB. Success code thôi chưa đủ.	VnpayService.isAmountMatched; PaymentController	Không nói VNPAY trả success là tự approve ngay.
Q045	Callback/return có thể bị gọi trùng. PaymentService xử lý idempotent; payment đã APPROVED thì không cộng tiền hoặc downgrade lại. Return success tới muộn sau EXPIRED/CANCELLED/FAILED cũng không hiển thị thành công giả.	PaymentService.markGatewaySuccess/markGatewayFailed; PaymentController.addStalePaymentResult; PaymentControllerTest	Không cập nhật trạng thái theo mọi callback đến sau.
Q046	Nếu user đóng trình duyệt trước return, booking/payment vẫn PENDING_PAYMENT cho đến khi nhận callback/return hợp lệ hoặc hết hạn. Nếu còn pending, user có thể kiểm tra ở Đặt phòng của tôi/thanh toán lại.	PaymentController.payBooking; mybooking.html	Không nói mất booking ngay khi đóng trình duyệt.
Q047	Payment APPROVED là trạng thái giao dịch tiền. Booking CONFIRMED là trạng thái đơn đặt phòng hợp lệ. Hai trạng thái liên quan nhưng không thay thế nhau.	PaymentStatus; BookingStatus; Payment.java; Booking.java	Không dùng payment status và booking status lẫn nhau.
Q048	Không nói thanh toán thật vì project dùng VNPAY Sandbox. Đồ án chứng minh flow kỹ thuật, chưa xử lý refund/đối soát ngân hàng production.	VnpayConfigProperties; README_DEMO; docs/07_Test_Report.md	Không nói hệ thống tự hoàn tiền ngân hàng thật.

Câu tổng hợp: TravelMate tạo payment, chuyển user sang VNPAY Sandbox, nhận return/IPN, verify chữ ký, validate amount rồi mới cập nhật payment/booking. Sandbox chứng minh nghiệp vụ kỹ thuật, không phải giao dịch tiền thật.

F.4 — Cọc 30% và Full Payment (Q049-Q056)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q049	FULL_PAYMENT là thanh toán 100% online. DEPOSIT_30 là thanh toán 30% online để giữ phòng, 70% còn lại trả tại cơ sở.	PaymentOption; BookingService.calculatePaidAmount	Không nói cọc 30% là đã thanh toán đủ.
Q050	70% còn lại do khách trả trực tiếp cho Partner tại cơ sở. TravelMate chỉ ghi nhận trạng thái thu tại cơ sở để vận hành.	RemainingPaymentStatus; BookingService.confirmRemainingPaymentByPartner	Không nói TravelMate/VNPAY thu 70% này.
Q051	70% không vào doanh thu online TravelMate. Với cọc 30%, TravelMate chỉ tính khoản online 30% đã thu.	RevenueService; SettlementService; partner/admin revenue templates	Không cộng 100% vào doanh thu online nếu chỉ thu 30%.
Q052	No-show tự động áp dụng rõ cho DEPOSIT_30: booking NO_SHOW, payment DEPOSIT_FORFEITED, giữ cọc và mở lại quota. FULL_PAYMENT no-show không tự xử lý trong demo; Admin xử lý theo chính sách riêng.	BookingService.markNoShowByPartner; BookingService.markNoShow; NoShowDepositTest	Không demo full-payment no-show như case mất cọc hoặc check-in giả.
Q053	Đơn cọc 30% phải xác nhận đã thu 70% tại cơ sở trước check-in/check-out trong Partner flow.	checkInWithRemainingConfirmByPartner; confirmRemainingPaymentByPartner; RemainingPaymentStatus	Không cho hiểu 70% được TravelMate thu online.
Q054	Full payment hủy là case refund theo chính sách. Code ghi nhận REFUND_PENDING/REFUNDED, hoàn dự kiến 70% và giữ 30% phí hủy; không chuyển tiền ngân hàng thật.	BookingService.cancelBooking; markRefundedByAdmin; PaymentStatus.REFUND_PENDING/REFUNDED	Không nói hệ thống tự refund production.
Q055	Cọc 30% hủy sau khi đã thanh toán có thể CANCELLED + DEPOSIT_FORFEITED; 70% chưa thu nên không phát sinh.	BookingService.cancelBooking; PaymentStatus.DEPOSIT_FORFEITED	Không nhầm hủy với no-show: hủy là CANCELLED, không đến là NO_SHOW.
Q056	Commission/settlement tính trên tiền TravelMate thu online: full payment là 100%, cọc là 30%, phần 70% tại cơ sở không vào settlement online.	SettlementService; RevenueService; SettlementServiceTest	Không tính hoa hồng trên 70% nếu TravelMate không thu khoản đó.

Câu tổng hợp: TravelMate tách rõ tiền online và tiền tại cơ sở. Full payment là 100% online; cọc 30% chỉ là phần TravelMate giữ qua VNPAY, còn 70% Partner thu trực tiếp. Payout vẫn dựa trên khoản online TravelMate giữ; riêng commission snapshot của DEPOSIT_30 tính trên tổng đơn gốc.

F.5 — Câu hỏi xoắn để gặp

Câu hỏi xoắn	Cách trả lời an toàn
Nếu booking CONFIRMED rồi Partner còn thao tác gì?	CONFIRMED là hệ thống đã ghi nhận booking sau thanh toán. TravelMate đã giữ phòng/căn sau thanh toán hợp lệ; Partner tiếp nhận và xử lý check-in/check-out.
Nếu User đóng trình duyệt, tiền đã trừ nhưng hệ thống chưa update thì sao?	Production nên ưu tiên IPN/server callback. Trong đồ án, hệ thống có return/IPN, pending/expire và đối soát để kiểm soát trạng thái.
Tại sao cọc 30% không tính 100% doanh thu?	Vì TravelMate chỉ thu 30% online. 70% Partner thu tại cơ sở, không phải tiền hệ thống quản lý.
Full payment no-show xử lý sao?	Không demo tự động. Đây là chính sách refund/ giữ phí phức tạp; hiện hệ thống không chuyển check-in giả và để Admin xử lý theo chính sách riêng.
Thanh toán sandbox có giá trị gì nếu không phải tiền thật?	Nó chứng minh flow kỹ thuật: tạo giao dịch, redirect, verify chữ ký, validate amount, cập nhật booking/payment.
Nếu callback gọi hai lần thì sao?	Service idempotent: payment đã xử lý thì không cộng tiền lại, không tạo booking mới và không downgrade trạng thái.

F.6 — Checklist học nhanh Q031-Q056

- Học kỹ các câu có xác suất bị hỏi cao: Q033, Q034, Q035, Q041, Q045, Q049, Q051, Q052, Q056.
- Khi demo, ưu tiên luồng thành công và cọc 30% có dữ liệu seed; không demo refund ngân hàng thất hoặc full-payment no-show.
- Mở sẵn file dẫn chứng: BookingService.java, PaymentService.java, PaymentController.java, ReviewService.java, SettlementService.java, NoShowDepositTest.java.
- Trả lời đúng phạm vi: VNPAY Sandbox, settlement nội bộ, Partner thu 70% tại cơ sở, Admin xử lý ngoại lệ ngoài hệ thống khi production chưa tích hợp.

Phần G — Câu hỏi bảo vệ Q057-Q083: Voucher, Availability, Partner và Admin workflow

Phần này bổ sung cho nhóm câu hỏi Q057-Q083. Nội dung đã được đối chiếu với VoucherService, AvailabilityService, BookingService, PartnerPageController, AdminPageController và các test hiện có.

- Kết luận rà soát: gợi ý ChatGPT Plus đúng hướng ở voucher, partner workflow và admin workflow.
- Điểm cần chỉnh theo source: AvailabilityService chỉ tính PENDING_PAYMENT, PENDING_ADMIN_APPROVAL, CONFIRMED và CHECKED_IN là trạng thái giữ phòng; CANCELLED, NO_SHOW, COMPLETED không còn giữ phòng sau khi đã xử lý.
- Voucher hiện không có usage limit trong source; khi bảo vệ chỉ nói backend kiểm tra active/date/min order/scope/property type/room assignment/max discount/VNPAY minimum, không nói số lượt dùng nếu chưa có trường dữ liệu.
- DB seed hiện đã đủ: USER_GLOBAL, PARTNER_ROOM, room_voucher_assignments, direct booking, manual block, partner ownership và admin approval; chưa cần thêm data mới.

G.1 — Voucher (Q057-Q064)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q057	Voucher scope xác định phạm vi áp dụng. Source hiện có USER_GLOBAL cho voucher toàn hệ thống và PARTNER_ROOM cho voucher Admin đưa vào kho để Partner gắn vào phòng/căn đã duyệt. PARTNER_ACCOMMODATION là scope cũ/legacy, không tạo mới từ UI.	VoucherScope.java; VoucherService; room_voucher_assignments	Không nói voucher nào cũng dùng cho mọi phòng hoặc mọi cơ sở.
Q058	Cost bearer cho biết ai chịu chi phí giảm giá. ADMIN nghĩa là TravelMate/Admin chịu, không trừ payout Partner. PARTNER nghĩa là khoản giảm được trừ vào settlement/payout của Partner.	VoucherCostBearer.java; SettlementService; SettlementServiceTest	Không nói voucher chỉ là giảm giá giao diện, không ảnh hưởng tài chính.
Q059	Partner không tự tạo voucher mới. Admin tạo voucher; Partner chỉ gắn voucher PARTNER_ROOM được cấp vào phòng/căn thuộc quyền quản lý và đã Admin duyệt.	VoucherService.createAdminVoucher; assignVoucherToRoom; PartnerPageController	Không nói Partner toàn quyền tự phát hành mã giảm giá.
Q060	Khi User áp dụng voucher, backend kiểm tra active, ngày hiệu lực, min order, scope, loại lưu trú, room assignment và tính discount. Source hiện chưa có usage limit nên không tự nói có kiểm lượt dùng.	VoucherService.validateVoucher; VoucherApiController; VoucherCalculationTest	Không chỉ kiểm tra voucher ở frontend; không nói có usage limit nếu source chưa có.
Q061	Voucher PARTNER_ROOM chỉ	VoucherService.validateVoucher;	Không nói nhập đúng code là

	hợp lệ khi Partner đã gán voucher vào đúng phòng/căn đang chọn và đúng property type. Nếu sai room/sai loại lưu trữ, backend từ chối.	RoomVoucherAssignmentRepository	dùng được.
Q062	Mìn order và max discount kiểm ở backend. Frontend chỉ hỗ trợ hiển thị; User có thể sửa dữ liệu phía client nên BookingService/VoucherService vẫn phải tính lại từ DB.	VoucherService.calculateDiscount; BookingService.createBooking	Không nói JavaScript kiểm là đủ.
Q063	Nếu voucher làm tiền thanh toán quá thấp, hệ thống giữ mức thanh toán VNPAY tối thiểu 5.000đ; nếu paidAmount sau giảm vẫn thấp hơn mức này thì BookingService từ chối tạo payment.	VoucherService.calculateDiscount; BookingService.createBooking	Không cho booking 0đ đi qua VNPAY nếu flow chưa hỗ trợ.
Q064	Settlement phụ thuộc cost bearer: Admin chịu thì không trừ Partner; Partner chịu thì payout = online amount - commission - partner voucher deduction.	SettlementService; Booking financial snapshot; SettlementServiceTest	Không trừ voucher Admin chịu khỏi payout Partner.

Câu tổng hợp: Voucher của TravelMate là nghiệp vụ backend và tài chính, không chỉ là mã giảm giá trên UI. Scope quyết định voucher dùng ở đâu, cost bearer quyết định ai chịu chi phí trong settlement.

G.2 — Availability / Overbooking (Q065-Q072)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q065	Availability tính theo khoảng ngày. Source có availableQuantity hiện tại và cộng lại active bookings để suy ra platform quantity, sau đó trừ các booking overlap trong khoảng ngày đang hỏi.	AvailabilityService; BookingRepository.sumOverlappingBookingsByRoom; BookingService.createBooking	Không nói cứ đặt là trừ stock vĩnh viễn. Quota được trả lại khi cancel/no-show/checkout theo flow.
Q066	Overlap dùng điều kiện $oldCheckIn < newCheckOut$ và $oldCheckOut > newCheckIn$. Nếu khách A checkout đúng ngày khách B check-in thì không overlap.	BookingRepository overlap query; AvailabilityOverlapTest	Không dùng $\leq$ làm checkout/checkin cùng ngày bị khóa sai.
Q067	Hai user đặt slot cuối cùng lúc là bài toán concurrency. Source đã dùng @Transactional và PESSIMISTIC_WRITE lock khi lấy Room để tạo booking/direct/block; production vẫn cần stress test/concurrency test.	RoomRepository.findByIdForUpdate; BookingService.createBooking	Không tuyên bố tuyệt đối không bao giờ overbooking nếu chưa có kiểm thử tải production.
Q068	Pessimistic lock dùng ở đoạn tạo booking/direct booking/manual block để request sau phải chờ request trước cập nhật xong rồi kiểm tra availability lại.	RoomRepository.findByIdForUpdate; BookingService	Không nói lock dùng cho mọi truy vấn; chỉ dùng đoạn ghi nhạy cảm.
Q069	DIRECT booking ảnh hưởng availability vì Partner dùng nó để ghi nhận khách walk-in/ngoài hệ thống. Nhưng paymentStatus là NOT_REQUIRED, không qua TravelMate, không tính commission/settlement online.	BookingService.createDirectBooking; BookingSource.DIRECT; BookingSource.java	Không đưa direct booking vào revenue TravelMate.

Q070	MANUAL_BLOCK là Partner chặn phòng/căn cho bảo trì, giữ nội bộ hoặc bán ngoài hệ thống. Nó giữ quota theo ngày nhưng không phải booking doanh thu.	BookingService.blockRoom; BookingSource.MANUAL_BLOCK	Không gọi manual block là đơn thanh toán hoặc settlement.
Q071	COMPLETED là booking lịch sử đã hoàn tất. Trong source, sau checkout quota được mở lại và COMPLETED không nằm trong ACTIVE_STATUSES của AvailabilityService, nên không còn giữ phòng cho lần tìm kiếm mới.	BookingService.markCompletedByPartner; AvailabilityService.ACTIVE_STATUSES	Không nói COMPLETED tiếp tục chặn availability sau khi đã xử lý checkout.
Q072	NO_SHOW cọc 30% giữ cọc về tài chính nhưng sau khi xử lý no-show, quota được mở lại và NO_SHOW không còn là trạng thái giữ phòng trong AvailabilityService. 70% tại cơ sở không phát sinh.	BookingService.markNoShowByPartner; AvailabilityService; PaymentStatus.DEPOSIT_FORFEITED	Không nói no-show tiếp tục khóa phòng vô hạn hoặc cộng 70% vào revenue.

Câu tổng hợp: TravelMate chống overbooking bằng kiểm tra khoảng ngày, overlap query và lock ghi khi tạo booking/block. Các trạng thái active giữ phòng là PENDING_PAYMENT, PENDING_ADMIN_APPROVAL, CONFIRMED và CHECKED_IN; các trạng thái đã xử lý như CANCELLED, NO_SHOW, COMPLETED không còn giữ availability.

G.3 — Partner Workflow (Q073-Q080)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q073	Partner quản lý accommodation/room thuộc quyền sở hữu, gắn voucher được Admin cấp, xem booking của cơ sở mình, tiếp nhận booking đã được TravelMate giữ, check-in/check-out, no-show cọc 30%, direct booking/manual block, revenue/settlement/wallet.	PartnerPageController; BookingService; AccommodationService; VoucherService	Không nói Partner quản lý toàn hệ thống.
Q074	Partner không xem booking người khác. Backend kiểm tra owner và partnerPropertyType trong service, không chỉ ẩn menu.	BookingService.validatePartnerOwnership; getVisibleBookingForPartner; BookingCalculationTest	Không nói không thấy trên UI là đủ bảo mật.
Q075	TravelMate tự giữ phòng/căn sau khi VNPAY/TravelMate ghi nhận booking CONFIRMED và payment APPROVED. Partner tiếp nhận booking để check-in/check-out.	confirmBookingHoldByPartner; PartnerBookingStatus.PARTNER_CONFIRMED	Không check-in/tiếp nhận khi booking còn PENDING_PAYMENT.
Q076	Partner chỉ check-in booking đã được TravelMate giữ phòng/căn. Booking phải CONFIRMED, payment APPROVED/NOT_REQUIRED và partnerStatus PARTNER_CONFIRMED.	checkInByPartner; checkInWithRemainingConfirmByPartner	Không check-in booking chưa thuộc Partner hoặc chưa thanh toán hợp lệ.
Q077	Partner báo no-show khi booking đã được TravelMate giữ phòng nhưng khách không đến. Source chỉ cho no-show tự động với DEPOSIT_30; full payment no-show để Admin/chính sách xử lý riêng.	markNoShowByPartner; NoShowDepositTest	Không áp dụng no-show tùy tiện cho full payment.

Q078	Direct booking dùng cho khách offline/walk-in/đặt ngoài TravelMate để khóa availability và ghi nhận vận hành. Nó không qua VNPAY, không commission, không settlement online.	createDirectBooking; BookingSource.DIRECT	Không gọi direct booking là né commission; mục tiêu chính là tránh overbooking.
Q079	Manual block dùng để chặn phòng/căn khi bảo trì, giữ nội bộ hoặc không mở bán trong một khoảng ngày. Nó không có payment hoặc doanh thu online.	blockRoom; BookingSource.MANUAL_BLOCK	Không gọi manual block là booking khách hàng có doanh thu.
Q080	Partner tạo/sửa accommodation/room nhưng không tự duyệt public. Accommodation/room mới hoặc thay đổi quan trọng chuyển PENDING để Admin duyệt.	AccommodationService.createAccommodationForPartner; createRoomForPartner; approveRoom	Không nói Partner tự public dữ liệu không qua Admin.

Câu tổng hợp: Partner là người vận hành cơ sở của mình, không phải Admin. Mọi thao tác nhạy cảm đều phải qua owner/property-type guard ở backend.

G.4 — Admin Workflow (Q081-Q083)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q081	Admin quản lý user, duyệt accommodation/room, xem booking toàn hệ thống, voucher, review, support, travel post, revenue, settlement, withdrawal và export. Admin kiểm soát hệ thống, không phải người check-in/check-out chính.	AdminPageController; UserService; AccommodationService; SettlementService; PartnerWalletService	Không nói Admin làm thay toàn bộ Partner.
Q082	Admin duyệt accommodation/room để kiểm soát dữ liệu Partner đưa lên trước khi public: loại hình, địa chỉ, ảnh, giá, sức chứa, tiện nghi và trạng thái mở bán.	AccommodationService.approveAccommodation; approveRoom; ApprovalStatus	Không cho Partner tự public nếu flow đang dùng Admin approval.
Q083	Admin xem toàn bộ booking, lọc trạng thái và xử lý ngoại lệ như pending quá hạn, refund pending, partner hủy giữ phòng, đối soát. Flow lưu trữ chính vẫn do Partner xử lý.	AdminPageController.bookings; BookingService; PaymentService; BookingRepository.findAllForAdminPage	Không nói Admin là người vận hành check-in/check-out chính.

Câu tổng hợp: Admin là lớp kiểm soát và đối soát của hệ thống. Partner xử lý lưu trữ thực tế; Admin quản lý dữ liệu, chính sách, voucher, ngoại lệ và quyết toán.

G.5 — Câu hỏi xoắn để gặp

Câu hỏi xoắn	Cách trả lời an toàn
Partner có tự tạo voucher không?	Không. Admin tạo voucher; Partner chỉ gắn voucher PARTNER ROOM được cấp vào phòng/căn thuộc quyền quản lý.
Voucher Admin chịu thì Partner có mất tiền không?	Không. Voucher ADMIN là chi phí nền tảng, không trừ payout Partner.
Nếu khách A trả phòng ngày 10, khách B nhận phòng ngày 10 được không?	Được theo overlap chuẩn, vì oldCheckOut == newCheckIn thì không overlap.
Hai người bấm đặt cùng slot cuối thì sao?	Backend dùng transaction/lock khi tạo booking để kiểm tra lại availability; production vẫn cần test concurrency mạnh hơn.
Completed/no-show có còn giữ phòng không?	Không còn giữ availability sau khi đã xử lý trong source hiện tại; chúng là lịch sử/trạng thái tài chính, không nằm trong active statuses.
Partner có xem booking người khác bằng URL được không?	Không. Service kiểm tra owner và loại lưu trữ của Partner trước khi

	trả chi tiết hoặc cho thao tác.
Manual block có doanh thu không?	Không. Manual block chỉ khóa phòng/căn theo ngày, không payment, không commission, không settlement.

G.6 — Checklist học nhanh Q057-Q083

- Ưu tiên học: Q058, Q064, Q065, Q066, Q067, Q069, Q071, Q072, Q074, Q077, Q080, Q082.
- Khi demo voucher, dùng ví dụ USER_GLOBAL và PARTNER_ROOM; nhấn mạnh Admin tạo, Partner gán, backend validate.
- Khi demo availability, nói theo active statuses và overlap; không demo concurrency hai trình duyệt nếu chưa có script ổn định.
- Khi demo Partner, mở booking thuộc đúng partner và thử giải thích backend owner guard; không cần cố tình thao tác phá URL trong buổi bảo vệ nếu thiếu thời gian.
- Khi demo Admin, tập trung duyệt accommodation/room, voucher, booking exception, settlement/export; không nói Admin là người vận hành lưu trú chính.

Phần H — Câu hỏi bảo vệ Q084-Q110: Admin, settlement/wallet, security và transaction

Phần này bổ sung cho nhóm Q084-Q110. Nội dung đã được đối chiếu với SecurityConfig, AdminPageController, BookingService, PaymentService, SettlementService, PartnerWalletService, FileStorageService, PasswordResetService, OAuth2LoginSuccessHandler, travelmate_db.sql và các test hiện có.

Kết luận rà soát: gợi ý ChatGPT Plus đúng hướng. Điểm cần nói chặt hơn là TravelMate đang ở mức đồ án/demo: refund, settlement paid và withdrawal là ghi nhận trạng thái nội bộ hoặc xử lý ngoài hệ thống, không phải chuyển tiền ngân hàng/VNPAY production tự động.

Không cần sửa code hoặc thêm seed DB cho phần này: source đã có guard phân quyền, transaction, lock/idempotency ở các nghiệp vụ nhạy cảm; travelmate_db.sql gốc đã có dữ liệu ví, settlement, withdrawal và audit log để demo trực quan.

H.1 — Admin workflow (Q084-Q088)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q084	Admin không thay Partner check-in/check-out trong flow chính. Partner vận hành lưu trú thực tế; Admin chỉ có quyền override khi cần can thiệp ngoại lệ hoặc Partner không thao tác được.	AdminPageController check-in/complete/no-show; BookingService; PartnerPageController	Không nói Admin là người vận hành lưu trú chính.
Q085	Refund trong đồ án là cập nhật trạng thái REFUND_PENDING -> REFUNDED và ghi nhận Admin đã xử lý ngoài hệ thống. Chưa có API refund thật qua ngân hàng/VNPAY production.	BookingService.markRefundedByAdmin; PaymentStatus; Admin booking detail	Không nói hệ thống tự hoàn tiền thật.
Q086	Admin tạo và kiểm soát voucher: scope, thời hạn, điều kiện đơn, mức giảm và người chịu chi phí. Partner chỉ gán voucher kho PARTNER_ROOM được cấp vào phòng/căn đúng quyền.	VoucherService; VoucherScope; VoucherCostBearer; admin/partner voucher pages	Không nói Partner tự tạo voucher tùy ý.
Q087	Excel export dùng để đối chiếu settlement/withdrawal và hỗ trợ báo cáo quản trị. Đây là công cụ hỗ trợ kiểm tra, không phải hệ thống kế toán đầy đủ.	ExcelExportService; AdminPageController export settlement/withdrawal	Không gọi Excel export là accounting system.

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q088	Audit log ghi lại thao tác Admin quan trọng để truy vết khi demo: duyệt, refund, settlement, withdrawal, support. Mức hiện tại là audit cơ bản cho đồ án.	AdminActionLog; AdminActionLogRepository; AdminPageController.logAction	Không nói audit log đạt chuẩn compliance/production.

Câu tổng hợp: Admin là lớp kiểm soát hệ thống, quản lý chính sách và xử lý ngoại lệ. Partner mới là người tiếp nhận booking đã được TravelMate giữ, check-in/check-out và làm việc trực tiếp với khách tại cơ sở.

H.2 — Settlement / wallet / withdrawal (Q089-Q096)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q089	Settlement là kỳ quyết toán nội bộ giữa TravelMate và Partner, thường theo tháng. Hệ thống tổng hợp tiền online đã thu, commission, voucher deduction và payout.	SettlementService; PartnerSettlement; PaymentRepository	Không nói settlement là chuyển khoản tự động.
Q090	Booking vào settlement khi đã đủ điều kiện đối soát, thường là đã hoàn tất/lưu trú xong và có payment online hợp lệ. Direct/manual block không vào settlement vì không qua TravelMate payment.	SettlementService.generateMonthlySettlements; SettlementEligibilityTest	Không đưa booking mới CONFIRMED/chưa lưu trú xong vào payout.
Q091	Payout của Partner xuất phát từ tiền TravelMate đã thu online, sau đó tách hoa hồng nền tảng và voucher Partner chịu. Hoa hồng của cả FULL_PAYMENT và DEPOSIT_30 đều tính theo tổng đơn gốc trước voucher; riêng đơn cọc 30% không cộng 70% tại cơ sở vào dòng tiền online.	SettlementService; VoucherCostBearer; Booking financial snapshots	Không cộng 70% Partner thu tại cơ sở vào doanh thu online TravelMate.
Q092	Settlement theo tháng giúp gom nhiều booking vào một kỳ đối soát, dễ kiểm tra và xuất Excel hơn chi trả từng booking lẻ.	SettlementService; Admin settlements page	Không nói cứ đến tháng là ngân hàng tự chuyển tiền.
Q093	Khi Admin mark PAID, settlement chuyển PAID và PartnerWalletService credit payout vào ví nội bộ đúng một lần nếu chưa có transaction SETTLEMENT_CREDIT.	SettlementService.markSettlementPaid; PartnerWalletService.creditSettlement	Không nói mark PAID là chuyển tiền thật. Không bấm lặp để cộng ví.
Q094	Partner wallet là sổ sổ dư nội bộ: available balance, pending withdrawal, total earned, total withdrawn. Nó không phải ví ngân hàng hoặc ví điện tử thật.	PartnerWallet; PartnerWalletTransaction; partner/wallet page	Không gọi là ví ngân hàng thật.
Q095	Withdrawal PENDING nghĩa là Partner đã gửi yêu cầu rút; hệ thống giữ số tiền đó bằng cách giảm available balance và tăng pending withdrawal amount.	PartnerWalletService.requestWithdrawal; PartnerWithdrawalRequest	Không nói PENDING là đã chuyển khoản.
Q096	Reject withdrawal trả lại số dư: giảm pending withdrawal và cộng amount về available balance. Mark paid chỉ ghi nhận Admin đã chuyển ngoài hệ thống.	PartnerWalletService.rejectWithdrawal/markWithdrawalPaid	Không để hiểu reject làm mất tiền Partner.

Câu tổng hợp: Settlement/wallet của TravelMate là sổ đối soát nội bộ. Hệ thống tính tiền dựa trên phần TravelMate thu online; Admin xử lý chuyển khoản ngoài hệ thống rồi ghi nhận trạng thái trong TravelMate.

H.3 — Security (Q097-Q104)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q097	Route /admin/** được bảo vệ ở Spring Security bằng role	SecurityConfig	Không nói chỉ ẩn menu là đủ bảo

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
	ADMIN; /partner/** chỉ role PARTNER; booking/my-bookings chỉ role USER.		mật.
Q098	Mật khẩu không lưu plain text. Project dùng BCryptPasswordEncoder; khi login Spring Security so khớp password nhập vào với hash trong DB.	SecurityConfig.passwordEncoder; UserService; User.java	Không nói có thể xem lại mật khẩu cũ.
Q099	Google login chỉ dành cho USER. Email Google mới tạo role USER; nếu email đã là Admin/Partner thì OAuth2LoginSuccessHandler chặn để tránh leo quyền.	OAuth2LoginSuccessHandler; OAuth2LoginSuccessHandlerTest	Không cho Admin/Partner login bằng Google nếu không có flow quản trị riêng.
Q100	Forgot password dùng token reset có thời hạn, token cũ bị invalidate và password mới được hash lại bằng BCrypt. Hệ thống không gửi mật khẩu cũ.	PasswordResetService; PasswordResetToken	Không nói gửi lại mật khẩu qua email.
Q101	CSRF hiện đang tắt để thuận tiện demo/local test. Khi production phải bật CSRF token cho các request thay đổi dữ liệu như booking, voucher, admin action.	SecurityConfig.csrf	Không nói CSRF không cần thiết.
Q102	Upload ảnh có giới hạn 10MB, chỉ JPG/PNG/WEBP, kiểm extension khớp content-type và đổi tên UUID. Production nên thêm magic bytes, virus scan/object storage.	FileStorageService; UserPortalFlowTemplateTest	Không nói upload đã an toàn tuyệt đối.
Q103	Backend không tin hidden price từ client. BookingService lấy Room từ DB, kiểm room thuộc accommodation, kiểm trạng thái, tồn phòng và tự tính total từ giá server-side.	BookingService.createBooking; calculateTotalAmount; RoomRepository.findByIdForUpdate	Không tính tiền dựa trên price user gửi từ DevTools.
Q104	User chỉ xem danh sách booking của mình và cancel phải check booking.user.id. Partner cũng bị guard theo accommodation.owner.	BookingService.getBookingsByUser/cancelBooking; BookingRepository; PartnerPageController	Không dùng bookingId từ URL rồi trả dữ liệu nếu không check owner.

Câu tổng hợp: TravelMate có phân quyền theo role, hash password bằng BCrypt, kiểm ownership ở nghiệp vụ nhạy cảm và backend tự tính dữ liệu quan trọng. Các điểm như CSRF, secret manager, rate limit và kiểm thử bảo mật chuyên sâu là hướng nâng cấp production.

H.4 — Transaction và data consistency (Q105-Q110)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q105	@Transactional đảm bảo nhiều cập nhật DB trong một nghiệp vụ cùng thành công hoặc rollback cùng nhau: booking/payment/quota, payment callback, settlement, wallet, withdrawal.	BookingService; PaymentService; SettlementService; PartnerWalletService	Không update nhiều bảng rồi rạc rồi bỏ mặc lỗi giữa chừng.
Q106	Pessimistic lock khóa bản ghi khi xử lý, phù hợp giữ phòng cuối cùng và ví/rút tiền. Optimistic lock phù hợp xung đột thấp nhưng source hiện dùng lock ghi cho các điểm nhạy cảm.	RoomRepository.findByIdForUpdate; PartnerWalletRepository; PartnerWithdrawalRequestRepository	Không nói lock càng nhiều càng tốt; lock nhiều có thể giảm hiệu năng.
Q107	Payment callback chống trùng bằng idempotency: chỉ xử lý khi payment còn PENDING_PAYMENT; callback sau khi đã APPROVED/FAILED/CANCELLED/EXPIRED sẽ bị bỏ qua. Return URL cũng có guard trạng thái cũ.	PaymentService.markGatewaySuccess/Failed; PaymentController	Không update payment mù quáng theo mọi callback.
Q108	Wallet double credit được chặn bằng kiểm tra transaction SETTLEMENT_CREDIT và unique key ở DB. Settlement đã PAID hoặc đã credit không được cộng ví thêm.	PartnerWalletService.creditSettlement; partner_wallet_transactions unique key	Không cộng available balance mỗi lần bấm/refresh.
Q109	Withdrawal chỉ xử lý request PENDING. Service dùng lock,	PartnerWalletService.ensurePending;	Không cho paid/reject lại cùng

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
	ensurePending và transaction type để chặn paid/reject lặp; reject hoàn tiền một lần, paid ghi nhận một lần.	markWithdrawalPaid; rejectWithdrawal	request.
Q110	Nếu DB lỗi trong transaction, thao tác rollback để tránh dữ liệu nửa vời. Demo cần backup/import SQL ổn định; production cần migration, backup, monitoring và retry chính sách rõ ràng.	@Transactional; travelmate_db.sql; docs/sql health checks	Không nói hệ thống vẫn bình thường nếu DB tắt hoặc mất kết nối.

Câu tổng hợp: Các nghiệp vụ có tiền và trạng thái nhiều bảng đều cần transaction, lock và idempotency. Điểm quan trọng khi bảo vệ là biết vì sao không được cập nhật trùng payment, settlement paid hoặc withdrawal.

H.5 — Câu hỏi xoắn dễ gặp

Câu hỏi xoắn	Cách trả lời an toàn
Admin bấm PAID là chuyển tiền luôn hả?	Không. Đó là ghi nhận nội bộ sau khi Admin xử lý chuyển khoản ngoài hệ thống.
70% còn lại của cọc có vào settlement không?	Không. Đó là tiền Partner thu tại cơ sở, TravelMate không giữ nên không tính vào online amount.
Ví Partner có phải ví ngân hàng không?	Không. Nó là sổ theo dõi sổ dư quyết toán nội bộ.
User sửa giá 1 đồng bằng DevTools thì sao?	Backend lấy giá Room từ DB và tự tính lại tổng tiền, không tin hidden field giá.
CSRF đang tắt có nguy hiểm không?	Có nếu production. Trong demo tắt để dễ test; khi triển khai thật phải bật CSRF token.
Callback VNPAY gửi trùng thì sao?	Service kiểm trạng thái hiện tại, chỉ xử lý payment còn PENDING_PAYMENT nên không approve/cộng dữ liệu trùng.
Withdrawal reject có mất tiền Partner không?	Không. Amount đang hold được trả lại available balance.
Audit log đã đạt chuẩn kiểm toán chưa?	Chưa nên nói như vậy. Hiện là log thao tác cơ bản cho đồ án; production cần chống sửa log và phân quyền xem log tốt hơn.

H.6 — Checklist học nhanh Q084-Q110

- Ưu tiên học: Q084, Q085, Q089, Q091, Q093, Q094, Q097, Q098, Q103, Q107, Q108, Q109.
- Khi demo Admin, mở /admin/bookings, /admin/settlements, /admin/withdrawals và /admin/activity-logs; nhấn mạnh Admin xử lý ngoại lệ/đối soát.
- Khi demo Partner wallet, nói rõ available balance, pending withdrawal, total earned và total withdrawn là sổ nội bộ.
- Test nhanh trước bảo vệ: user thường vào /admin bị chặn; Partner không xem booking người khác; withdrawal thiếu bank info/vượt số dư bị chặn; reject trả lại balance.
- Không demo live các tình huống khó kiểm soát như DB lỗi, CSRF attack, race condition hai máy, refund ngân hàng thật hoặc VNPAY production refund.

H.7 — Đoạn trả lời liên hoàn cần thuộc

Trong TravelMate, Admin kiểm soát hệ thống, xử lý ngoại lệ và ghi nhận đối soát; Partner mới vận hành lưu trú thực tế như tiếp nhận booking đã được TravelMate giữ, check-in và check-out. Settlement được tính trên số tiền online TravelMate thu, trừ commission và voucher do Partner chịu; ví Partner là sổ nội bộ chứ không phải ví ngân hàng. Các nghiệp vụ nhạy cảm như payment callback, settlement paid và withdrawal đều cần transaction,

lock hoặc idempotency để tránh cập nhật trùng. Về bảo mật, hệ thống dùng phân quyền theo role, BCrypt cho mật khẩu, Google login USER-only và backend tự tính giá booking thay vì tin dữ liệu client. Một số điểm như CSRF, secret manager, kiểm thử bảo mật chuyên sâu và tích hợp chuyển tiền thật là hướng nâng cấp production, không nên nói quá trong đồ án cơ sở.

Phần I — Câu hỏi bảo vệ Q111-Q136: Gateway/DB lỗi, testing, demo/deployment và hạn chế production

Phần này bổ sung cho nhóm Q111-Q136. Nội dung đã được đối chiếu với PaymentController, PaymentService, VnpayService, application.properties, README_DEMO.md, docs/07_Test_Report.md, docs/08_Demo_Script.md, docs/Security-Notes.md và BaoCaoTravelMate_1.docx.

Kết luận rà soát: gợi ý ChatGPT Plus đúng hướng nhưng cần cập nhật theo source hiện tại. Số test đúng là 279/279 pass, không dùng số cũ 247/247, 250/250, 255/255, 279/279, 267/267, 269/269 hoặc 279/279.

TravelMate hiện là demo/local/sandbox, chưa phải production system.

Không cần sửa code hoặc thêm seed DB cho phần này: travelmate_db.sql gốc đã đủ dữ liệu demo, application.properties đã cấu hình local 8080/MySQL/VNPAY Sandbox qua env, và báo cáo Word đã ghi hạn chế production ở Chương 4.

I.1 — Gateway/DB consistency (Q111-Q112)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q111	Nếu VNPAY báo success nhưng DB lỗi khi cập nhật, đây là lệch trạng thái giữa gateway và database. Bản demo có transaction, raw payload và trạng thái payment/booking; production cần retry job, log đối soát và màn Admin reconciliation.	PaymentController; PaymentService; Payment.rawReturnPayload/rawipnPayload; VnpayService	Không nói “không bao giờ xảy ra”. Không nói F5 là đủ xử lý.
Q112	Đối soát production cần mã giao dịch, raw callback/IPN, thời gian nhận, trạng thái gateway, log retry, màn Admin kiểm pending/mismatch và API tra cứu lại giao dịch nếu gateway hỗ trợ.	PaymentController.vnpayIpN; VnpayService.verifySignature/validateAmount; PaymentRepository	Không nói hệ thống đã đối soát ngân hàng đầy đủ.

Câu tổng hợp: Trong đồ án, TravelMate xử lý payment bằng verify chữ ký, validate amount, transaction và idempotency. Nếu triển khai thật, cần thêm cơ chế đối soát gateway vì giao dịch tiền thật có thể thành công bên cổng thanh toán nhưng hệ thống local chưa cập nhật được.

I.2 — Testing (Q113-Q120)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q113	TravelMate có test tự động và test thủ công. Test tự động kiểm service/controller/security/template/SQL; manual test kiểm UI, session, redirect, VNPAY Sandbox và demo trên trình duyệt.	src/test/java; docs/07_Test_Report.md; docs/08_Demo_Script.md	Không nói chỉ test tay. Không nói test bao phủ 100%.
Q114	Số đúng hiện tại là 279/279 pass, lấy từ docs/07_Test_Report.md và lần chạy Maven mới nhất. Khi bảo vệ chỉ đọc số này nếu đã chạy lại hoặc có report/ảnh BUILD SUCCESS.	docs/07_Test_Report.md; mvnw.cmd test	Không dùng số cũ trong gợi ý nếu chưa chạy lại Maven test trên source hiện tại.
Q115	Vẫn cần test thủ công vì unit/integration test không thay thế trải nghiệm UI thật: VNPAY redirect, upload ảnh, Excel mở bằng app thật, wording trạng thái và thao tác Admin/Partner.	Demo script; templates; browser manual checklist	Không nói có unit test là khỏi cần demo test.
Q116	Unit test không gọi gateway thật vì phụ thuộc mạng, sandbox và	PaymentServiceTest;	Không để unit test phụ thuộc

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
	secret. Nên test xử lý callback giả lập/mocked; VNPAY Sandbox thật kiểm bằng manual/integration demo.	PaymentControllerTest; VnpayService	internet/gateway thật.
Q117	Availability overlap được test bằng AvailabilityOverlapTest, AvailabilityServicePartnerTypeGuardTest và BookingService: overlap ngày, checkout==checkin không overlap, quantity, direct/manual block.	AvailabilityService; BookingService; AvailabilityOverlapTest	Không chỉ nhìn số lượng phòng tổng rồi kết luận còn phòng.
Q118	Settlement test kiểm eligibility, online amount, commission, voucher deduction, payout, settlement paid và wallet/withdrawal. Direct/manual block không vào settlement.	SettlementServiceTest; SettlementEligibilityTest; PartnerWalletServiceTest	Không test settlement bằng booking chưa hoàn tất hoặc direct booking.
Q119	OAuth/forgot password có test handler/service và cần test thủ công khi có env. Google login chỉ USER; forgot password dùng token một lần, hết hạn 30 phút, token cũ bị vô hiệu.	OAuth2LoginSuccessHandlerTest; GoogleOAuthClientConfigTest; PasswordResetServiceTest	Không nói luôn gửi mail/login Google được nếu chưa cấu hình SMTP/OAuth.
Q120	Nếu chưa chạy lại test trên máy bảo vệ, nói theo report hiện tại và cam kết chạy lại trước demo. Trong phiên này kết quả mới nhất là 279 tests pass, 0 fail/error/skipped.	docs/07_Test_Report.md; terminal Maven	Không bịa “vừa chạy pass” nếu chưa chạy trên máy đó.

Câu tổng hợp: Test tự động chứng minh logic chính không lệch, nhưng demo vẫn cần test thủ công vì gateway, UI, browser session, upload, Excel và cấu hình môi trường không thể được unit test bao phủ hoàn toàn.

I.3 — Demo/deployment environment (Q121-Q128)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q121	App chạy local Spring Boot ở port 8080 theo server.port=8080, truy cập http://localhost:8080. Nếu port bị chiếm thì đổi server.port hoặc dùng port khác khi demo.	application.properties; README_DEMO.md	Không nói đã deploy public nếu chỉ chạy local.
Q122	DB chính cần import travelmate/src/main/resources/travelmate_db.sql. File gốc hiện đã có seed booking, voucher, settlement, wallet, withdrawal và audit log; không cần file wallet bổ sung cho phần này.	travelmate_db.sql; UserPortalSqlSeedTest; README_DEMO.md	Không quên import DB hoặc học theo file seed cũ.
Q123	VNPAY cần TMN code, hash secret, pay URL sandbox, return URL và IPN URL qua env hoặc .env local. Secret không đưa lên báo cáo/trình chiếu.	application.properties; VnpayConfigProperties; docs/demo-env.example	Không lộ secret; không gọi sandbox là production merchant.
Q124	Thiếu SMTP thì forgot password không gửi mail thật nhưng vẫn có logic token/fallback an toàn; email xác nhận booking sau VNPAY success cũng là luồng phụ trợ và lỗi gửi mail không rollback booking. Thiếu Google OAuth thì không demo Google login; form login vẫn chạy bình thường.	EmailService; PasswordResetService; GoogleOAuthClientConfig	Không cố demo OAuth/SMTP khi chưa test cấu hình.
Q125	ddl-auto=update phù hợp dev/demo vì tiện cập nhật schema, nhưng production nên dùng Flyway/Liquibase hoặc validate để kiểm soát migration.	application.properties	Không nói update là best practice production.
Q126	Log demo chủ yếu console và AdminActionLog cho một số thao tác nghiệp vụ. Production cần log tập trung, phân quyền xem log, che secret và monitoring.	AdminActionLog; logging config; console log	Không in secret/API key/hash secret ra log.
Q127	Source hiện chủ yếu chạy local bằng Spring Boot + MySQL. Chưa coi Docker là công nghệ triển khai chính; nếu có dùng Docker thì chỉ nói như hỗ trợ môi trường dev/demo.	root project; README_DEMO.md; docs/18	Không ghi Docker/CI-CD là phần đã triển khai chính nếu source không có bằng chứng.

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q128	Nếu MySQL lỗi khi demo: kiểm service, user/password/database, import lại SQL, dùng backup dump/ảnh/video. Không sửa code tại chỗ; ưu tiên khôi phục môi trường.	application.properties; travelmate_db.sql; README_DEMO.md	Không đợi đến buổi bảo vệ mới import DB.

Câu tổng hợp: Môi trường TravelMate hiện là local demo: Spring Boot port 8080, MySQL local, SQL seed gốc, VNPAY Sandbox và SMTP/OAuth/Groq optional qua cấu hình. Production cần domain HTTPS, public IPN, secret manager, migration, monitoring và backup.

I.4 — Limitations (Q129-Q136)

ID	Cách trả lời nên học	File/báo cáo nên mở	Điểm cần tránh
Q129	Điểm yếu lớn nhất: project ở mức demo/local với dữ liệu mẫu, VNPAY Sandbox, chưa production security đầy đủ, chưa load test và refund/đối soát thật còn mờ phông.	BaoCaoTravelMate_1.docx Chương 4; docs/18; Security-Notes.md	Không nói không có điểm yếu.
Q130	Chưa production vì đồ án cơ sở tập trung nghiệp vụ chính. Production cần domain, HTTPS, cloud/server, backup, monitoring, CI/CD, secret manager, payment production và load/security test.	Báo cáo Chương 4; docs/18	Không nói chỉ upload code lên hosting là xong.
Q131	Chưa có rate limit thì login, forgot password, chatbot/search có thể bị spam hoặc tổn tài nguyên. Demo chưa nghiêm trọng; production cần rate limit/captcha/cache/log.	SecurityConfig; ChatbotApiController; Auth routes	Không phủ nhận rủi ro spam/API abuse.
Q132	Chưa có load test nên chỉ khẳng định ổn trong phạm vi demo/chức năng. Production cần đo concurrent users, response time, DB pool và race condition.	docs/07_Test_Report.md; Báo cáo Chương 4	Không nói chịu được nhiều người dùng nếu chưa đo.
Q133	CSRF tắt là rủi ro production. Demo tắt để tiện form/payment test; triển khai thật phải bật CSRF token cho form POST/API thay đổi dữ liệu.	SecurityConfig; Security-Notes.md	Không nói CSRF không cần thiết.
Q134	Refund thật chưa có. Project dùng REFUND_PENDING/REFUNDED và Admin ghi nhận đã xử lý ngoài hệ thống; chưa tích hợp API refund ngân hàng/VNPAY production.	PaymentStatus; BookingService.markRefundedByAdmin	Không nói hệ thống đã hoàn tiền thật.
Q135	IPN local có hạn chế: VNPAY server-to-server không gọi được localhost nếu không có public tunnel/ngrok. Demo local thường dựa vào return URL; production cần domain public HTTPS.	PaymentController.vnpayIpN; application.properties VNPAY_IPN_URL	Không nói localhost tự nhận IPN thật từ internet.
Q136	Dữ liệu mẫu đủ demo luồng chính nhưng chưa đại diện kinh doanh thật: ảnh, chính sách hủy, mùa cao điểm, nhiều lỗi nhập liệu, review thật, vận hành nhiều cơ sở.	travelmate_db.sql; Báo cáo Chương 4	Không nói seed demo đủ cho kinh doanh thật.

Câu tổng hợp: Hạn chế của TravelMate không phải lỗi nghiêm trọng trong phạm vi đồ án, nhưng phải nói đúng: local demo, dữ liệu mẫu, sandbox payment, chưa load test, chưa rate limit/CSRF production, chưa refund/đối soát thật.

I.5 — Câu hỏi xoắn để gặp

Câu hỏi xoắn	Cách trả lời an toàn
VNPAY báo thành công mà DB chết thì khách mất tiền à?	Production phải có đối soát gateway, retry job và Admin reconciliation. Đồ án hiện dùng sandbox, transaction và log payload cơ bản, chưa phải hệ thống tài chính production.
279 test pass là vừa chạy à?	Trong phiên rà soát này đã chạy lại full suite 279/279 pass. Trên máy bảo vệ, nhóm

Câu hỏi xoắn	Cách trả lời an toàn
	nên chạy lại hoặc chuẩn bị ảnh BUILD SUCCESS.
Không có load test sao dám nói ổn định?	Chỉ khẳng định ổn trong phạm vi demo/chức năng; production cần load test riêng.
CSRF tắt thì có bảo mật không?	Chỉ phù hợp demo local; production cần bật CSRF token cho request thay đổi dữ liệu.
Refund đã chuyển tiền thật chưa?	Chưa. Hệ thống chỉ ghi nhận trạng thái refund; hoàn tiền thật cần API/cổng thanh toán production.
IPN localhost có nhận được từ VNPAY không?	Không ổn định nếu không có tunnel/public URL. Local demo thường dựa vào Return URL; production cần HTTPS public.
Dữ liệu mẫu có đủ chứng minh vận hành thật không?	Đủ để demo nghiệp vụ chính, chưa đủ chứng minh vận hành thực tế quy mô lớn.
Docker/CI/CD đã có chưa?	Chưa coi là phần triển khai chính nếu source không có bằng chứng; đưa vào hướng phát triển.

I.6 — Checklist học nhanh Q111-Q136

- Ưu tiên học: Q111, Q114, Q120, Q123, Q125, Q128, Q129, Q133, Q134, Q135.
- Trước demo: chạy app 8080, MySQL ổn, import travelmate_db.sql, đăng nhập được User/Partner/Admin.
- VNPAY Sandbox chỉ demo khi env chắc; nếu không, dùng booking seed/ảnh/video fallback.
- Không show application secret, VNPAY hash secret, Google client secret hoặc SMTP password.
- Không nói production-ready; nói hệ thống đạt mức đồ án với guard nghiệp vụ và có hướng nâng cấp production.
- Không nói refund/settlement/withdrawal là chuyển tiền thật; đó là trạng thái/sổ đối soát nội bộ trong bản demo.

I.7 — Đoạn trả lời liên hoàn cần thuộc

TravelMate hiện là đồ án cơ sở chạy ở môi trường local/demo. Nhóm đã hoàn thiện các luồng chính như đăng nhập, đặt phòng, VNPAY Sandbox, Partner xử lý lưu trú, Admin quản lý và settlement nội bộ; full test suite hiện pass 279/279. Tuy nhiên nhóm không khẳng định đây là production system. Các phần như payment thật, refund thật, IPN public, load test, CSRF đầy đủ, rate limit, monitoring, secret manager và DB migration cần bổ sung nếu triển khai thực tế. Khi bảo vệ, nên trình bày đúng phạm vi demo và nêu rõ hạn chế này ở Chương 4.

Phần J — Câu hỏi bảo vệ Q137-Q160: Mở rộng, so sánh thực tế và hard defense

Phần này bổ sung cho nhóm Q137-Q160. Nội dung đã được đối chiếu với BookingService, PaymentService, VnpayService, SecurityConfig, SettlementService, PartnerWalletService, ChatbotService, Accommodation, PaymentMethod, InternationalizationConfig, messages_vi/en và Chương 4 báo cáo.

Kết luận rà soát: gợi ý ChatGPT Plus đúng hướng. Điểm cần nói chặt hơn theo source hiện tại: TravelMate là monolith MVC phù hợp đồ án; MoMo mới có enum dự phòng, chưa có provider thật; bản đồ chưa có latitude/longitude; chatbot là rule-based + Groq optional, chưa phải recommendation AI production; i18n hiện mới ở mức cơ bản.

Không cần sửa code/DB cho phần này. Đây là nhóm câu hỏi bảo vệ và hướng phát triển, mục tiêu là giúp nhóm trả lời thực tế, biết mở source chứng minh, và không so sánh ngang Booking/Agoda/Airbnb.

J.1 — Khả năng mở rộng (Q137-Q144)

ID	Cách trả lời nên học	File/source nên mở	Điểm cần tránh
Q137	Nếu production cần thêm cloud/server, domain, HTTPS, profile production, secret manager, DB backup, monitoring/log tập trung, CI/CD, Flyway/Liquibase, bật CSRF, rate limit, payment/refund production, IPN public và load/security test.	application.properties; SecurityConfig; PaymentController; docs/18; BaoCao Chương 4	Không nói chỉ upload code lên hosting là xong.
Q138	Nhiều user đồng thời cần load test, tối ưu query, connection pool, cache, transaction/locking cho booking, monitoring và retry. Source có overlap/lock ở mức demo nhưng chưa khẳng định chịu tải lớn.	BookingService; AvailabilityService; RoomRepository.findByIdForUpdate; PartnerWalletRepository locks	Không nói chắc chắn chịu tải lớn nếu chưa đo.
Q139	Không cần microservices trong đồ án. Nếu hệ thống lớn, có thể tách Payment, Notification/Email, Chatbot/AI, Search/Recommendation, Settlement/Finance. Booking phải tách rất cẩn thận vì là lỗi.	PaymentService; EmailService; ChatbotService; SettlementService; BookingService	Không nói microservices luôn tốt hơn monolith.
Q140	Mobile app cần REST API/JSON đầy đủ, auth token/JWT hoặc cơ chế phù hợp mobile, CORS/API security, API versioning, chuẩn response, upload qua API và push notification nếu cần.	controller/api; controller/page; SecurityConfig	Không nói mobile chỉ copy giao diện web.
Q141	Thêm MoMo có thể reuse booking, Payment entity, PaymentStatus, tính tiền/voucher/settlement. Cần provider/service riêng để tạo URL/QR, verify signature, callback và map trạng thái. Hiện MOMO_DEMO chỉ là enum dự phòng.	PaymentMethod; Payment; PaymentService; BookingService; VnpayService	Không trộn logic MoMo vào VnpayService. Không nói MoMo thật đã có.
Q142	Bản đồ cần thêm latitude/longitude vào Accommodation, form Admin/Partner nhập/chọn vị trí, hiển thị ở detail/search. Production cần geocoding, validate địa chỉ và quản lý API key bản đồ.	Accommodation.java; accommodation templates; admin/partner forms	Không nói hiện đã có bản đồ tương tác nếu chưa có tọa độ.
Q143	Recommendation AI cần dữ liệu hành vi: search, click, booking, điểm đến, ngân sách, số người, ngày ở, review/rating, wishlist. Hiện ChatbotService là rule-based + Groq optional, chưa đủ dữ liệu cho AI recommendation thật.	ChatbotService; Booking; Review; TravelPost; TravelDestination	Không gọi chatbot hiện tại là recommendation AI hoàn chỉnh.
Q144	Đa ngôn ngữ mở rộng cần đưa text UI vào message files, chuẩn hóa key, dùng i18n Spring/Thymeleaf, dịch nội dung DB bằng cột/bảng bản dịch. Source đã có messages_vi/en và lang switch cơ bản.	InternationalizationConfig; messages_vi/en; user-header.html	Không chỉ dịch vài menu rồi gọi là i18n hoàn chỉnh.

Câu tổng hợp: Nếu mở rộng, TravelMate nên đi từ monolith ổn định sang production-ready từng bước. Hướng nâng cấp gồm hạ tầng thật, API mobile, payment provider mới, bản đồ, i18n hoàn chỉnh và recommendation khi có đủ dữ liệu. Không cần đẩy sang microservices trong phạm vi đồ án cơ sở.

J.2 — So sánh với thực tế (Q145-Q152)

ID	Cách trả lời nên học	Dẫn chứng/source nên mở	Điểm cần tránh
Q145	So với Booking/Agoda/Airbnb, TravelMate còn thiếu dữ liệu thật quy mô lớn, dynamic pricing, channel manager, fraud/dispute, mobile app, multi-currency, monitoring, payment/refund thật và vận hành production.	BaoCao Chương 4; docs/18; module hiện có	Không nói TravelMate ngang các nền tảng thương mại lớn.
Q146	Hủy phòng thực tế có nhiều chính sách: miễn phí trước ngày X, mất cọc, hoàn một phần, không hoàn, đổi ngày, theo mùa/phòng/đối tác. TravelMate hiện xử lý mức cơ bản bằng trạng thái cancel/refund/cọc.	BookingStatus; PaymentStatus; BookingService.cancelBooking	Không áp một rule cho mọi trường hợp thật.
Q147	Payout Partner thực tế cần thông tin ngân hàng xác thực, hóa đơn/chứng từ, lịch chi trả, thuế/phí, dispute, KYC/AML nếu quy mô	SettlementService; PartnerWalletService; PartnerWithdrawalRequest	Không gọi wallet là ví ngân hàng thật.

ID	Cách trả lời nên học	Dẫn chứng/source nên mở	Điểm cần tránh
	lớn. TravelMate hiện là sổ quyết toán nội bộ.		
Q148	Fraud payment cần signature/callback check, chống replay, rate limit, device/IP risk, blacklist, manual review, alert và đối soát gateway. Đồ án mới có verify signature/amount/status/idempotency cơ bản.	VnpayService; PaymentService; PaymentController; AdminActionLog	Không nói sandbox payment đã chống fraud đầy đủ.
Q149	No-show thực tế phụ thuộc chính sách: giữ cọc, charge một đêm, giữ toàn bộ non-refundable, hoặc đổi ngày. TravelMate hiện rõ nhất với cọc 30%: khách không đến thì mất cọc, 70% không phát sinh.	BookingService.markNoShow; PaymentStatus.DEPOSIT_FORFEITED	Không áp cứng rule no-show full payment nếu chưa chốt chính sách.
Q150	Giá theo mùa chưa có đầy đủ. TravelMate chủ yếu dùng pricePerNight cơ bản; production cần seasonal/weekend/holiday pricing, surcharge, discount thời gian và giá theo kênh.	Room.pricePerNight; BookingService.calculateTotalAmount	Không nói đã dynamic pricing nếu chưa có.
Q151	Channel manager cần khi bán nhiều kênh để đồng bộ tồn phòng/giá. TravelMate hiện quản lý quota trong hệ thống, có direct booking/manual block để Partner quản lý thủ công.	BookingService.createDirectBooking/blockRoom; AvailabilityService	Không nói manual block thay thế hoàn toàn channel manager.
Q152	Dispute/support thực tế cần SLA, bằng chứng, log trao đổi, refund policy, phân loại ticket và escalation. TravelMate có support ticket ở mức cơ bản, chưa phải dispute center đầy đủ.	SupportTicketService; Admin support page; SupportTicket entity	Không gọi support hiện tại là dispute center production.

Câu tổng hợp: TravelMate là OTA-mini/demo mô phỏng nghiệp vụ lõi. Hệ thống có giá trị học thuật và demo, nhưng không so sánh ngang Booking/Agoda/Airbnb vì thiếu dữ liệu thật, scale, channel manager, dynamic pricing, dispute, fraud và payment/refund production.

J.3 — Hard defense (Q153-Q160)

ID	Cách trả lời nên học	File/code nên mở	Điểm cần tránh
Q153	Nếu AI hỗ trợ code: nói trung thực là có dùng công cụ hỗ trợ nhưng nhóm đã đọc source, chạy test, hiểu luồng booking/payment/security/partner/settlement và có thể mở code giải thích.	BookingService; PaymentService; SecurityConfig; SettlementService	Không nói AI làm hết hoặc không hiểu code.
Q154	Nếu mở code booking: mở BookingService.createBooking và giải thích validate ngày/số lượng/người, check accommodation/room approved, lock room, overlap availability, validate voucher, tính tiền từ DB, tạo booking/payment PENDING_PAYMENT.	BookingService.createBooking; RoomRepository.findByIdForUpdate; PaymentStatus.PENDING_PAYMENT	Không né mở code. Không nói giá lấy từ form client.
Q155	CSRF chưa bật vì demo/local cần form/payment test đơn giản hơn. Production phải bật CSRF cho request thay đổi dữ liệu và thêm token đúng trong Thymeleaf/API.	SecurityConfig.csrf; Security-Notes.md	Không nói CSRF không cần thiết.
Q156	ddl-auto=update dùng cho dev/demo để sync schema nhanh. Production nên dùng Flyway/Liquibase/validate để tránh thay đổi DB ngoài kiểm soát.	application.properties	Không nói ddl-auto=update là best practice production.
Q157	Refund thật chưa có. Project mô phỏng bằng REFUND_PENDING/REFUNDED và Admin ghi nhận xử lý ngoài hệ thống. Production cần API refund/cổng thanh toán/ngân hàng và đối soát.	PaymentStatus; BookingService.markRefundedByAdmin; PaymentService	Không nói hệ thống tự hoàn tiền thật.
Q158	Return URL có thể test qua browser về localhost. IPN là server-to-server nên local cần tunnel/public HTTPS; production cần domain HTTPS ổn định.	PaymentController.vnpayReturn/vnpayIp n; application.properties	Không nói IPN chắc chắn chạy trên localhost.
Q159	Overbooking production: source có overlap và lock ghi cho room, đủ demo cơ bản; production cần concurrency/load test, transaction isolation, retry và monitoring.	AvailabilityService; BookingService; RoomRepository.findByIdForUpdate	Không nói tuyệt đối không bao giờ overbooking nếu chưa load test.

ID	Cách trả lời nên học	File/code nên mở	Điểm cần tránh
Q160	Wallet không phải ví thật. Nó là sổ quyết toán nội bộ ghi settlement, balance, withdrawal request và transaction history. Admin chuyển khoản ngoài hệ thống rồi cập nhật trạng thái.	PartnerWalletService; PartnerWallet; PartnerWithdrawalRequest	Không gọi là ví điện tử/ví ngân hàng production.

Câu tổng hợp: Hard defense cần trả lời bằng source thật. Nhóm phải mở được BookingService, PaymentService/VnpayService, SecurityConfig và SettlementService/PartnerWalletService để chứng minh hiểu flow, không chỉ đọc báo cáo.

J.4 — Câu hỏi xoắn để gặp

Câu hỏi xoắn	Cách trả lời an toàn
Sao không làm microservices ngay?	Monolith MVC phù hợp đồ án cơ sở vì dễ demo, dễ test và ít rủi ro. Microservices chỉ hợp lý khi hệ thống/team lớn và cần scale độc lập.
Thêm MoMo có phải viết lại booking không?	Không. Booking/tính tiền/voucher/status reuse được; chỉ thêm payment provider riêng và callback mapper. Hiện chưa có MoMo thật.
So với Booking.com thiếu gì?	Thiếu dữ liệu thật, scale, dynamic pricing, channel manager, dispute, fraud, mobile app, payment/refund production.
AI code thì em hiểu không?	Có. Nhóm đã đọc lại source và có thể mở code giải thích booking/payment/security/settlement.
CSRF tắt mà bảo mật à?	Demo có thể relax; production phải bật. Không phủ nhận rủi ro.
Wallet không chuyển tiền thì để làm gì?	Để ghi nhận quyết toán nội bộ, số dư và yêu cầu rút tiền; chuyển khoản thật nằm ngoài hệ thống demo.
Chatbot có phải AI recommendation thật không?	Không nên nói vậy. Hiện là rule-based + Groq optional; recommendation nâng cao cần dữ liệu hành vi lớn hơn.
Bản đồ đã có chưa?	Chưa có bản đồ tương tác đầy đủ; muốn làm thật cần thêm tọa độ vào Accommodation và tích hợp map provider.

J.5 — Checklist mở code trong 10 giây

- Booking: mở BookingService.createBooking để nói validate, lock room, availability, voucher, tính tiền từ DB, tạo booking/payment PENDING_PAYMENT.
- Payment: mở PaymentService và VnpayService để nói verify signature, validate amount, idempotency và trạng thái VNPAY.
- Security: mở SecurityConfig để nói role guard, BCrypt, OAuth optional và CSRF đang tắt cho demo.
- Settlement/wallet: mở SettlementService và PartnerWalletService để nói payout, settlement PAID credit một lần, withdrawal hold/reject/paid.
- AI/chatbot: mở ChatbotService để nói rule-based intent + Groq optional, không claim recommendation AI production.
- i18n: mở InternationalizationConfig và messages_vi/en để nói đa ngôn ngữ cơ bản, chưa dịch toàn bộ dữ liệu DB.

J.6 — Đoạn trả lời liên hoàn cần thuộc

TravelMate có tính thực tế ở mức mô phỏng OTA-mini: User đặt phòng, Partner quản lý cơ sở/phòng, Admin kiểm duyệt, VNPAY Sandbox, review và quyết toán nội bộ. Tuy nhiên nhóm không khẳng định hệ thống đã đạt mức production như Booking hay Agoda. Nếu triển khai thật cần bổ sung payment/refund production, đối soát, fraud detection, channel manager, dynamic pricing, mobile API, CSRF đầy đủ, load test, monitoring, dữ liệu thật và quy trình vận hành. Trong phạm vi đồ án cơ sở, nhóm tập trung hoàn thiện nghiệp vụ chính, demo ổn định và hiểu được source khi giảng viên yêu cầu mở code.

Phần K — Rà soát ChatGPT Plus: tình huống bị bắt lỗi cần học

Mục này chốt lại phần ChatGPT Plus gợi ý về các tình huống bị giảng viên bắt lỗi. Kết luận: phần lớn nhận định đúng hướng, nhưng khi bảo vệ phải phân biệt rõ chức năng đã có trong source với giới hạn production chưa triển khai.

K.1. Kết luận kiểm chứng nhanh

- Đã kiểm chứng bằng source và test: route /admin/** theo role ADMIN; /partner/** theo role PARTNER; booking/payment có guard trạng thái; settlement/wallet có guard chống cộng trùng; direct booking/manual block không vào revenue online.
- Đã chạy lại full test suite trong phiên rà soát này: 279 tests pass, 0 failure, 0 error, 0 skipped. Trước ngày bảo vệ, nhóm nên chạy lại trên máy demo hoặc chuẩn bị ảnh BUILD SUCCESS.
- travelmate_db.sql gốc đã đủ dữ liệu demo cho User/Partner/Admin, cọc 30%, no-show, pending partner confirmation, direct booking, manual block, settlement, wallet, withdrawal và voucher ADMIN/PARTNER chịu phí.
- Không nên tự nhận production-ready. Các điểm như CSRF, IPN public, refund thật, bank payout thật, maker-checker, load test và secret manager là hướng nâng cấp production.

K.2. Bảng phân loại tình huống bị hỏi xoắn

Tình huống	Source hiện tại	Cách trả lời khi bảo vệ	Cần demo?
Hai khách đặt cùng phòng cùng lúc	Có overlap theo ngày và lock ghi Room trong BookingService.	Nói đủ demo/service logic; production cần concurrency/load test và transaction isolation.	Không demo live nếu chưa có script ổn định.
VNPAY callback/return lặp hoặc user refresh	PaymentService chỉ xử lý khi PENDING_PAYMENT; txnRef dùng để tra payment.	Nói đây là idempotency; production cần log/audit/reconciliation đầy đủ hơn.	Có thể trả lời bằng test PaymentServiceTest.
Return success tới muộn sau EXPIRED/CANCELLED	PaymentController kiểm trạng thái cũ và không hiển thị success giả.	Nói hệ thống không downgrade/khôi phục payment đã xử lý sai ngữ cảnh.	Đã có PaymentControllerTest.
Cọc 30% khách không đến	DEPOSIT_30 chuyển NO_SHOW/DEPOSIT_FORFEITED; 70% tại cơ sở không phát sinh.	Nói rõ mất cọc 30%, không cộng 70% vào revenue online.	Có thể mở NoShowDepositTest.
Full payment no-show	Không tự động xử lý như cọc 30%; để Admin/Partner xử lý theo chính sách.	Không nói mất 100% hoặc hoàn tự động nếu source chưa chốt policy.	Không demo live.
Partner không giữ phòng/căn	Booking sau VNPAY success là CONFIRMED + PARTNER_CONFIRMED.	Nói đây là ngoại lệ vận hành; production cần SLA/notification/auto-timeout.	Chỉ trả lời, không cần demo.
Partner xem booking người	Service kiểm owner và	Nói backend guard là điểm chính; UI chỉ hỗ trợ trải	Có thể mở

khác	partnerPropertyType, không chỉ ẩn UI.	nghiệm.	BookingService/PartnerPageController.
Settlement paid cộng ví trùng	PartnerWalletService kiểm transaction SETTLEMENT_CREDIT đã tồn tại.	Nói settlement/wallet là sổ nội bộ, chưa phải chuyển khoản ngân hàng thật.	Có PartnerWalletServiceTest.
Voucher Partner chịu phí	Voucher Partner chịu được trừ khỏi khoản Partner nhận. Hoa hồng tính trên tổng đơn gốc trước voucher cho cả FULL_PAYMENT và DEPOSIT_30; với đơn cọc 30%, voucher Partner chịu tối đa 10% tổng đơn gốc và bị chặn nếu làm khoản nhận từ cọc bị âm.	Voucher ADMIN chịu không trừ payout Partner.	Mở SettlementServiceTest.
Direct booking/manual block	booking_source DIRECT/MANUAL_BLOCK, paymentStatus NOT_REQUIRED.	Chỉ chiếm quota quản lý phòng; không commission, không settlement online.	Seed SQL đã có dữ liệu.
User truy cập /admin	SecurityConfig hasRole("ADMIN") cho /admin/**.	CSRF là vấn đề khác với phân quyền route.	Có thể test thủ công bằng user thường.
Secret VNPAY/Google/SMTP/Groq	application.properties dùng placeholder/env, không nên show secret thật.	Nếu lộ secret production phải xóa khỏi repo và rotate key.	Không mở file chứa secret khi trình chiếu.
IPN localhost	Return URL có thể demo local; IPN server-to-server cần URL public/tunnel.	Không nói IPN thật chắc chắn chạy ở localhost.	Dùng Return flow hoặc booking seed.
Review/hạng sao	User review thang 1-10 sau booking COMPLETED; accommodation rating hiển thị điểm trung bình /10 hoặc "Chưa có đánh giá" nếu chưa có review.	Hạng sao/chất lượng cơ sở không phải review của khách.	Có ReviewService và UI my-bookings.
File upload	Có kiểm size, extension/content-type và tên UUID.	Production cần magic bytes, virus scan, object storage hoặc ngoài webroot.	Chỉ nói kiểm cơ bản.
Forgot password/SMTP	Có token reset; email quên mật khẩu và email xác nhận booking phụ thuộc SMTP cấu hình.	Không nói luôn gửi được mail thật nếu máy demo chưa cấu hình.	Có PasswordResetServiceTest.
CSRF, ddl-auto, Docker/CI/CD/JWT	Không phải lỗi P0 demo nếu đã trả lời đúng phạm vi.	Đưa vào hạn chế/hướng phát triển production.	Không tự nhận đã triển khai nếu source chưa có.

K.3. Câu trả lời liên hoàn nên học thuộc

Dạ, các tình huống thầy/cô nêu đều là rủi ro thực tế khi triển khai production. Trong phạm vi đồ án cơ sở, TravelMate đã xử lý các luồng chính như booking, VNPAY Sandbox, TravelMate giữ phòng/căn, Partner check-in/check-out, review và settlement nội bộ. Với các rủi ro như callback lặp, overbooking đồng thời, refund thật, IPN public, CSRF, secret và payout ngân hàng, nhóm em xử lý ở mức demo bằng phân quyền, kiểm trạng thái, transaction, lock và service guard; nếu triển khai thật cần bổ sung load test, audit log, retry/reconciliation job, secret manager, CSRF đầy đủ, monitoring và quy trình đối soát tài chính.

K.4. Checklist không làm vỡ demo

- Trước khi bảo vệ: import lại travelmate_db.sql, đăng nhập thử admin/user/partner và chạy lại mvn test nếu có thời gian.

- Khi demo: đi luồng chắc gồm tìm kiếm, đặt phòng, VNPAY hoặc booking seed, TravelMate giữ phòng/căn, Partner check-in/check-out, review, Admin settlement/wallet.
- Không demo các case khó kiểm soát như hai user đặt đồng thời, callback trễ, IPN localhost, refund ngân hàng thật, CSRF attack hoặc DB lỗi.
- Nếu bị hỏi production, trả lời trung thực là hướng phát triển, không nói hệ thống đã đạt chuẩn thương mại thật.

Phần L — Gap Analysis: TravelMate so với thực tế

Mục tiêu của phần này là giúp nhóm trả lời đúng khi giảng viên hỏi: “Nếu triển khai thật thì TravelMate còn thiếu gì?”. Không nên trình bày theo hướng project yếu; nên nói TravelMate đã đủ cho phạm vi đề án cơ sở, còn các thành phần production được nhận diện rõ để đưa vào hướng phát triển.

L.1 Cách nói an toàn khi bảo vệ

TravelMate đã đáp ứng các nghiệp vụ chính trong phạm vi đề án cơ sở: tìm kiếm nơi lưu trú, đặt phòng, thanh toán VNPAY Sandbox, workflow User/Partner/Admin, review và settlement nội bộ. Nếu triển khai thành sản phẩm OTA thương mại, hệ thống cần bổ sung channel manager, refund gateway thật, bank payout, monitoring, backup, rate limit, CSRF đầy đủ và load test.

L.2 Bảng gap cần học

Mảng	TravelMate hiện có	Gap và cách trả lời
Real-time availability	Overlap theo ngày, lock room, direct booking và manual block.	Chưa có channel manager. Trả lời: TravelMate quản lý quota mở bán trên nền tảng, chưa đồng bộ toàn bộ inventory vật lý như OTA thật.
Payment idempotency	`vnp_txn_ref` unique, verify signature/amount, chỉ update khi payment còn PENDING_PAYMENT.	Chưa có dashboard đối soát/alert production. Trả lời: đủ demo sandbox; production cần callback log, retry và reconciliation.
Refund	Có REFUND_PENDING và REFUNDED để Admin ghi nhận.	Chưa gọi API hoàn tiền thật. Trả lời: refund trong đề án là trạng thái nghiệp vụ, không phải chuyển tiền ngân hàng.
Cancellation policy	Có hủy booking, mất cọc 30%, full payment sang refund pending.	Chưa có policy matrix theo listing/thời gian/mùa. Trả lời: xử lý case chính; production cần bảng cancellation_policy.
No-show	Có NO_SHOW và DEPOSIT_FORFEITED cho cọc 30%.	Full payment no-show chưa sâu. Trả lời: nhóm ưu tiên luồng cọc 30%, phần còn lại là hướng mở rộng policy.
Commission	Có commission theo property type/room override và snapshot. Booking online tính hoa hồng theo tổng đơn gốc trước voucher cho cả FULL_PAYMENT và DEPOSIT_30.	Chưa có hợp đồng phức tạp theo mùa/partner/campaign. Trả lời: đủ cho đề án, production cần commission contract.
Settlement & wallet	Có settlement tháng, payout nội bộ, wallet và withdrawal.	Chưa bank transfer tự động, maker-checker, bank reconciliation. Trả lời: đây là sổ nội bộ, không phải ví ngân hàng thật.
Partner payout	Có bank info và request rút tiền nội bộ.	Chưa KYC/bank verification/payout processor. Trả lời:

		Admin xử lý chi trả ngoài hệ thống rồi cập nhật trạng thái.
Fraud & rate limit	Có verify payment flow và log cơ bản.	Chưa có rate limit, risk scoring, anomaly detection. Trả lời: production cần bổ sung, không phải lỗi chặn demo.
Security	Có RBAC, BCrypt, reset token, owner/service guard, upload validation.	CSRF đang tắt, headers/rate limit/secret manager chưa production-grade. Trả lời: demo config phù hợp local; production phải hardening.
Audit log	Có AdminActionLog và raw payload payment.	Chưa chắc mọi thao tác tài chính/Admin đều log đủ. Trả lời: audit ở mức đồ án; production cần mở rộng toàn diện.
Notification/email	Có notification entity/API, SMTP optional; hệ thống gửi email xác nhận booking sau VNPAY success và mail quên mật khẩu nếu SMTP cấu hình.	Chưa realtime push/SMS/email worker retry; lỗi gửi mail không rollback booking. Trả lời: phục vụ demo; production cần worker và retry.
File upload	Có extension/content-type/size và UUID filename.	Chưa magic-byte scan, virus scan, object storage. Trả lời: đủ demo; production cần kiểm file sâu hơn.
Backup/import DB	Có seed SQL demo đầy đủ tình huống.	Chưa backup automation/migration. Trả lời: trước demo phải test import SQL; production cần Flyway/Liquibase và backup job.
Scalability	Monolithic Spring Boot MVC phù hợp đồ án.	Chưa queue/cache/CDN/microservice/load test. Trả lời: monolith hợp lý cho đồ án cơ sở; scale là hướng phát triển sau.

L.3 Những gap không nên cố fix sát giờ

- Channel manager/OTA sync: phức tạp, cần tích hợp ngoài và thay đổi availability lớn.
- Payment/refund production và bank payout tự động: cần tài khoản merchant/ngân hàng thật, vượt phạm vi đồ án cơ sở.
- Fraud scoring, microservices, dynamic pricing, WebSocket realtime: nên để hướng phát triển.
- Bật CSRF toàn bộ sát giờ: nếu chưa test kỹ mọi form/payment callback thì dễ làm vỡ demo.

L.4 Checklist phải test thay vì mở rộng production

- User thường không vào được /admin và /partner.
- Partner không vào được /admin; Admin không vào workspace /partner.
- Partner chỉ xem/xử lý booking thuộc cơ sở của mình; User không hủy/review booking người khác.
- VNPAY callback/refresh lặp không cập nhật trùng.
- Cọc 30% không cộng 70% vào revenue TravelMate; direct booking/manual block không vào settlement.
- Settlement PAID không cộng ví trùng; withdrawal REJECTED hoàn lại available balance.
- Import travelmate_db.sql chạy được trên máy demo; không trình bày sandbox/refund/wallet như giao dịch ngân hàng thật.

L.5 Câu trả lời phòng thủ

Nếu thầy hỏi: “Project thiếu nhiều production gap vậy có phải chưa hoàn thiện không?”

Trả lời: Dạ không. Nhóm em xác định phạm vi là đồ án cơ sở, nên tập trung hoàn thiện các nghiệp vụ chính: User đặt phòng, thanh toán sandbox, Partner xử lý booking, Admin quản lý/đối soát, review và settlement nội bộ. Các gap như channel manager, refund gateway, KYC, fraud scoring, CI/CD, monitoring và load test là yêu

cầu của sản phẩm thương mại thật, nên nhóm đưa vào hạn chế và hướng phát triển thay vì nói quá là đã production-ready.

Phần M — Test plan hoàn chỉnh và ưu tiên fix trước bảo vệ

Phần này dùng để học và trình bày kiểm thử. Test plan không phải danh sách tính năng mới; đây là checklist ổn định để chứng minh TravelMate chạy được các luồng chính, có guard backend, và biết rõ giới hạn production.

M.1 Cách hiểu đúng về test plan

Tầng test	Mục tiêu	Ví dụ TravelMate
Unit/service	Kiểm tra công thức và rule nghiệp vụ ở backend.	Booking, payment, voucher, settlement, wallet.
Integration/controller	Kiểm tra route, phân quyền và luồng nhiều lớp.	Admin/Partner/User access guard, VNPAY return.
Static/template	Bắt lỗi UI template có thể gây lệch demo hoặc XSS.	Review/support dùng th:text, không render raw HTML.
Manual smoke	Chạy nhanh trên browser trước bảo vệ.	Login 3 role, đặt phòng, VNPAY Sandbox, Partner xử lý, Admin settlement.

M.2 Những test đã bổ sung theo gợi ý I/J

- Refund sai trạng thái: Admin chỉ được đánh dấu đã hoàn tiền khi booking đang ở REFUND_PENDING; booking CONFIRMED/APPROVED bị chặn và không lưu nhầm.
- XSS static check: nội dung người dùng ở review/support dùng th:text để escape HTML; các template này không dùng th:utext cho dữ liệu người dùng.
- Route protection đã bổ sung ở phần Gap Analysis: Partner không vào /admin/settlements; Admin không đi vào workspace /partner/wallet.
- Tổng test tự động hiện tại sau khi bổ sung: 279/279 pass, 0 fail, 0 error, 0 skipped.

M.3 Ưu tiên P0/P1/P2 trước bảo vệ

Mức	Nên làm	Không nên làm sát giờ
P0	Import DB, chạy app, login Admin/User/Partner, đặt phòng, VNPAY Sandbox, Partner confirm/check-in/check-out, Admin settlement/wallet.	Không đổi UI, schema hoặc business flow lớn nếu đã chạy ổn.
P1	Rà các case nhỏ: route sai role, refresh callback, voucher/date invalid, upload sai file, forgot password/OAuth khi thiếu cấu hình.	Không cố bật toàn bộ production security nếu chưa test hết form và callback.
P2	Ghi nhận hướng phát triển: concurrency/load test, refund ngân hàng thật, IPN public, bank payout automation, maker-checker.	Không tự nhận production-ready hoặc demo giao dịch ngân hàng thật.

M.4 10 test case nên đưa vào báo cáo

- Login đúng role: Admin, Partner, User vào đúng workspace.
- User đặt phòng và backend tự tính lại giá từ DB, không tin giá từ client.
- VNPAY Sandbox success cập nhật payment/booking đúng một lần.
- Partner chỉ xem/xử lý booking thuộc cơ sở của mình.
- TravelMate giữ phòng/căn sau thanh toán hợp lệ; Partner check-in, check-out đúng trạng thái.

6. Cọc 30%: hoa hồng tính theo tổng đơn gốc, payout chỉ lấy phần TravelMate thu online; no-show giữ cọc theo rule demo.
7. Voucher Admin chịu không trừ payout Partner; voucher Partner chịu có trừ.
8. Settlement PAID không cộng ví trùng; withdrawal reject hoàn lại số dư.
9. Review/support escape HTML để tránh XSS nội dung người dùng.
10. Admin refund chỉ cho booking REFUND_PENDING, sai trạng thái bị chặn.

M.5 Checklist nhanh trước buổi bảo vệ

- Trước 24 giờ: import lại travelmate_db.sql, chạy full test, chuẩn bị ảnh BUILD SUCCESS và kiểm tra tài khoản demo.
- Trước 2 giờ: mở browser test login 3 role, đặt phòng/VNPAY hoặc booking seed, Partner xử lý booking, Admin settlement/wallet.
- Trước 15 phút: không chỉnh code lớn, không đổi DB, không trình chiếu secret, chỉ kiểm tra app còn chạy và dữ liệu demo còn đúng.

M.6 Câu trả lời khi thầy hỏi “test đủ chưa?”

Dạ, trong phạm vi đồ án cơ sở, nhóm em đã có test tự động cho các nghiệp vụ trọng yếu như booking, payment, voucher, settlement, wallet, review, phân quyền route và template an toàn. Kết quả hiện tại là 279/279 pass. Tuy nhiên nhóm em không nói hệ thống đã đủ chuẩn production; các phần như load test, refund ngân hàng thật, IPN public, CSRF production và bank payout automation được ghi là hướng phát triển nếu triển khai thực tế.

Phần N — Script trả lời báo cáo và kết quả smoke test

Phần này dùng để nhóm học cách trình bày miệng. Nội dung dưới đây đã được chỉnh theo source hiện tại: không dùng số 247 test cũ; số đúng sau khi chạy lại là 279/279 pass.

N.1 Kết luận kiểm chứng

- Gọi ý ChatGPT Plus đúng hướng vì chia bài nói theo các phần: đề tài, kiến trúc, database, booking/payment, partner/admin/settlement, testing và hướng phát triển.
- Điểm cần sửa là phần Testing/exception phải nói 279/279 pass, không nói 247 test pass.
- Trong session này đã chạy lại full Maven test: 279/279 pass, 0 fail, 0 error, 0 skipped.
- Đã chạy smoke test app local qua HTTP/session: 23/23 checks pass cho login 3 role, route guard và các trang chính User/Partner/Admin.
- Chrome headless đã re-test được luồng User đặt lại đơn đã hủy/no-show ngày 2026-06-03 trên desktop và mobile: login, bấm/tap đặt lại, sang form booking có prefill và banner đơn mới độc lập. Trước bảo vệ vẫn cần mở browser thật trên máy demo để chạy lại các flow lớn như VNPAY, Partner và Admin.

N.2 Script trình bày ngắn

Phần trình bày	Script nên nói
Giới thiệu đề tài	Kính thưa thầy/cô, nhóm em thực hiện TravelMate - website đặt phòng và gọi ý du lịch. Hệ thống mô phỏng nền tảng trung gian giữa khách hàng, đối tác lưu trú và quản trị viên. Phạm vi là đồ án cơ sở chạy local/demo, chưa phải production thật.
Kiến trúc	TravelMate dùng Spring Boot MVC, Thymeleaf và MySQL. Source tách theo Controller - Service - Repository - Entity. Cách tổ chức này phù hợp đồ án vì dễ demo nhưng vẫn thể hiện phân tầng phần mềm.

Database	Database xoay quanh users, accommodations, rooms, bookings, payments, vouchers và reviews; ngoài ra có settlement, wallet, withdrawal, support, notification và travel posts. Booking lưu trạng thái nghiệp vụ, payment lưu giao dịch VNPAY.
Booking/payment	Backend không tin dữ liệu client mà kiểm lại ngày, sức chứa, availability, voucher và giá từ DB. VNPAY success được verify chữ ký/amount rồi cập nhật payment APPROVED, booking CONFIRMED. Booking online tách tiền TravelMate thu online với tiền tại cơ sở; hoa hồng luôn dựa trên tổng đơn gốc trước voucher.
Partner/Admin/settlement	Partner xử lý booking thuộc cơ sở của mình: confirm hold, check-in, check-out. Admin giám sát toàn hệ thống, quản lý booking/voucher/review/support và settlement. Vì partner là sở quyết toán nội bộ, không phải ví ngân hàng thật.
Testing/exception	Nhóm có 279/279 automated tests pass. Trong session này smoke test app local 23/23 checks pass cho login 3 role, User/Partner/Admin pages và cross-role guard. Nhóm vẫn nói trung thực là chưa đủ chuẩn production vì chưa có load/security/payment production test.
Kết luận	TravelMate đã đáp ứng mục tiêu đồ án cơ sở: phân quyền, booking/payment, partner/admin, review, voucher, chatbot và quyết toán nội bộ. Hướng phát triển là deploy cloud, bật hardening security, migration DB, rate limiting, refund API, payout thật, map/location và mobile app.

N.3 Kết quả test trong session này

Nhóm kiểm tra	Kết quả	Ghi chú
Full Maven test	PASS 279/279	0 fail, 0 error, 0 skipped.
App startup local	PASS	App start tại http://127.0.0.1:18080 và DataInitializer hoàn tất.
HTTP/session smoke	PASS 23/23	Public pages, login User/Partner/Admin, route guard và các trang chính đều pass.
Chrome headless rebook	PASS desktop/mobile	Ngày 2026-06-03: login User, bấm/tap đặt lại đơn đã hủy/no-show, redirect sang form booking có prefill và banner đơn mới độc lập.

N.4 Câu trả lời khi thầy hỏi đã test browser chưa

Dạ, nhóm em đã chạy smoke test trên app local cho các luồng chính và login đủ 3 role. Riêng luồng User đặt lại đơn đã hủy/no-show đã được re-test bằng Chrome headless ngày 2026-06-03 trên desktop và mobile: bấm/tap nút đặt lại, sang form booking có prefill và banner đơn mới độc lập. Trước buổi bảo vệ, nhóm vẫn sẽ mở browser thật trên máy demo để chạy lại flow VNPAY, Partner xử lý booking và Admin xem settlement/wallet.

N.5 Flow nên demo

- Nên demo chắc: trang chủ, tìm kiếm nơi lưu trú, User my-bookings/form booking, Partner dashboard/bookings/wallet/settlements/support, Admin dashboard/bookings/settlements/withdrawals/reviews/support.
- Chỉ demo VNPAY Sandbox thật nếu đã có sandbox secret và đã smoke test trên máy bảo vệ.
- Nên tránh nếu chưa kiểm kỹ: Google OAuth thật, SMTP thật, IPN server-to-server localhost, concurrent booking thật, refund production và payout ngân hàng thật.

Phần 13 — Cập nhật sau fix commission, voucher và UI/UX

Mục tiêu học lại: nắm đúng các điểm vừa đồng bộ để khi demo giảng viên thấy số tiền, trạng thái và trải nghiệm người dùng khớp giữa User, Partner, Admin và dữ liệu SQL gốc.

13.1 Những điểm ChatGPT Plus gợi ý đúng

- Cần thống nhất doanh thu/settlement cho đơn cọc 30%: phần TravelMate thu online khác với phần khách trả tại cơ sở.
- Voucher do Partner chịu phải ảnh hưởng quyết toán Partner; voucher do TravelMate/Admin chịu không trừ tiền Partner.
- Kho voucher nên cho khách nhìn được voucher toàn hệ thống và voucher theo phòng/căn đã được Partner gán, thay vì chỉ hiện một phần dữ liệu.
- Trạng thái sau thanh toán nên nói là TravelMate đã giữ phòng/căn, không nói như thể Partner duyệt lại thanh toán.
- Tiện nghi chung của LATA Hotel cần tránh làm người dùng hiểu bếp/gác lửng có ở mọi phòng.

13.2 Nghiệp vụ đã fix cần học lại

- Đơn cọc 30%: hoa hồng nền tảng dùng tổng đơn gốc làm cơ sở snapshot, nhưng quyết toán Partner chỉ lấy từ phần tiền online TravelMate đã thu; 70% tại cơ sở không đi vào ví TravelMate.
- No-show của đơn cọc: khách không đến thì cọc bị giữ, phòng/căn được mở lại sau xử lý; màn Admin/Partner hiển thị rõ tiền giữ cọc và phần quyết toán từ khoản TravelMate đã thu.
- Voucher Partner chịu: hệ thống chặn voucher làm quyết toán Partner bị âm. Với cọc 30%, voucher Partner vượt ngưỡng an toàn sẽ được gợi ý chọn thanh toán 100% hoặc voucher nhỏ hơn.
- Kho voucher /vouchers: khách bấm “Tìm nơi áp dụng” sẽ giữ mã voucher qua danh sách nơi lưu trú, trang chi tiết và màn booking; màn booking tự điền mã và kiểm tra hợp lệ theo phòng/căn đang chọn.
- Trạng thái hiển thị: dùng “TravelMate đã giữ phòng/căn”, “Khách đang lưu trú”, “Hoàn tất lưu trú” để người xem hiểu đúng bước nghiệp vụ.
- travelmate_db.sql gốc đã đồng bộ lại ghi chú no-show và khối chuẩn hóa snapshot, nên khi import máy khác số liệu không quay về logic cũ.

13.3 Cách demo trực quan cho giảng viên

- Mở kho voucher, chọn một voucher đang hoạt động rồi đi tiếp tới danh sách nơi lưu trú. Nhấn vào nơi lưu trú, chọn phòng/căn và quan sát mã voucher được điền sẵn ở màn đặt phòng.
- Ở màn booking, thử chọn cọc 30% với voucher Partner giảm sâu để thấy cảnh báo thân thiện; sau đó chọn thanh toán 100% hoặc voucher phù hợp hơn.
- Ở Admin/Partner booking detail, mở booking no-show cọc 30% để giải thích: khách mất cọc, TravelMate chỉ quyết toán trên phần tiền đã thu online, 70% tại cơ sở không phát sinh.
- Ở Partner bookings, chỉ ra nhãn còn thu/đã thu tại cơ sở cho đơn cọc 30% để chứng minh luồng thực tế hơn.
- Ở LATA Hotel detail, giải thích tiện nghi chung là khu lounge/khu ăn uống/khu sinh hoạt; bếp hoặc gác lửng nếu có là đặc điểm theo từng hạng phòng.

13.4 Câu trả lời ngắn gọn

- **Nếu bị hỏi vì sao cọc 30% không tính như đã thu 100%?** Vì TravelMate chỉ thu 30% online qua VNPAY; 70% còn lại khách trả trực tiếp tại cơ sở, nên không đưa vào doanh thu online hay ví TravelMate.

- **Nếu bị hỏi voucher Partner khác voucher TravelMate thế nào?** Voucher TravelMate/Admin chịu là chi phí nền tảng; voucher Partner chịu được trừ vào phần quyết toán Partner và phải được backend kiểm tra để không làm âm payout.
- **Nếu bị hỏi Partner thao tác gì sau khi khách trả tiền?** Partner không duyệt thanh toán. TravelMate đã giữ phòng/căn trên hệ thống sau khi thanh toán hợp lệ; Partner tiếp nhận và xử lý lưu trú.
- **Nếu bị hỏi vì sao phải cập nhật SQL gốc?** Vì nhóm có thể import DB trên máy khác; rule tài chính và dữ liệu seed phải khớp với code để demo không lệch số.

Phần 14 — Rà soát checklist bad-ending và fix tiếp

Mục tiêu học lại: phân biệt phần gợi ý ChatGPT Plus đúng hướng, phần cần chỉnh cho hợp ngữ cảnh đồ án, và các rule vừa bổ sung để UI/UX vận hành chặt chẽ hơn nhưng không làm vỡ project.

14.1 Kết luận rà soát gợi ý ChatGPT Plus

- Đúng hướng: cần test các case xấu như voucher vượt ngưỡng, no-show, trạng thái thanh toán, settlement và phân quyền role.
- Đúng hướng: cọc 30% phải tách rõ tiền TravelMate thu online và 70% khách trả tại cơ sở; không cộng 70% vào ví/quyết toán TravelMate.
- Cần chỉnh theo project: không bê mã PV/PF kiểu test vào SQL gốc. SQL đang dùng mã nghiệp vụ thật hơn như HOTEL10, WEEKEND15, ROOM100K, STAY150K, STAY200K và EXPIRED2024.
- Cần chỉnh theo UX: thao tác check-in hoặc báo no-show chỉ hợp lý khi đã tới ngày nhận phòng/căn; trước ngày đó UI nên hiển thị trạng thái chờ thay vì để Partner bấm nhầm.

14.2 Fix bổ sung trong lượt này

- BookingService chặn Partner check-in sớm, Partner báo no-show sớm và Admin đánh dấu no-show sớm nếu check-in còn ở tương lai.
- Trang Partner bookings và booking-detail có trạng thái “mở thao tác từ ngày nhận”, ấn nút check-in/no-show khi chưa tới ngày nhận phòng/căn.
- travelmate_db.sql chỉnh lại ghi chú booking vận hành BK-ANM-GDN-OPS01: đây là lịch no-show tiềm năng, chỉ báo no-show khi đã tới ngày nhận căn và khách không đến.
- Test tự động mới bảo vệ rule thời gian và UI guard. Full Maven test mới nhất: 279/279 pass, 0 fail, 0 error, 0 skipped.

14.3 Ma trận voucher nên demo bằng dữ liệu thật

Nhóm case	Mã nên dùng	Điều cần quan sát	Câu giải thích ngắn
Voucher Partner an toàn cho cọc	HOTEL10, EARLY10, STAY10	Cọc 30% vẫn có thể chọn nếu payout không âm.	Voucher Partner chịu được backend kiểm tra trước khi cho dùng.
Voucher Partner giảm sâu	WEEKEND15, FAMILY15	Có thể bị khuyến chọn thanh toán 100% hoặc voucher nhỏ hơn nếu cọc không đủ đối soát.	Không để Partner payout âm.
Voucher cố định	ROOM100K, STAY150K, STAY200K	Giảm tiền rõ ràng trên tổng đơn và quyết toán.	Dễ demo vì con số trực quan.
Voucher hết hạn/tắt	EXPIRED2024, INACTIVE01	Không hiện như voucher đang dùng và không áp	Dùng để trả lời case hết hạn hoặc tạm ngưng.

		dùng được.	
--	--	------------	--

14.4 Cách demo rule mới cho giảng viên

11. Đăng nhập Partner, mở danh sách đơn đặt phòng. Với đơn có ngày nhận trong tương lai, chỉ ra nhãn “mở thao tác từ ngày ...”, không có nút check-in/no-show sớm.
12. Mở chi tiết BK-ANM-GDN-OPS01 hoặc đơn tương tự trong SQL seed để giải thích: hệ thống đã giữ căn, nhưng chỉ báo no-show khi tới ngày nhận và khách không đến.
13. Mở một đơn check-in hôm nay để cho thấy nút check-in vẫn hoạt động đúng lúc; nếu là cọc 30%, Partner phải xác nhận đã thu phần còn lại tại cơ sở.
14. Kết nối với phần settlement: no-show cọc 30% giữ cọc, mở lại quota, hoa hồng tính trên tổng đơn gốc nhưng payout chỉ từ tiền TravelMate đã thu online.

14.5 Câu trả lời ngắn nên nhớ

- Vì sao không dùng mã PV/PF trong DB? Vì đó giống mã test kỹ thuật; SQL gốc nên dùng dữ liệu vận hành có tên, mô tả và phạm vi rõ ràng.
- Vì sao chặn no-show trước ngày nhận? Vì “khách không đến” chỉ kết luận được khi đã tới lịch nhận phòng/căn; chặn sớm giúp Partner không thao tác sai.
- Nếu UI chỉ hiện trạng thái chờ thì backend còn cần chặn không? Có. UI giúp trải nghiệm tốt hơn, backend là lớp bảo vệ thật nếu người dùng gọi tay endpoint.
- Con số test hiện tại là bao nhiêu? Lần chạy mới nhất trong workspace này là 279/279 pass, 0 fail/error/skipped; trước buổi bảo vệ nên chạy lại trên máy demo.

Phần 15 — Cập nhật luồng đặt lại đơn đã hủy/no-show

Mục tiêu học lại: hiểu phần gợi ý ChatGPT Plus nào đúng, phần nào đã được chỉnh theo project hiện tại, và cách demo trực quan cho giảng viên mà không làm người dùng hiểu nhầm đơn cũ được nối thanh toán sang đơn mới.

15.1 Kết luận rà soát gợi ý ChatGPT Plus

- Đúng hướng: đơn CANCELLED hoặc NO_SHOW nên có nút đặt lại rõ ràng, không chỉ đưa khách quay về trang chi tiết nơi lưu trú.
- Đúng hướng: đặt lại phải tạo một đơn mới độc lập; voucher, thanh toán, hoàn tiền hoặc khoản cọc của đơn cũ không chuyển sang đơn mới.
- Đúng hướng: phải có guard quyền sở hữu để user không đặt lại đơn của người khác, đồng thời phải chặn nếu phòng/căn hoặc nơi lưu trú đã ngừng mở bán.
- Cần chỉnh theo source hiện tại: các phần revenue, settlement, voucher 10%, email booking success và VNPAY idempotency đã có code/test bảo vệ, nên không sửa chồng để tránh làm vỡ project.

15.2 Fix bổ sung trong lượt này

- BookingService có luồng chuẩn bị đặt lại: chỉ nhận đơn CANCELLED/NO_SHOW, kiểm tra đúng chủ đơn, kiểm tra phòng/căn và nơi lưu trú còn được duyệt/mở bán.
- Nếu ngày cũ đã qua hoặc không hợp lệ, hệ thống tự đổi sang ngày mai và ngày kia để khách thấy form đặt phòng dùng được ngay.
- Form đặt phòng được prefill phòng/căn, ngày, số người lớn, trẻ em và số lượng; mã voucher cũ không được truyền sang form.

- Trang “Đặt chỗ của tôi” đổi nút thành “Đặt lại phòng này/Đặt lại căn này” hoặc “Đặt phòng mới/Đặt căn mới” với đơn đang chờ hoàn tiền.
- Trang booking hiển thị banner xanh nhắc đây là đơn mới độc lập, giúp khách không nhầm với cọc/hoàn tiền/voucher của đơn cũ.
- Test tự động mới bảo vệ các case: đặt lại đơn quá khứ đổi ngày an toàn, chặn user khác đặt lại, chặn phòng/căn đã ngừng mở bán. Full Maven test mới nhất: 279/279 pass, 0 fail, 0 error, 0 skipped.

15.3 Cách demo cho giảng viên

15. Đăng nhập User, mở “Đặt chỗ của tôi”, chọn tab Đã hủy hoặc No-show.
16. Chỉ vào nút “Đặt lại phòng này” hoặc “Đặt lại căn này” và đọc ghi chú nhỏ bên dưới: đơn mới dùng lại thông tin phù hợp nhưng thanh toán/voucher/hoàn tiền cũ không chuyển sang.
17. Bấm nút đặt lại để mở form booking; chỉ ra ngày, số khách và số lượng đã được điền lại. Nếu ngày cũ đã qua, form tự dùng ngày sắp tới để người dùng không bị kẹt.
18. Không cần thêm DB mới vì travelmate_db.sql đã có dữ liệu CANCELLED, NO_SHOW, REFUND_PENDING và DEPOSIT_FORFEITED đủ để demo luồng này.

15.4 Câu trả lời ngắn nên nhớ

- Vì sao đặt lại không chuyển cọc cũ? Vì đó là đơn mới độc lập; cọc/hoàn tiền của đơn cũ là nghiệp vụ riêng đã được ghi nhận theo trạng thái cũ.
- Vì sao không tự dùng lại voucher cũ? Vì voucher có thời hạn, phạm vi áp dụng và điều kiện theo phòng/căn tại thời điểm mới, nên phải kiểm tra lại.
- Nếu phòng/căn cũ đã ngừng mở bán thì sao? Backend chặn và thông báo để khách chọn phòng/căn khác, tránh tạo đơn không thể phục vụ.
- Con số test hiện tại là bao nhiêu? Lần chạy mới nhất trong workspace này là 279/279 pass, 0 fail/error/skipped; trước buổi bảo vệ nên chạy lại trên máy demo.

15.5 Browser test và responsive UX sau khi fix

Sau khi bổ sung luồng đặt lại, nhóm đã chạy lại app local tại <http://127.0.0.1:18085> bằng Chrome headless. Tài khoản user@travelmate.vn đăng nhập thành công, trang “Đặt chỗ của tôi” hiển thị 5 nút đặt lại từ dữ liệu thật trong DB.

Desktop: bấm /my-bookings/95/rebook chuyển sang form /booking với roomId, checkIn, checkOut, adults, children và rooms được prefill; banner xanh nhắc rõ đây là đơn mới độc lập, không chuyển cọc/voucher/thanh toán cũ.

Mobile: sau khi kiểm ảnh, đã tinh chỉnh responsive để card đặt lại dùng một cột, nút đặt lại full-width, ghi chú không bị chồng và form booking sau khi đặt lại xếp một cột thay vì bị bóp hai cột. Đây là điểm nên demo nếu giảng viên hỏi về trải nghiệm người dùng.

Ảnh kiểm tra lưu trong workspace: C:\TravelMate\browser-desktop-rebook-booking-form.png và C:\TravelMate\browser-mobile-rebook-booking-form.png.

Phần 16 — Cập nhật ngày 04/06/2026: quy tắc tài chính và UX sau rà soát

- Các nhận định của ChatGPT Plus đúng ở hướng ưu tiên, nhưng source hiện tại đã có sẵn nhiều phần: email xác nhận sau VNPAY success, đặt lại đơn hủy/no-show, rating theo review thay vì nhập hạng sao, và guard voucher Partner cho cọc 30%.

- Phần mới được chuẩn hóa là tài chính: hoa hồng của booking online tính theo tổng đơn gốc trước voucher cho cả thanh toán 100% và cọc 30%; dòng tiền quyết toán vẫn xuất phát từ phần TravelMate đã thu online.
- Voucher Partner chịu được trừ vào khoản Partner nhận; voucher Admin chịu là chi phí nền tảng nên không trừ Partner. Với cọc 30%, voucher Partner vượt 10% tổng đơn gốc hoặc làm khoản nhận từ cọc bị âm sẽ bị chặn.
- travelmate_db.sql đã được đồng bộ để khi import trên máy khác vẫn có booking, voucher, settlement, ticket và snapshot tài chính khớp quy tắc mới.
- UI doanh thu/quyết toán đã chuyển sang mô tả nghiệp vụ dễ hiểu, tránh hiện công thức khô cứng trên màn hình demo. Khi bảo vệ, hãy giải thích bằng luồng tiền: TravelMate thu online, khách trả tại cơ sở, hoa hồng nền tảng, voucher và khoản chờ quyết toán.

Phần 17 — Cập nhật ngày 04/06/2026: checklist kiểm thử browser trước bảo vệ

Mục tiêu: phần này giúp nhóm tự kiểm tra lại các luồng có khả năng bị hỏi trong buổi bảo vệ sau khi đã rà soát finance, voucher, review/rating, rebook và quyết toán. Bộ checklist đầy đủ nằm tại docs/20_Defense_Browser_Test_Checklist.md; trong Word chỉ giữ các mục cần học nhanh và bấm trước demo.

Số liệu test tự động hiện tại: 279/279 test pass, 0 fail, 0 error, 0 skipped. Khi bảo vệ chỉ nói “vừa chạy pass” nếu nhóm đã chạy lại trên máy trình bày.

Nhóm kiểm thử	Ưu tiên	Cần bấm lại	Kết quả mong muốn
Smoke/Auth	P0	Start app, login User/Partner/Admin, vào sai role	App chạy ổn; đúng role vào được, sai role bị chặn hoặc access denied.
Voucher cọc 30%	P0	Apply voucher Partner 10%, 11%, 15% khi chọn cọc	10% hợp lệ; 11% và 15% bị chặn trước payment; thông báo dễ hiểu.
Revenue/Settlement	P0	Mở Admin revenue, Admin settlements, Partner revenue/settlements	Hoa hồng dựa trên tổng đơn gốc trước voucher; 70% tại cơ sở tách riêng; không còn wording công thức cũ.
Rating/Review	P0	Mở listing/detail chưa có review; review booking completed	Chưa có review thì hiện “Chưa có đánh giá”; chỉ booking completed mới review được.
Partner vận hành	P0	Confirm, check-in, check-out; riêng cọc 30% xác nhận thu 70%	Trạng thái rõ: giữ phòng, đang lưu trú, đã hoàn tất; cọc có dòng đã thu tại cơ sở.
Rebook	P0	Bấm đặt lại từ đơn cancelled/no-show	Sang booking mới độc lập, có note cọc cũ không chuyển, tạo booking/payment mới.
Direct/manual block	P0	Tạo direct booking hoặc chặn bán thủ công	Chiếm phòng đúng nhưng không vào doanh thu online và quyết toán TravelMate.
Security/ownership	P0	User/Partner đổi URL dữ	Không xem, hủy, review

		liệu người khác	hoặc sửa được dữ liệu không thuộc quyền.
Cancel/refund/no-show	P1	Hủy pending, hủy cọc, refund full, no-show cọc	Trạng thái và wording đúng chính sách; không tạo refund/cancel trùng.
Rủi ro UI	P1	Zoom 100%/125%, reload màn chính, thử XSS review/support	Không vỡ layout, không lộ enum kỹ thuật, script không chạy.

Kịch bản demo nên tập trước

Mã demo	Luồng	Điểm cần nói khi trình bày
Demo A	Cọc 30% + voucher Partner 10%	User đặt phòng, apply voucher, thanh toán VNPAY, Admin revenue đối soát tổng đơn/cọc/70%/commission/payout.
Demo B	Voucher Partner vượt 10% bị chặn với cọc	User chọn voucher 15%, chọn cọc 30%, hệ thống chặn; đổi sang thanh toán 100% nếu đủ điều kiện.
Demo C	Partner vận hành booking	TravelMate giữ phòng sau thanh toán hợp lệ; Partner check-in, xác nhận thu 70% nếu là đơn cọc, check-out; User review.
Demo D	Đặt lại đơn đã hủy/no-show	User bấm “Đặt lại phòng này”, thấy note đơn mới độc lập, chọn ngày mới và tạo booking/payment mới.

Lưu ý khi ghi kết quả: mỗi case nên ghi Status = PASS/FAIL/BLOCKED/NOT TESTED, kèm URL, dữ liệu test và ảnh chụp nếu fail. Với VNPAY Sandbox, SMTP, Google OAuth và import MySQL sạch, cần test lại trên máy trình bày vì phụ thuộc cấu hình ngoài project.

Cập nhật browser QA cuối ngày 04/06/2026

Kết quả kiểm lại trên browser tại <http://localhost:18085>: 16/16 checks pass cho Admin, Partner và User. Admin Quyết toán đã mở sẵn phần nguyên tắc, hiển thị rõ hoa hồng tính theo tổng đơn gốc trước voucher và phần 70% tại cơ sở cho đơn cọc 30%. Partner revenue/settlement, form đăng ký nơi lưu trú không có hạng sao, User voucher/my-bookings/rebook và phân quyền chéo đều đúng output mong muốn.

Ghi chú khi demo: nếu giảng viên hỏi phần quyết toán, nhóm nên mở Admin > Quyết toán để chỉ trực tiếp dòng nguyên tắc và sau đó mở Partner > Quyết toán để đối chiếu cách partner nhìn payout.

Phần 18 — Cập nhật ngày 04/06/2026: TravelMate tự giữ phòng và Admin xem trước trang user

18.1 Kết luận đối chiếu ChatGPT Plus

Gợi ý ChatGPT Plus đúng ở lỗi nghiệp vụ: Partner không xác nhận cọc hoặc xác nhận thanh toán online. VNPAY/TravelMate là bên ghi nhận payment; Partner chỉ check-in/check-out và vận hành check-in/check-out.

Source hiện tại đã có guard phù hợp: booking online chỉ vào Partner sau khi payment được ghi nhận; DEPOSIT_30 khi check-in phải xác nhận đã thu 70% tại cơ sở; FULL_PAYMENT không có nút no-show tự động cho Partner.

Không cần sinh thêm booking mới vì travelmate_db.sql đã đủ case để demo: DEPOSIT_30, FULL_PAYMENT, PARTNER_CONFIRMED, PARTNER_CONFIRMED, NO_SHOW và PAID_AT_PROPERTY. Lướt này chỉ chuẩn hóa ghi chú seed để dữ liệu không còn làm người xem hiểu nhầm là Partner duyệt cọc.

18.2 Fix UX vừa bổ sung

Thêm Admin Preview Bar trên các trang public/user-facing như trang chủ, danh sách nơi lưu trú, chi tiết nơi lưu trú, voucher, travel, news và contact. Bar chỉ hiện với ROLE_ADMIN và có lối quay về Admin, Booking, Voucher.

Admin vẫn có thể xem giao diện người dùng để kiểm tra UI, nhưng không đặt phòng như User. Ở chi tiết nơi lưu trú, nút đặt chuyển thành trạng thái “Đang xem trước” và nhắc dùng tài khoản User nếu muốn đặt thử.

Header, dropdown tài khoản và shortcut ở trang Contact cũng ưu tiên đưa Admin về khu quản trị, tránh click nhầm vào /my-bookings hoặc /profile là các luồng chỉ dành cho USER.

18.3 Cách demo trực quan

19. Đăng nhập Admin, mở trang chủ hoặc trang chi tiết nơi lưu trú, chỉ thanh “Đang xem trang người dùng với vai trò Admin” và nút Quay về Admin.
20. Ở chi tiết nơi lưu trú, chỉ nút “Đang xem trước” để giải thích Admin không đặt phòng bằng role Admin.
21. Đăng nhập Partner, mở danh sách booking và chỉ trạng thái “TravelMate đã giữ phòng/căn”, “Check-in khách”, “Check-out / Hoàn tất”; nhấn mạnh Partner không xác nhận cọc online.
22. Với đơn cọc 30%, mở check-in để chỉ phần xác nhận thu 70% tại cơ sở; với đơn thanh toán 100%, giải thích nếu khách không đến thì liên hệ Admin xử lý theo chính sách.
23. Đăng nhập User hoặc Guest để xác nhận không thấy Admin Preview Bar và luồng đặt phòng của User vẫn hoạt động bình thường.

18.4 Câu trả lời ngắn nên nhớ

- Partner không xác nhận cọc. TravelMate tự giữ phòng/căn sau khi ghi nhận payment hợp lệ; Partner không xác nhận cọc online.
- Với cọc 30%, Partner thu 70% còn lại tại cơ sở khi khách check-in và hệ thống ghi nhận PAID_AT_PROPERTY.
- Admin xem trang User để kiểm tra trải nghiệm, nhưng không đặt phòng bằng role Admin; nếu muốn thử đặt phòng phải dùng tài khoản User.
- travelmate_db.sql đã đủ dữ liệu cho các tình huống này, nên khi chạy máy khác chỉ cần import bản SQL gốc đã cập nhật.